

第八章 自下而上的语法分析

赵银亮 西安交通大学 2025

概述

- 自上而下语法分析是对输入串的一种确定性的最左推导过程
  - 等价于一种确定性的最左扩展语法树的过程（先根遍历语法树）
  - 目标是构建LL(1)分析框架

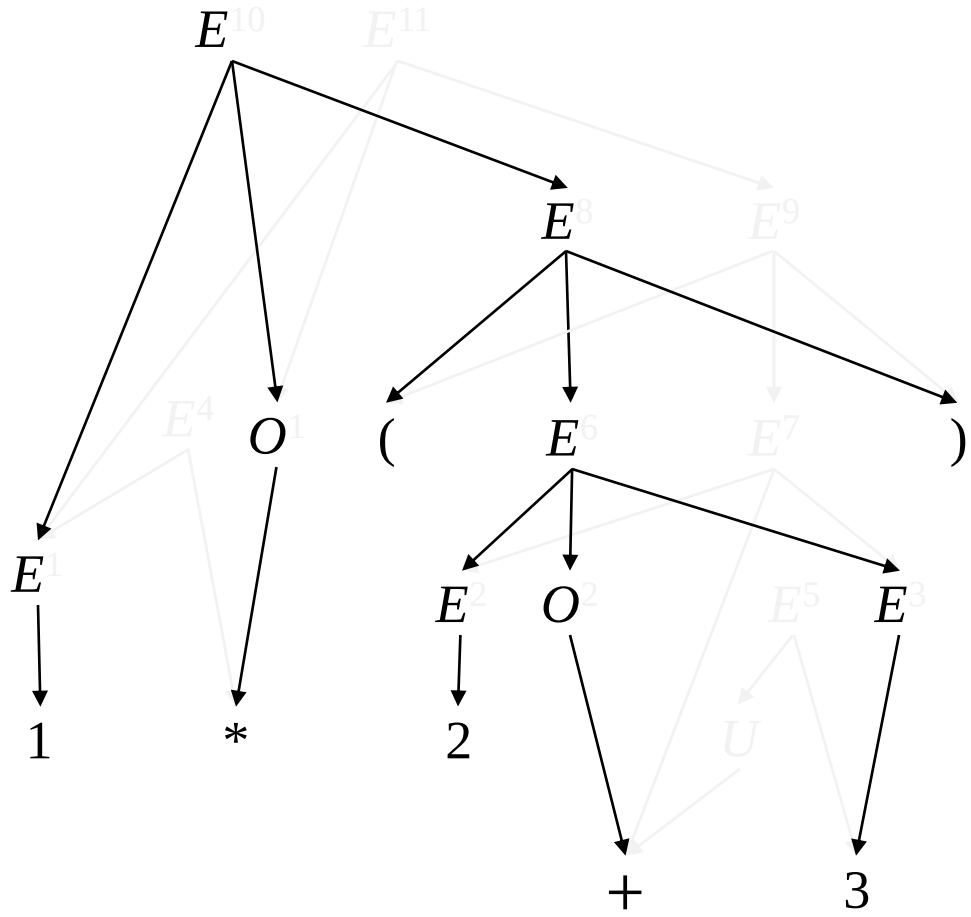
- 自下而上语法分析是输入串的一种确定性的归约过程（称为规范归约，是最右推导的逆过程）
  - 等价于对语法树的句柄剪枝过程
- 自下而上的分析适用更广泛的文法，优点是不需要像LL分析中那么大量的文法修剪即可进行LR分析。也能容忍一些二义性存在，如运算符优先级这种常见情形。
- 本章目标是构建SLR(1)分析框架（规范归约模拟器）

5.1 自下而上分析的基本问题

- 自下而上分析：
  - 从输入串开始连续归约，每个归约步都是将当前符号串中某个候选式的一次出现替换成对应变元。如此连续归约，直到归约成为文法初始符号，表示分析成功。
  - 分析过程中每一步直接归约都与语法树构建一一对应。

- 问题是归约过程存在不确定性。

示例：归约的不确定性



$E \longrightarrow EOE$

$E \longrightarrow E+E$

$E \longrightarrow (E)$

$E \longrightarrow E*$

$E \longrightarrow i$

$O \longrightarrow +$

$O \longrightarrow *$

$U \longrightarrow +$

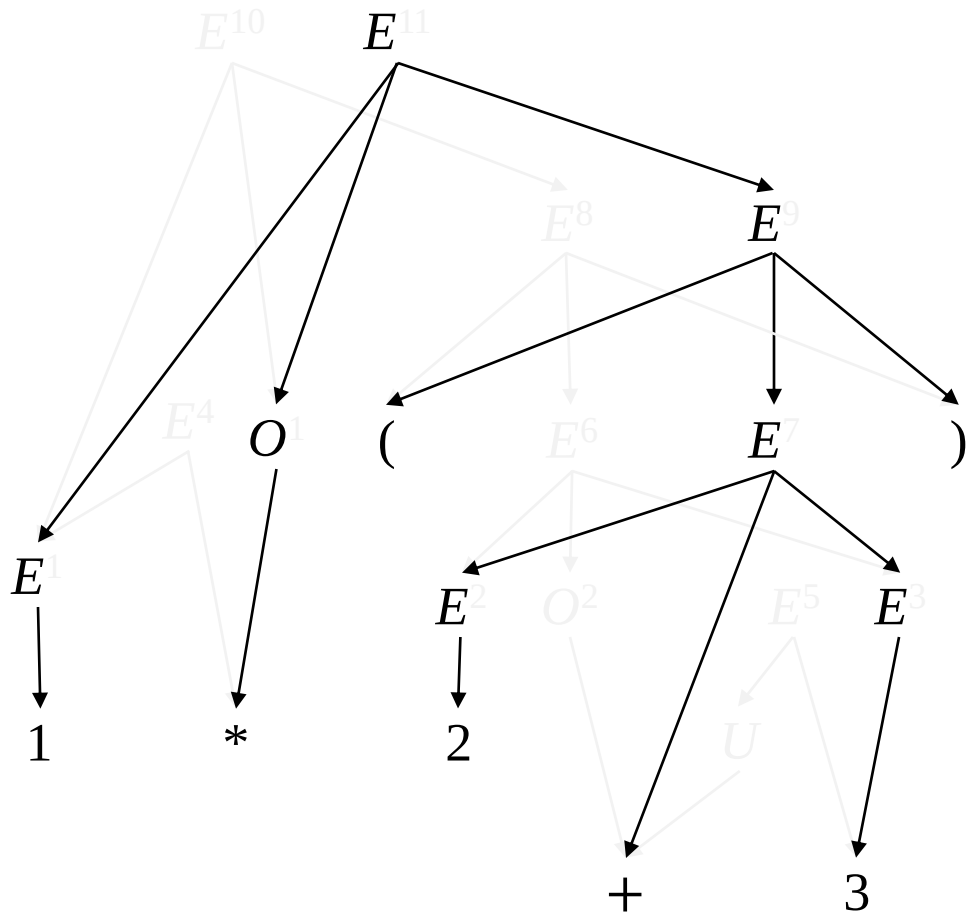
$E \longrightarrow Ui$

$\underline{1}*(2+3) \Leftarrow E\underline{*}(2+3) \Leftarrow EO(\underline{2}+3) \Leftarrow EO(E\underline{+}3) \Leftarrow EO(EO\underline{3}) \Leftarrow EO(\underline{EOE}) \Leftarrow EO(\underline{E}) \Leftarrow \underline{EOE} \Leftarrow E$

...

$1*(2+\underline{3}) \Leftarrow 1*(2\underline{+}E) \Leftarrow 1*(\underline{2}OE) \Leftarrow 1*(\underline{EOE}) \Leftarrow 1*\underline{(E)} \Leftarrow 1\underline{*}E \Leftarrow \underline{1}OE \Leftarrow \underline{EOE} \Leftarrow E$

示例：归约的不确定性（续）



$E \longrightarrow EOE$

$E \longrightarrow E+E$

$E \longrightarrow (E)$

$E \longrightarrow E*$

$E \longrightarrow i$

$O \longrightarrow +$

$O \longrightarrow *$

$U \longrightarrow +$

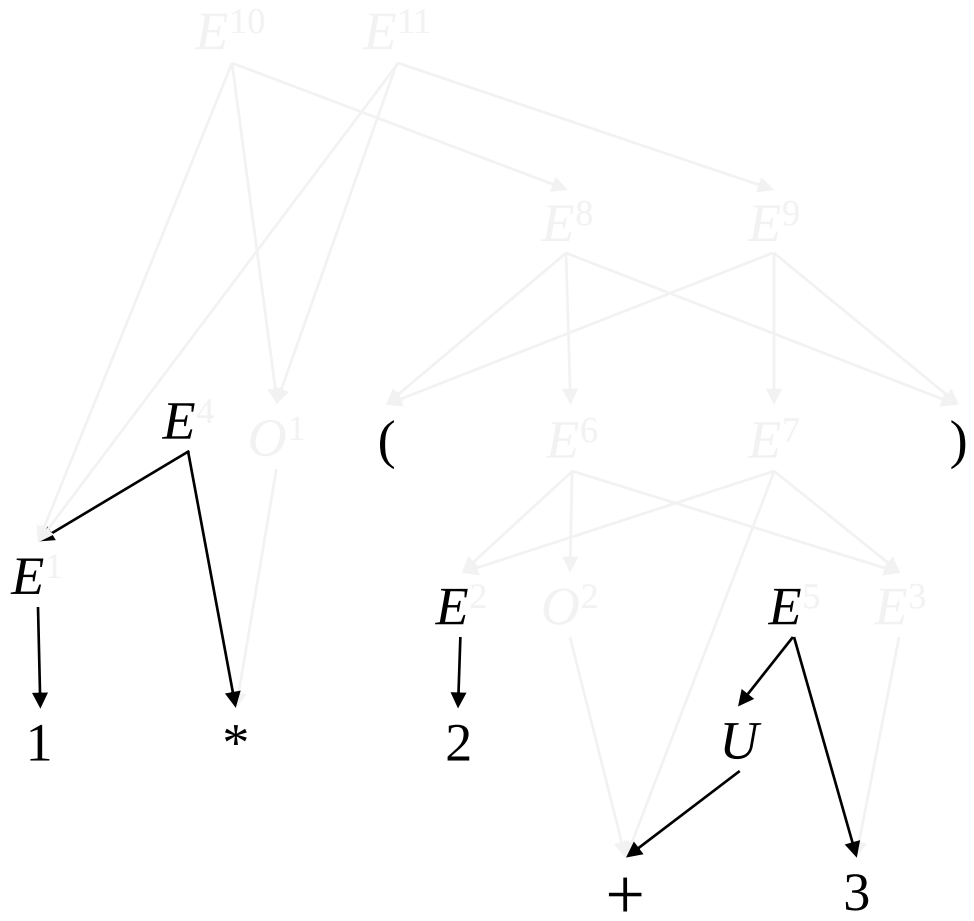
$E \longrightarrow Ui$

$\underline{1}*(2+3) \Leftarrow \underline{E}*(2+3) \Leftarrow EO(\underline{2}+3) \Leftarrow EO(E+\underline{3}) \Leftarrow EO(\underline{E+E}) \Leftarrow EO(\underline{E}) \Leftarrow \underline{EOE} \Leftarrow E$

...

$1*(2+\underline{3}) \Leftarrow 1*(\underline{2}+E) \Leftarrow 1*(\underline{E+E}) \Leftarrow 1*\underline{(E)} \Leftarrow 1\underline{*}E \Leftarrow \underline{1}OE \Leftarrow \underline{EOE} \Leftarrow E$

示例：归约的不确定性（续）



$E \longrightarrow EOE$

$E \longrightarrow E+E$

$E \longrightarrow (E)$

$E \longrightarrow E*$

$E \longrightarrow i$

$O \longrightarrow +$

$O \longrightarrow *$

$U \longrightarrow +$

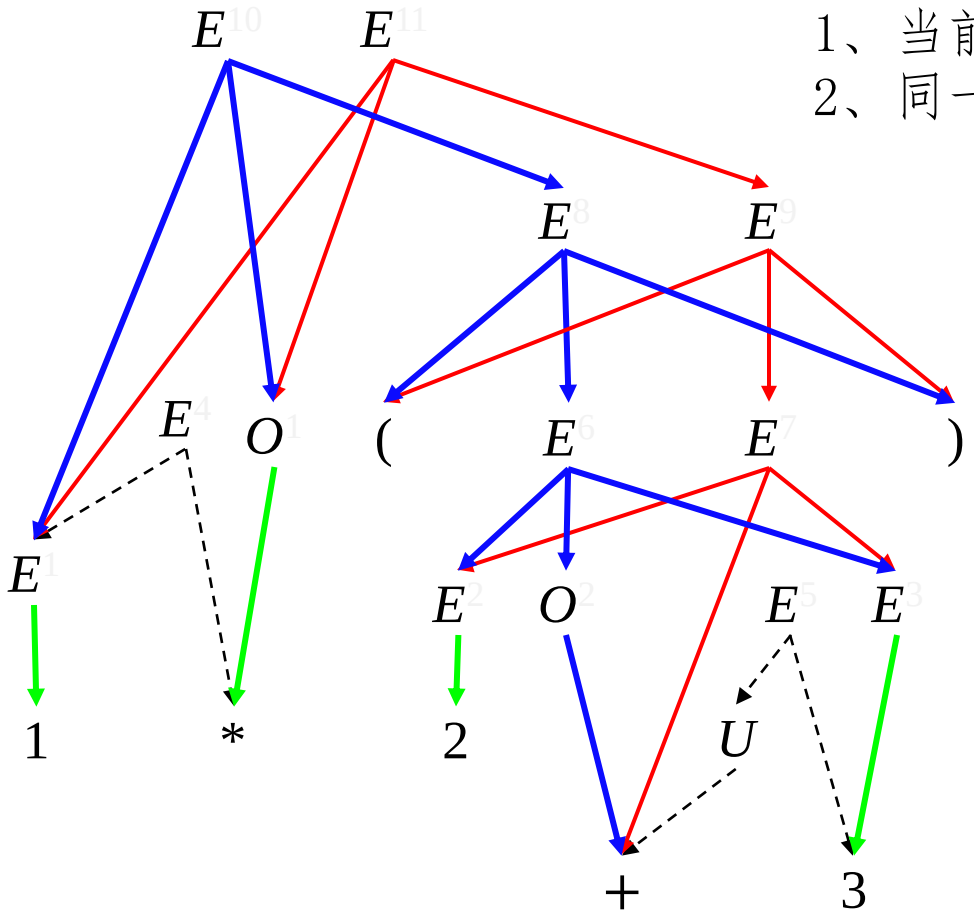
$E \longrightarrow Ui$

$\underline{1}*(2+3) \Leftarrow \underline{E*}(2+3) \Leftarrow E(\underline{2}+3) \Leftarrow E(E\underline{+}3) \Leftarrow E(E\underline{U3}) \Leftarrow E(EE) \not\Leftarrow$

...

$1*(2\underline{+}3) \Leftarrow 1*(2\underline{U3}) \Leftarrow 1*(\underline{2}E) \Leftarrow \underline{1}*(EE) \Leftarrow \underline{E*}(EE) \Leftarrow E(EE) \not\Leftarrow$

归纳：归约面临的问题



1、当前归约步中有多个可归约串

2、同一个可归约串是多个变元的候选式

$E \longrightarrow EOE$

$E \longrightarrow E+E$

$E \longrightarrow (E)$

$E \longrightarrow E*$

$E \longrightarrow i$

$O \longrightarrow +$

$O \longrightarrow *$

$U \longrightarrow +$

$E \longrightarrow Ui$

归约过程的不确定性

- **定义8.1** 归约 $w \Leftarrow^* \alpha \Leftarrow^* \ldots$ 是非确定性的，如果存在如下两种情形使得$m*n>1$：

- （情形1）归约产生的串中可能含有多个可归约串。即α有n个子串，它们都是候选式；（α 中包含的每个候选式都被称为是一个可归约串）；

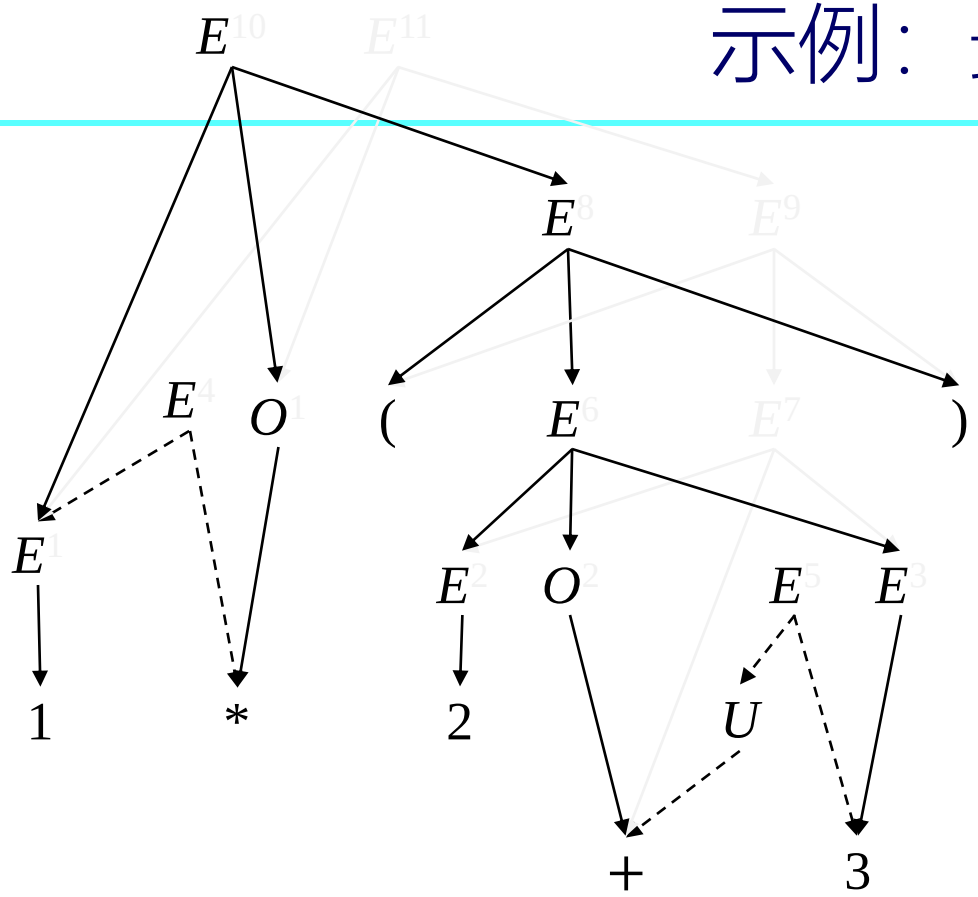
- （情形2）归约产生的串中的一个可归约串可能是多个（m个）变元的候选式。即α含有共享候选式。

- 对于最左归约而言，情形1仅限于考虑α有n个最左可归约串。

最左归约

- $w \Leftarrow_{lm}^{*} \alpha \Leftarrow_{lm}^{*} \cdots$
- α_1最左归约为α_2是指将α_1中的一个最左可归约串替换为它的一个变元而成为α_2，写为$\alpha_1 \Leftarrow_{lm} \alpha_2$。（事实上是直接最左归约）
- 如果γ_1和γ_2都是α的最左可归约串且$\gamma_1 \neq \gamma_2$，那么$\alpha=\rho\beta$且$\gamma_1,\gamma_2 \in \tau(\rho)$。
- 如果γ和γ'都是α的可归约串并且γ是最左可归约串，那么$\alpha=\rho\beta$且$\gamma \in \tau(\rho)$, 而$\rho \neq \ldots \gamma' \ldots$（$\gamma'$不是$\rho$的子串）
- 如果α只有一个可归约串γ，那么它是最左可归约串。

示例：最左归约的不确定性



$E \longrightarrow EOE$

$E \longrightarrow E+E$

$E \longrightarrow (E)$

$E \longrightarrow E*$

$E \longrightarrow i$

$O \longrightarrow +$

$O \longrightarrow *$

$U \longrightarrow +$

$E \longrightarrow Ui$

$E*$和$*$都是$E*(2+3)$的最左可归约串

$+$是$EO(E\underline{+}3)$的最左可归约串，是O和U的共享候选式

不完备

$1*(2+3) \Leftarrow \boldsymbol{E*}(2+3) \Leftarrow EO(2+3) \Leftarrow EO(E\underline{+}3) \Leftarrow EO(EO3) \Leftarrow EO(EOE) \Leftarrow EO(E) \Leftarrow EOE \Leftarrow E$

$1*(2+3) \Leftarrow E*(2+3) \Leftarrow EO(2+3) \Leftarrow EO(E\underline{+}3) \Leftarrow EO(E\boldsymbol{U3}) \Leftarrow EO(EE) \not\Leftarrow$

$1*(2+3) \Leftarrow \boldsymbol{E*}(2+3) \Leftarrow E(2+3) \Leftarrow E(E+3) \Leftarrow E(EO3) \Leftarrow E(EOE) \Leftarrow E(E) \Leftarrow EE \not\Leftarrow$

$1*(2+3) \Leftarrow E*(2+3) \Leftarrow E(2+3) \Leftarrow E(E+3) \Leftarrow E(E\boldsymbol{U3}) \Leftarrow E(EUE) \not\Leftarrow$

$1*(2+3) \Leftarrow E*(2+3) \Leftarrow EO(2+3) \Leftarrow \boldsymbol{EO(E+3) \Leftarrow EO(E+E)} \Leftarrow EO(E) \Leftarrow EOE \Leftarrow E$

是最右推导的逆但不是最左归约

最左归约的不确定性

- **定义8.3** 最左归约
  $w \Leftarrow_{lm}^{*} \rho x \Leftarrow_{lm}^{*} \ldots$, s.t. $\rho = \delta\gamma\beta \wedge \gamma \in ran(\mathcal{P}) \rightarrow \beta = \varepsilon, x \in \tau(w)$，
  是非确定性的过程，如果下列两个情形中$n*m>1$：
- （情形1）ρ有n个后缀都是可归约串，形式地，
  $$|\tau(\rho) \cap ran(\mathcal{P})| = n$$
- （情形2）有一个由m个变元共享的候选式是ρ的可归约串，
  形式地，$\exists \gamma \in (\tau(\rho) \cap ran(\mathcal{P})) \cdot |\{A : (A, \gamma) \in \mathcal{P}\}| = m$

$\rho = \delta\gamma\beta \wedge \gamma \in ran(\mathcal{P}) \rightarrow \beta = \varepsilon$表示$\rho$中每个可归约串都只能是它的后缀。而且这些可归约串都是最左的，这是由ρ处在最左归约过程中这一事实来保证。

最左归约适合基于栈的模拟过程。

基于栈的移进-归约分析框架SRF

- 利用栈来完成语法分析任务 $\varepsilon\cdot w \Leftarrow_{lm}^{*} \rho\cdot ay \Leftarrow_{lm}^{*} S\cdot\varepsilon$
  - 若ρ有最左可归约串γ则有$\rho\cdot ay = \alpha\gamma\cdot ay \Leftarrow_{lm} \alpha A\cdot ay$
  - 否则， $\rho\cdot ay \Leftarrow_{lm}^{*} \rho a\cdot y$
  - 若把栈设为逻辑空的话，$Z_0\cdot w\# \Leftarrow_{lm}^{*} Z_0\rho\cdot ay\# \Leftarrow_{lm}^{*} Z_0 S\cdot\#$
- 从而形成**移进-归约**（shift-reduce）分析框架：
  - 初始化栈为Z_0，剩余串为$w\#$，形成初始归约串$\varepsilon\cdot w$。
  - 过程中对于归约串$\rho\cdot ay$，若有$\gamma\in\tau(\rho)$且$(A,\gamma)\in\mathcal{P}$，则$\rho=\alpha\gamma$被更新为αA并形成一步直接归约$\alpha\gamma\cdot ay \Leftarrow_{lm} \alpha A\cdot ay$
  - 过程中对于$\rho\cdot ay$，形成0步归约$\rho\cdot ay \Leftarrow_{lm}^{*} \rho a\cdot y$（移进）
  - 过程进行到$Z_0 S\cdot\#$时断言接受之，否则有语法错误。

$1*(2+3)\Leftarrow E*(2+3)\Leftarrow EO(2+3)\Leftarrow \boldsymbol{EO(E+3)\Leftarrow EO(E+E)}\Leftarrow EO(E)\Leftarrow EOE\Leftarrow E$

$EO(2+3)\Leftarrow \boldsymbol{EO(E+\cdot 3)\Leftarrow^{*} EO(E+3\cdot)\Leftarrow EO(E+E\cdot)}\Leftarrow EO(E)$

完备

示例：移进-归约分析过程

步骤	符号栈	剩余输入串	分析动作
0	Z_0	$i*(i+i)\#$	shift
1	Z_0i	$*(i+i)\#$	reduce: $E\longrightarrow i$; 还是shift ?
2	Z_0E	$*(i+i)\#$	shift
3	Z_0E*	$(i+i)\#$	reduce:$O\longrightarrow *$; $E\longrightarrow E*$; 还是shift ?
4	Z_0EO	$(i+i)\#$	shift
5	$Z_0EO($	$i+i)\#$	shift
6	$Z_0EO(i$	$+i)\#$	reduce:$E\longrightarrow i$; 还是shift ?
7	$Z_0EO(E$	$+i)\#$	shift
8	$Z_0EO(E+$	$i)\#$	reduce: $O\longrightarrow +$; $U\longrightarrow +$; 还是shift ?
9	$Z_0EO(E+i$	$)\#$	reduce: $E\longrightarrow i$; 还是shift ?
10	$Z_0EO(E+E$	$)\#$	reduce: $E\longrightarrow E+E$; 还是shift ?
11	$Z_0EO(E$	$)\#$	shift
12	$Z_0EO(E)$	$\#$	reduce: $E\longrightarrow (E)$; 还是shift ?
13	Z_0EOE	$\#$	reduce: $E\longrightarrow EOE$; 还是shift ?
14	Z_0E	$\#$	acc

$E\longrightarrow EOE$

$E\longrightarrow E+E$

$E\longrightarrow (E)$

$E\longrightarrow E*$

$E\longrightarrow i$

$O\longrightarrow +$

$O\longrightarrow *$

$U\longrightarrow +$

$E\longrightarrow Ui$

规范归约

- 最右推导的逆过程是确定化的，称为规范归约。
- 最左归约虽然与规范归约不相同，但可发展为SRF
- 注意到，SRF（移进-归约框架）允许无条件移进。
- 展望：在SRF上可模拟规范归约。

1*(2+3)⇐**E***(2+3)⇐*EO*(2+3)⇐*EO*(*E*+3)⇐*EO*(*EO*3)⇐*EO*(*EOE*)⇐*EO*(*E*)⇐*EOE*⇐*E*

~~1*(2+3)⇐*E**(2+3)⇐*EO*(2+3)⇐*EO*(*E*+3)⇐*EO*(*EU*3)⇐*EO*(*EE*)⇍~~

~~1*(2+3)⇐***E***(2+3)⇐*E*(2+3)⇐*E*(*E*+3)⇐*E*(*EO*3)⇐*E*(*EOE*)⇐*E*(*E*)⇐*EE*⇍~~

~~1*(2+3)⇐*E**(2+3)⇐*E*(2+3)⇐*E*(*E*+3)⇐*E*(*EU*3)⇐*E*(*EE*)⇍~~

1*(2+3)⇐*E**(2+3)⇐*EO*(2+3)⇐***EO*(*E*+3)⇐*EO*(*E*+*E*)**⇐*EO*(*E*)⇐*EOE*⇐*E*

- “归约-归约”冲突：栈顶有多个可归约串，选哪个归约？
- “移进-归约”冲突：栈顶有可归约串，归约呢还是移进？
- 栈顶没有可归约串，只须移进即可。

对示例文法的讨论

- 对于有二义性的文法进行处理：
  - EOE与$E+E$有一定重复，导致文法有二义性，去除$E+E$，但是不如将O省去更优
  - 单目运算符*的结合性与其他运算不一致，去除E^*，但或许有别的用途，重新设计
  - 让单目运算符+存在的话，就让i为无符号整数，当然还应该有单目运算符-，这样的话采用UE取代Ui，并使得单目+的优先级高于双目+。
  - 剩下与运算符的结合性、优先级有关的二义性未消除

$E \longrightarrow E+E$
$E \longrightarrow E^*E$
$E \longrightarrow (E)$
$E \longrightarrow i$
$U \longrightarrow +$
$E \longrightarrow UE$

~~$E \longrightarrow EOE$~~
$E \longrightarrow E+E$
$E \longrightarrow (E)$
~~$E \longrightarrow E^*$~~
$E \longrightarrow i$
~~$O \longrightarrow +$~~
~~$O \longrightarrow *$~~
$U \longrightarrow +$
$E \longrightarrow Ui$

$1^*(2+3) \Leftarrow E^*(2+3) \Leftarrow EO(2+3) \Leftarrow EO(E+3) \Leftarrow EO(EO3) \Leftarrow EO(EOE) \Leftarrow EO(E) \Leftarrow EOE \Leftarrow E$

~~$1^*(2+3) \Leftarrow E^*(2+3) \Leftarrow EO(2+3) \Leftarrow EO(E+3) \Leftarrow EO(E+E) \Leftarrow EO(E) \Leftarrow EOE \Leftarrow E$~~

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

Stack

Input String

num	*	(	num	+	num	)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

num	*	(	num	+	num	)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

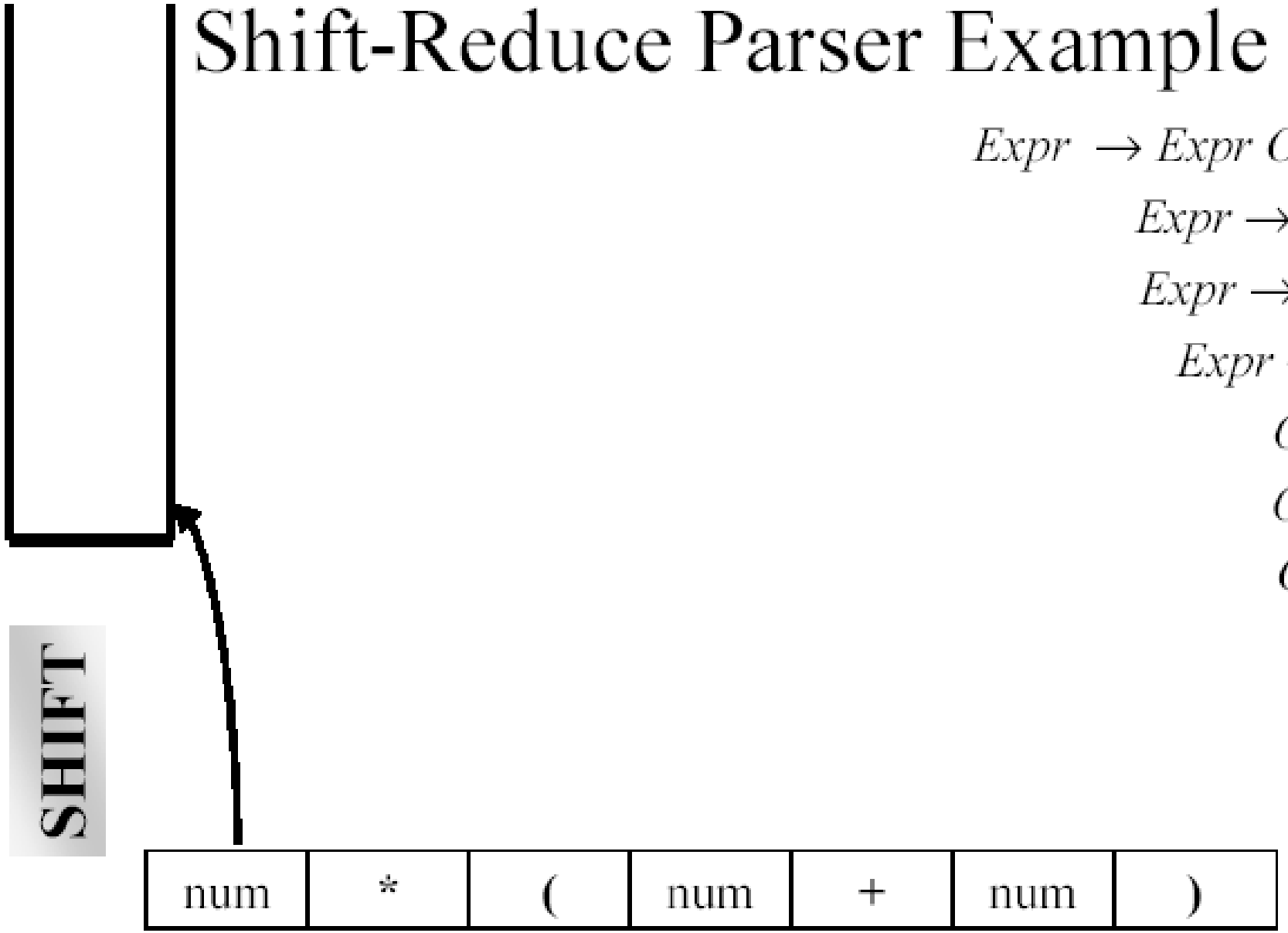
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ \ -$

$Op \rightarrow\ *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

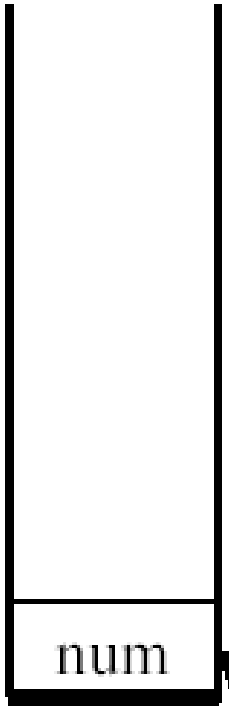
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



SHIFT

*	(	num	+	num	)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

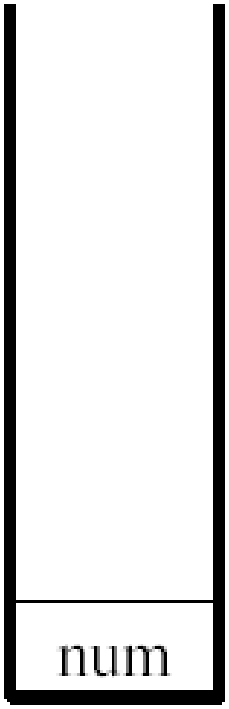
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ \ -$

$Op \rightarrow\ *$



REDUCE

*	(	num	+	num	)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

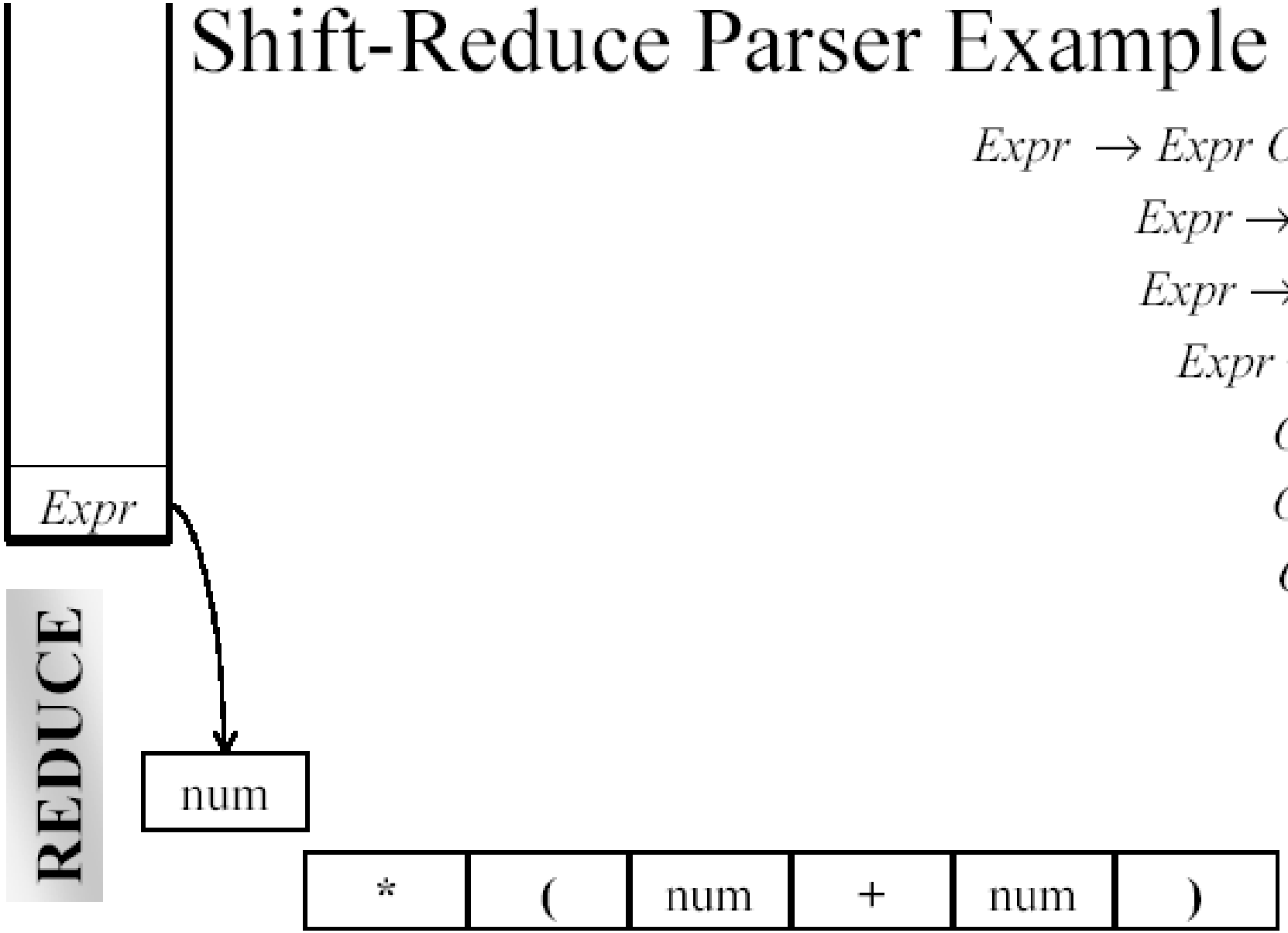
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

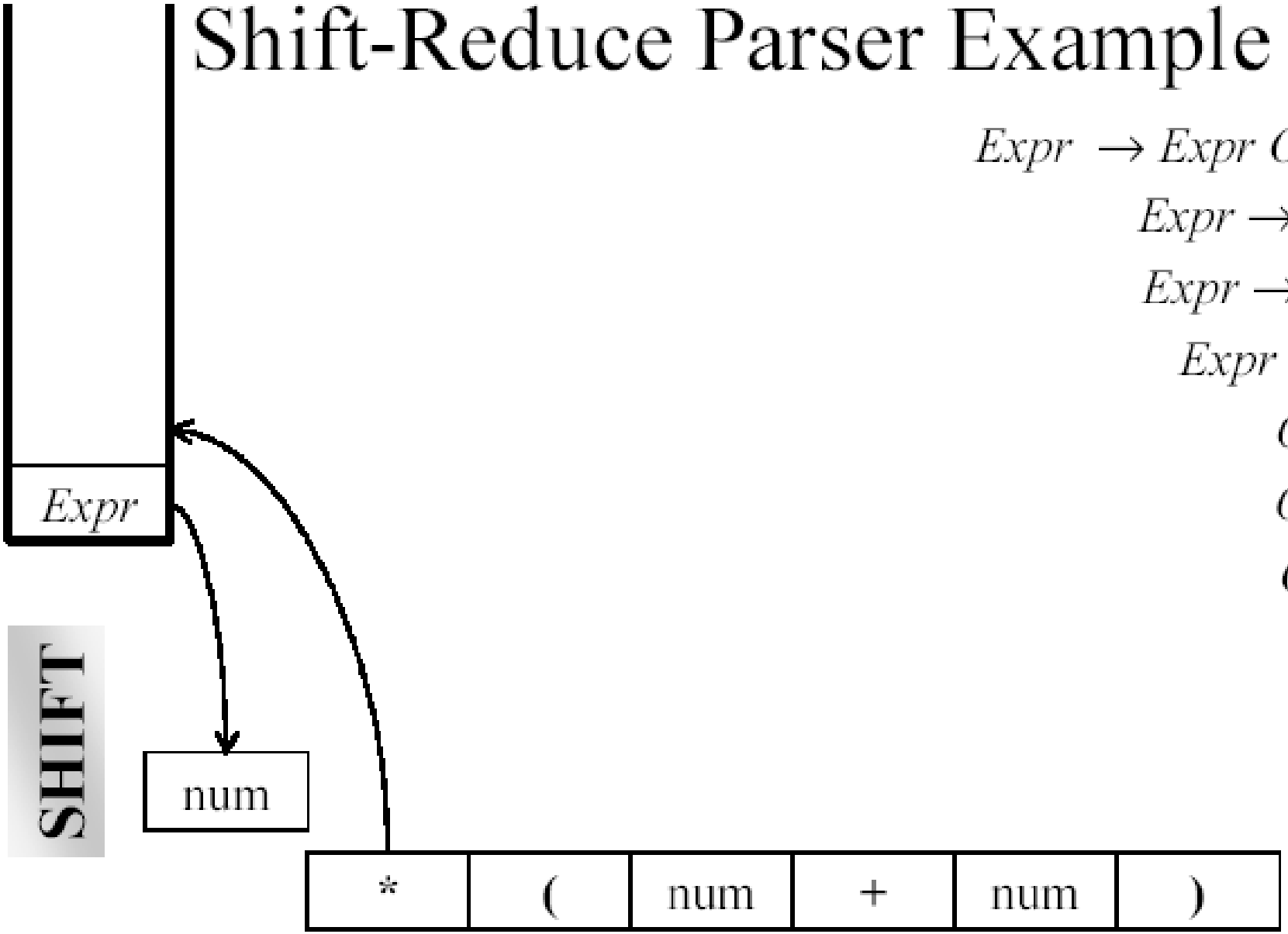
$Expr \rightarrow -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

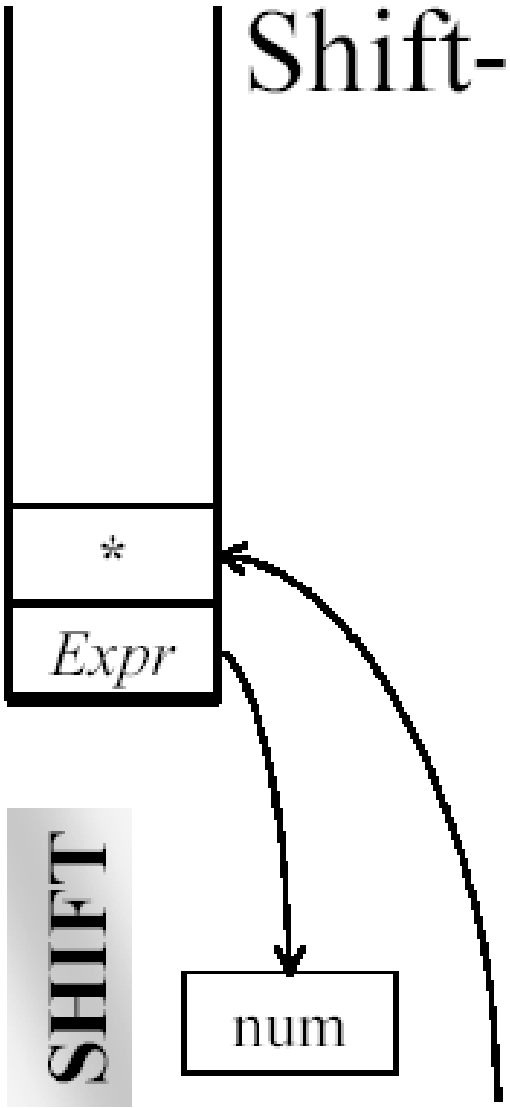
$Expr \rightarrow -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$



(	num	+	num	)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

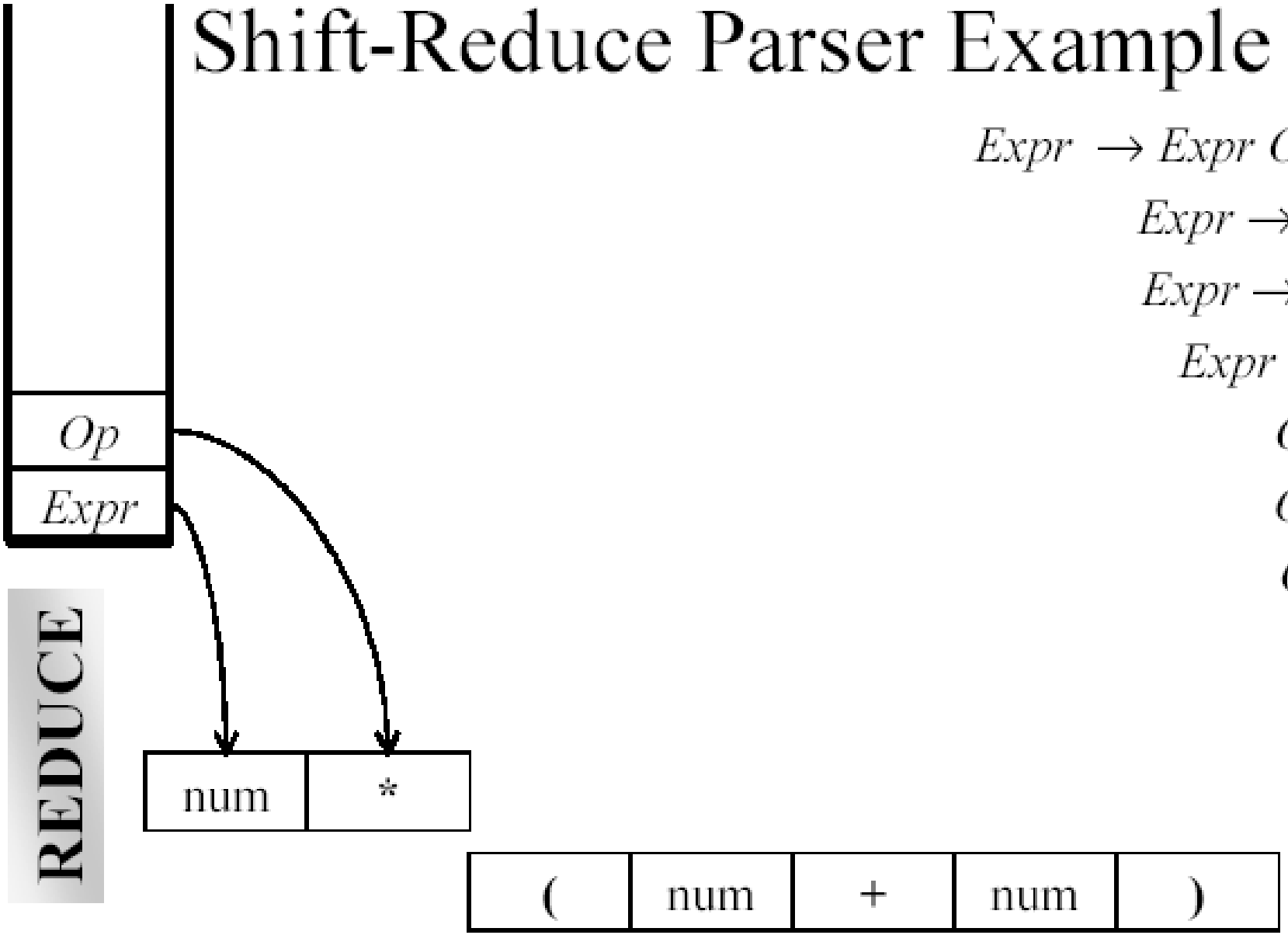
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

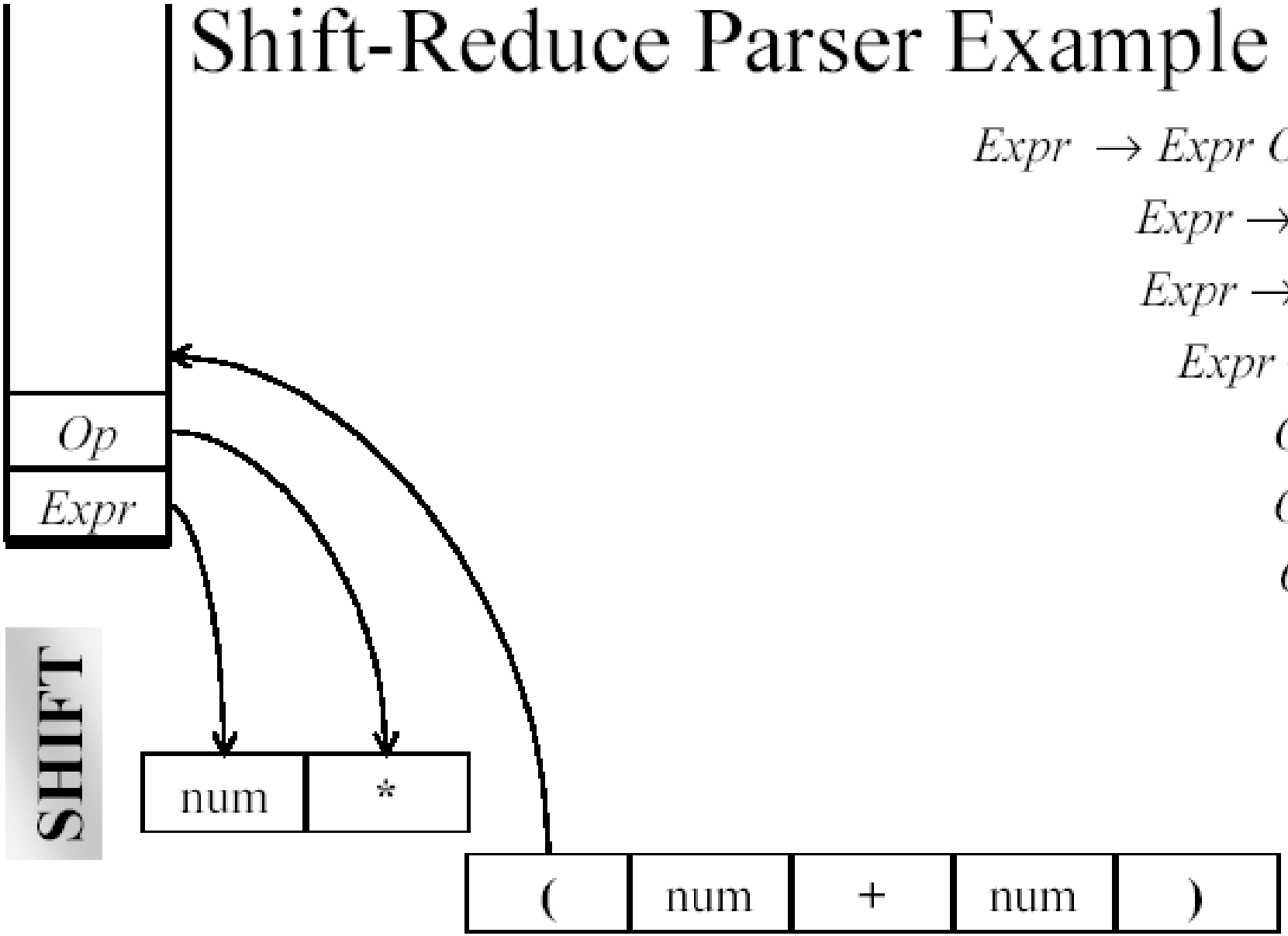
$Expr \rightarrow -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

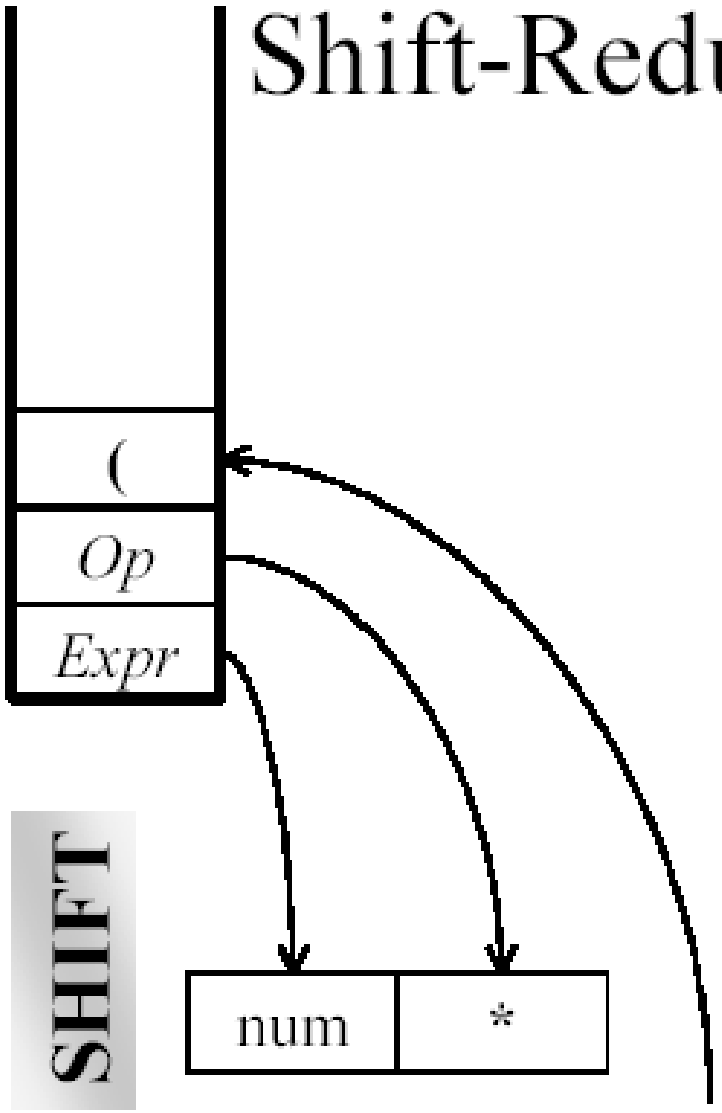
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



num	+	num	)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

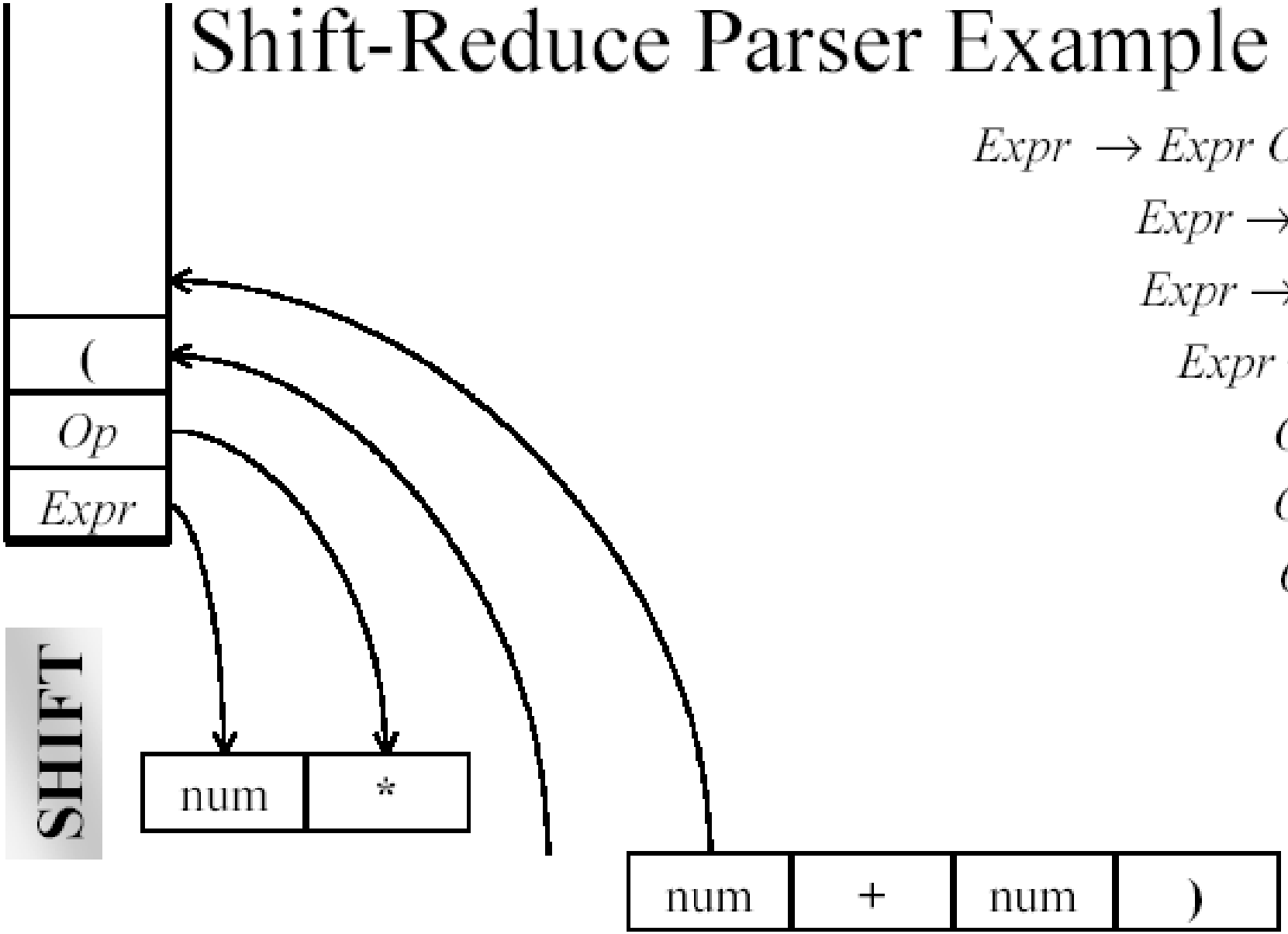
$Expr \rightarrow -Expr$

$Expr \rightarrow \text{num}$

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

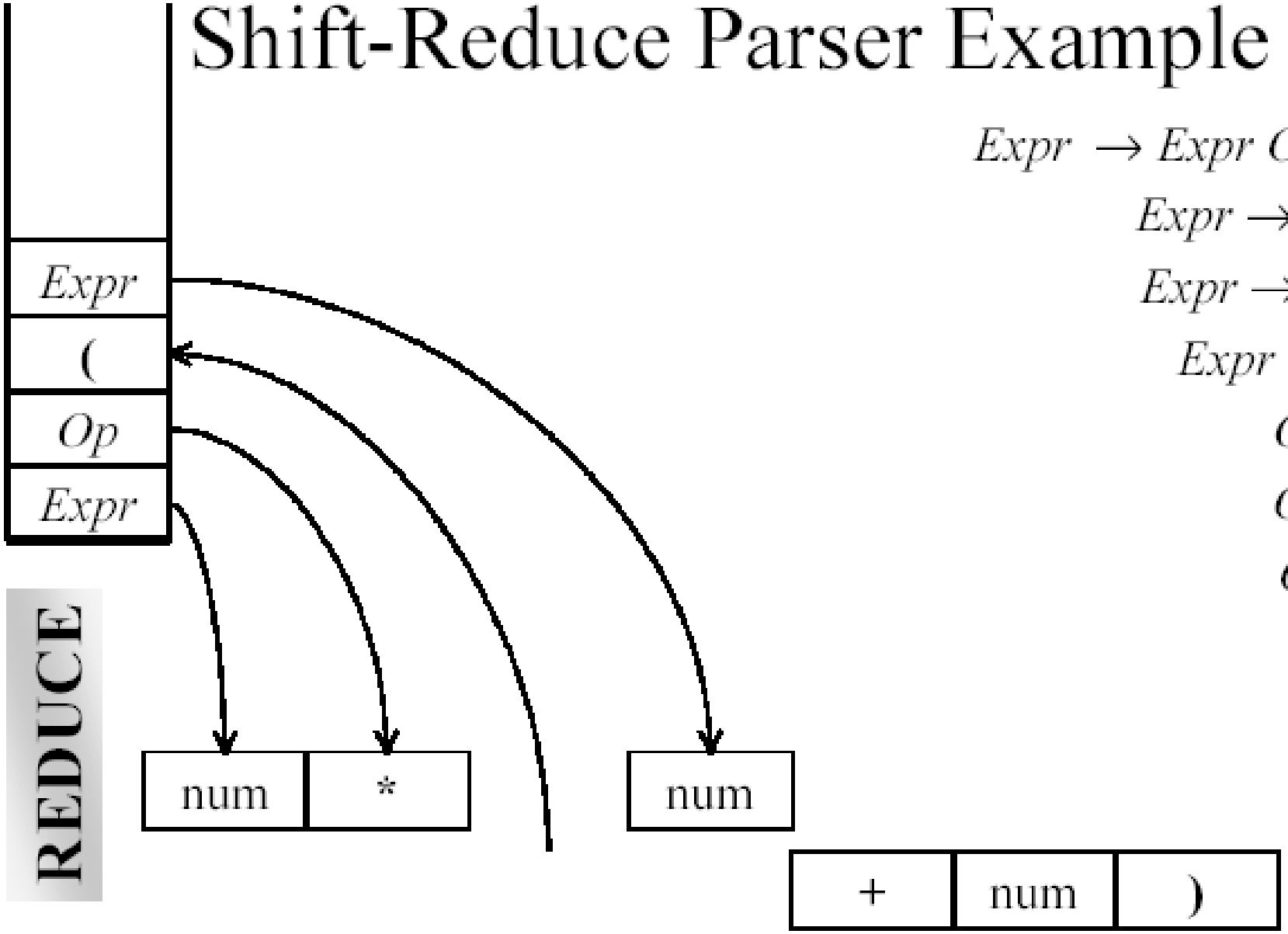
$Expr \rightarrow -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

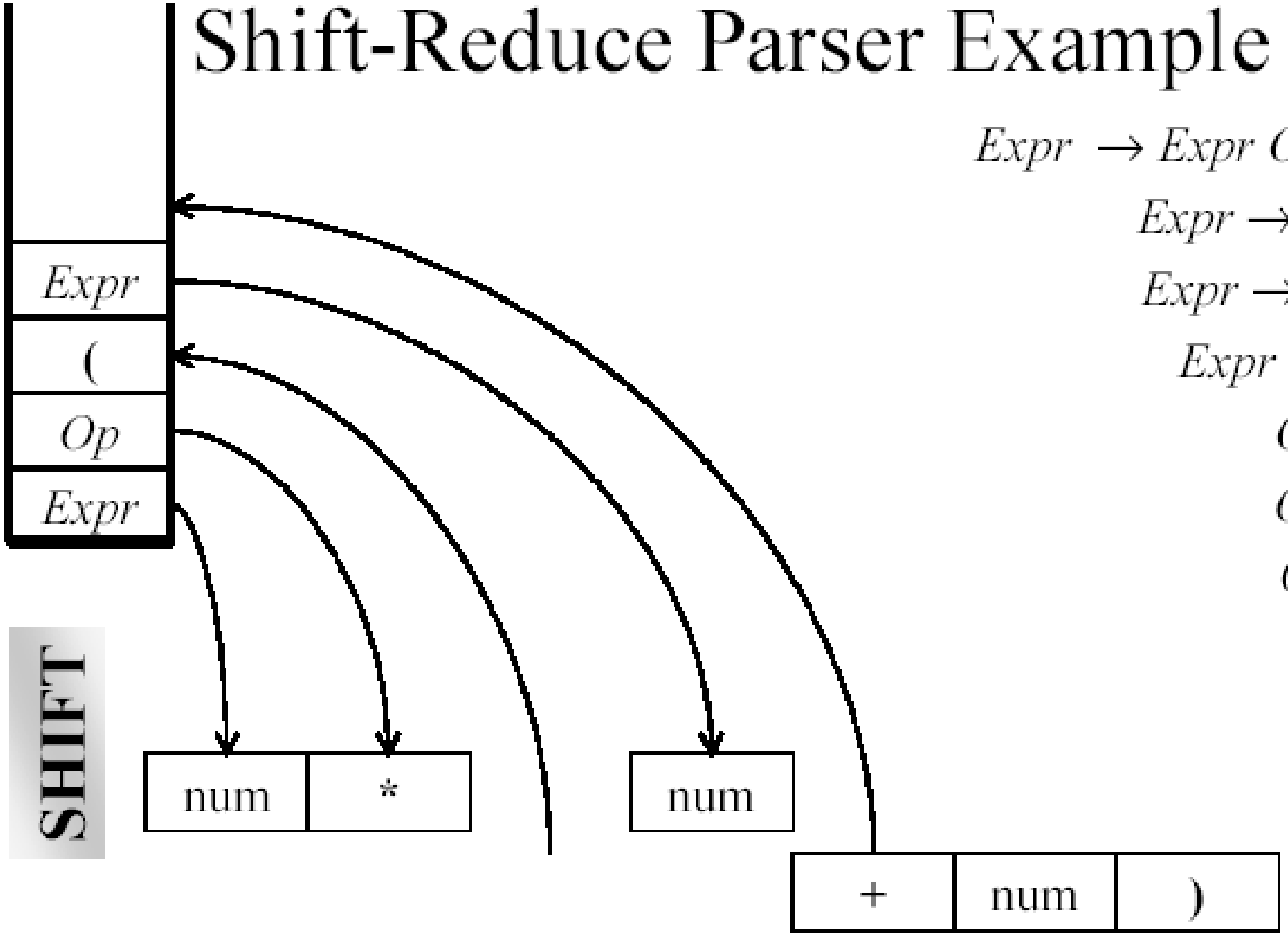
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

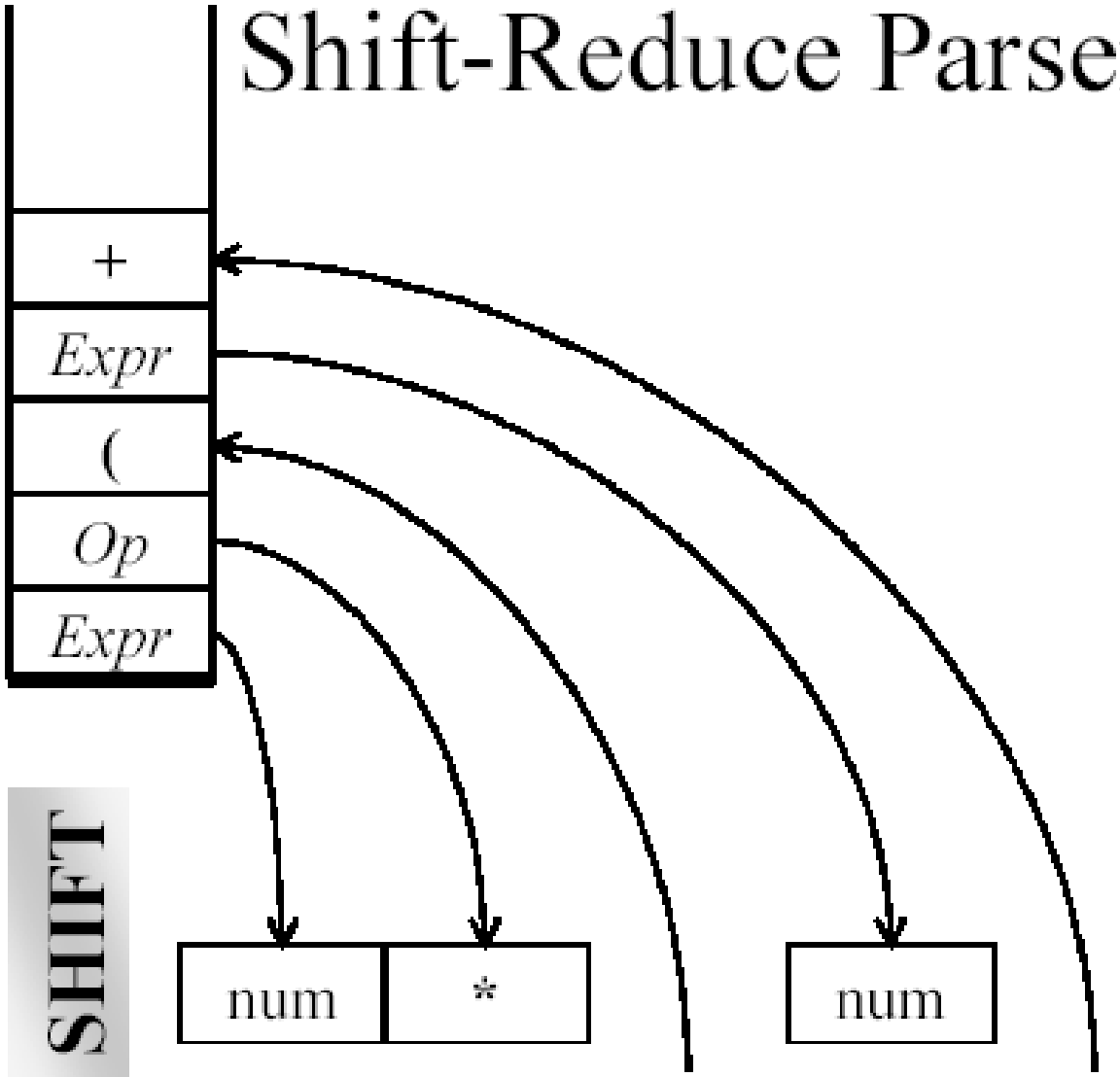
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



num |)

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

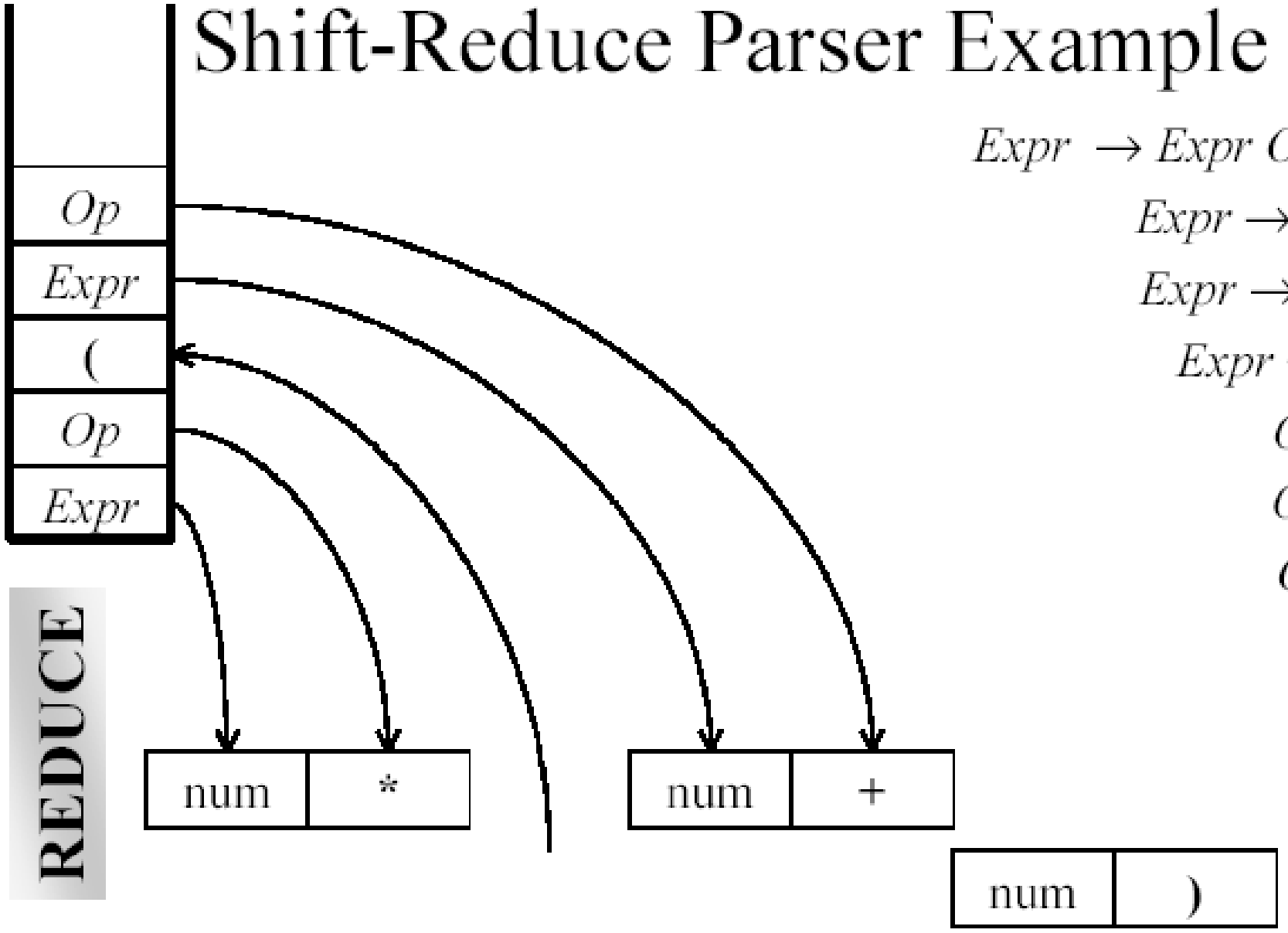
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

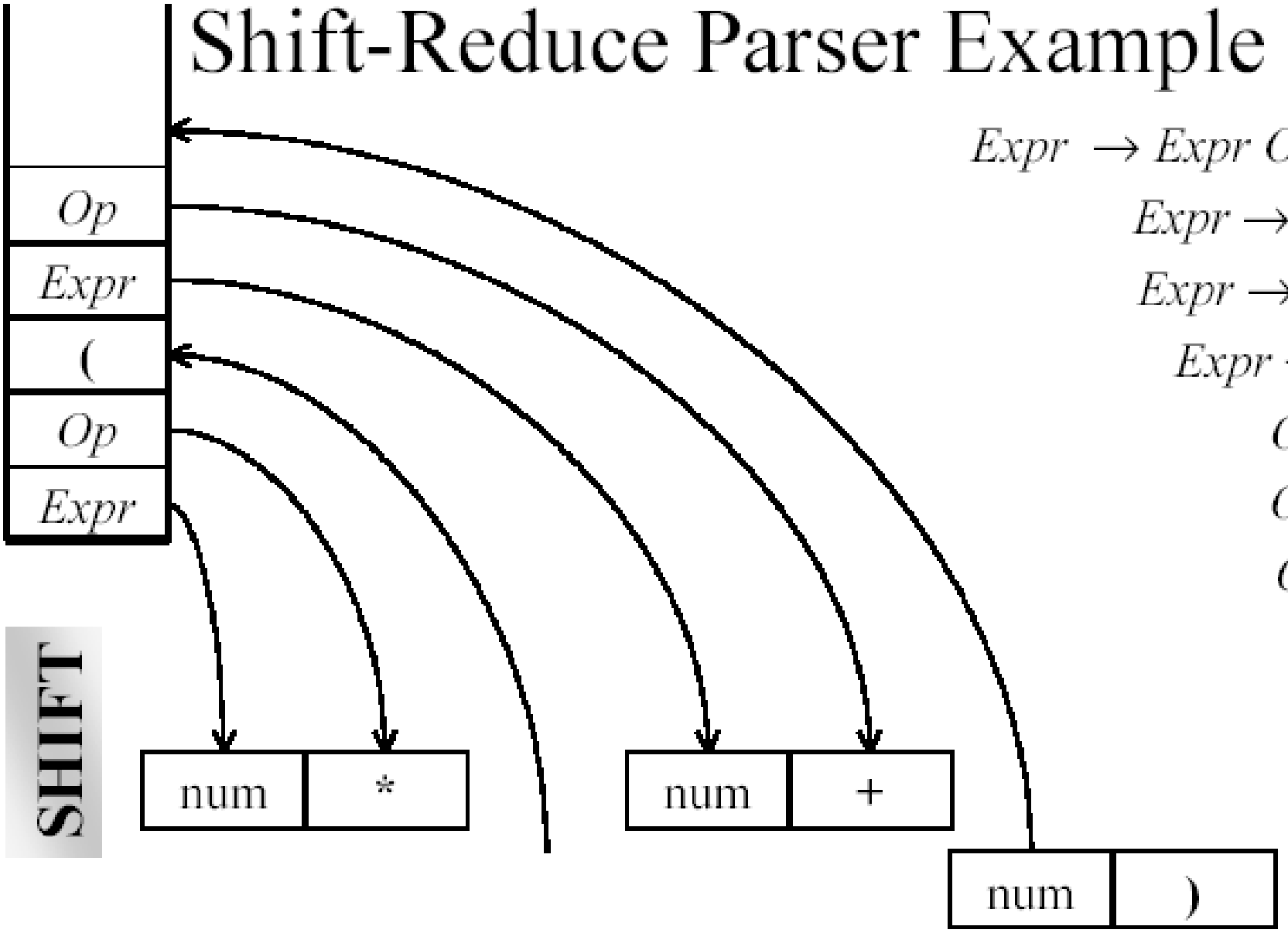
$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$



Shift-Reduce Parser Example



$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

$Expr \rightarrow\ -\ Expr$

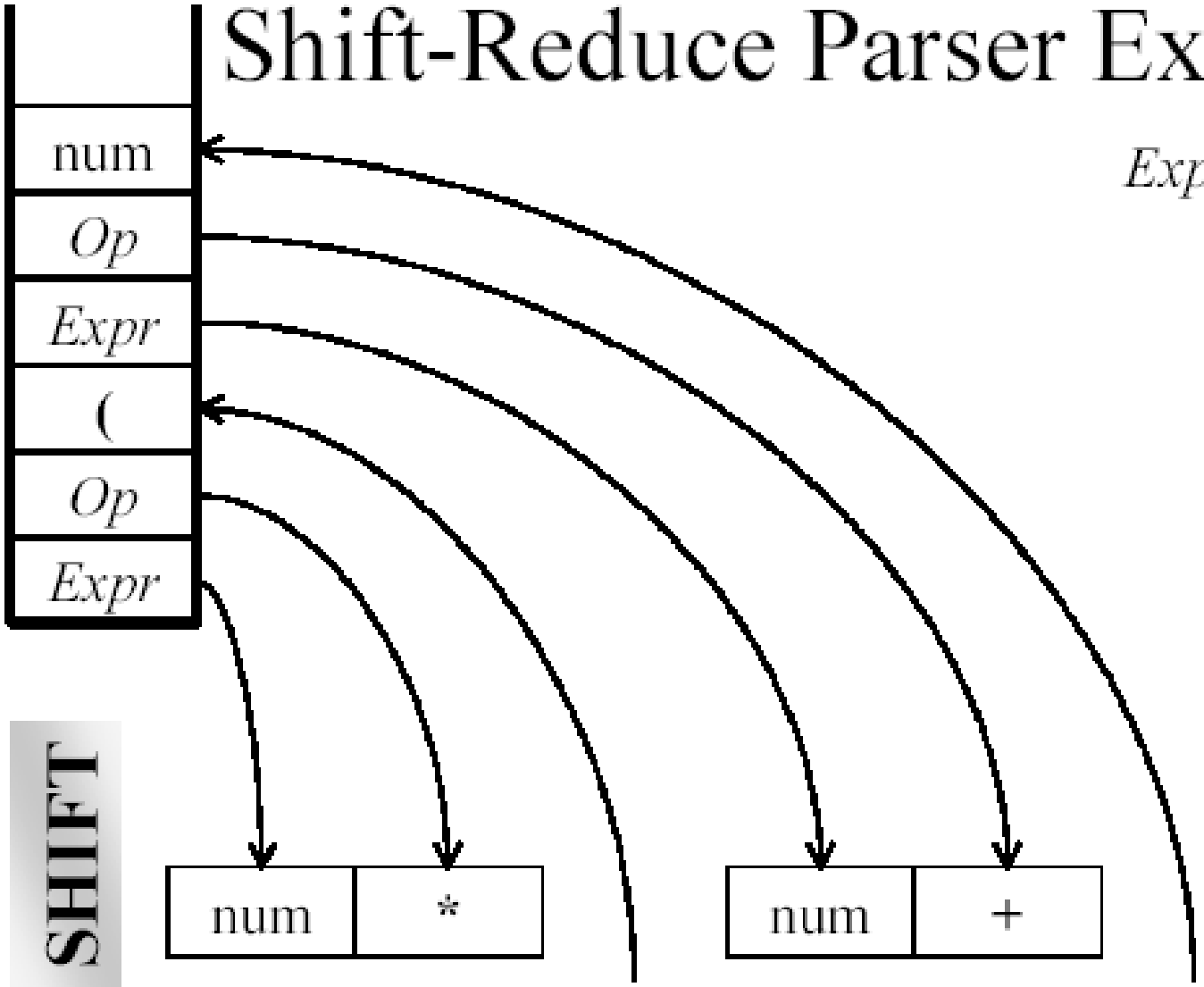
$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$

Shift-Reduce Parser Example



$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

$Expr \rightarrow\ -\ Expr$

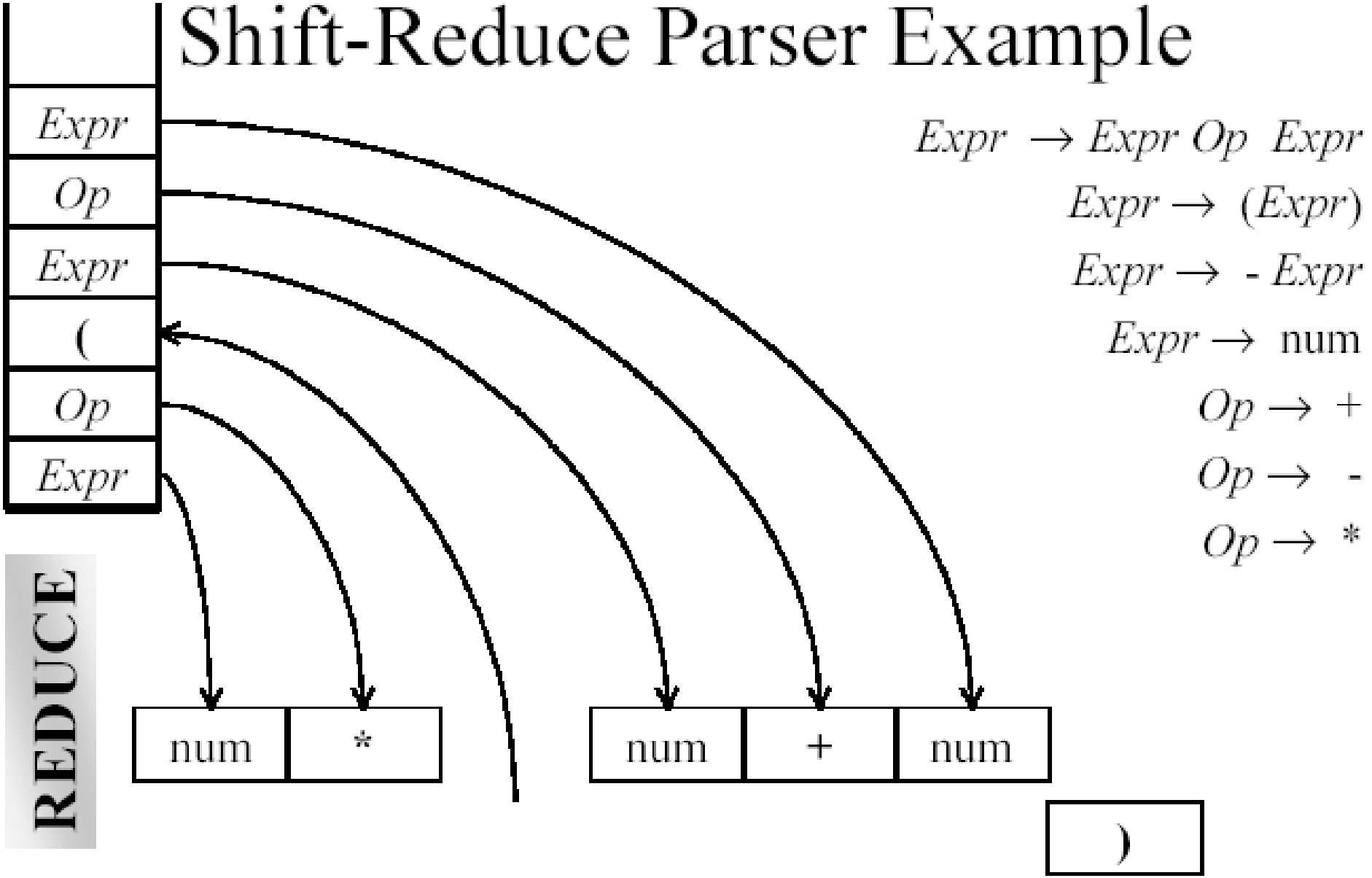
$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$

Shift-Reduce Parser Example



Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

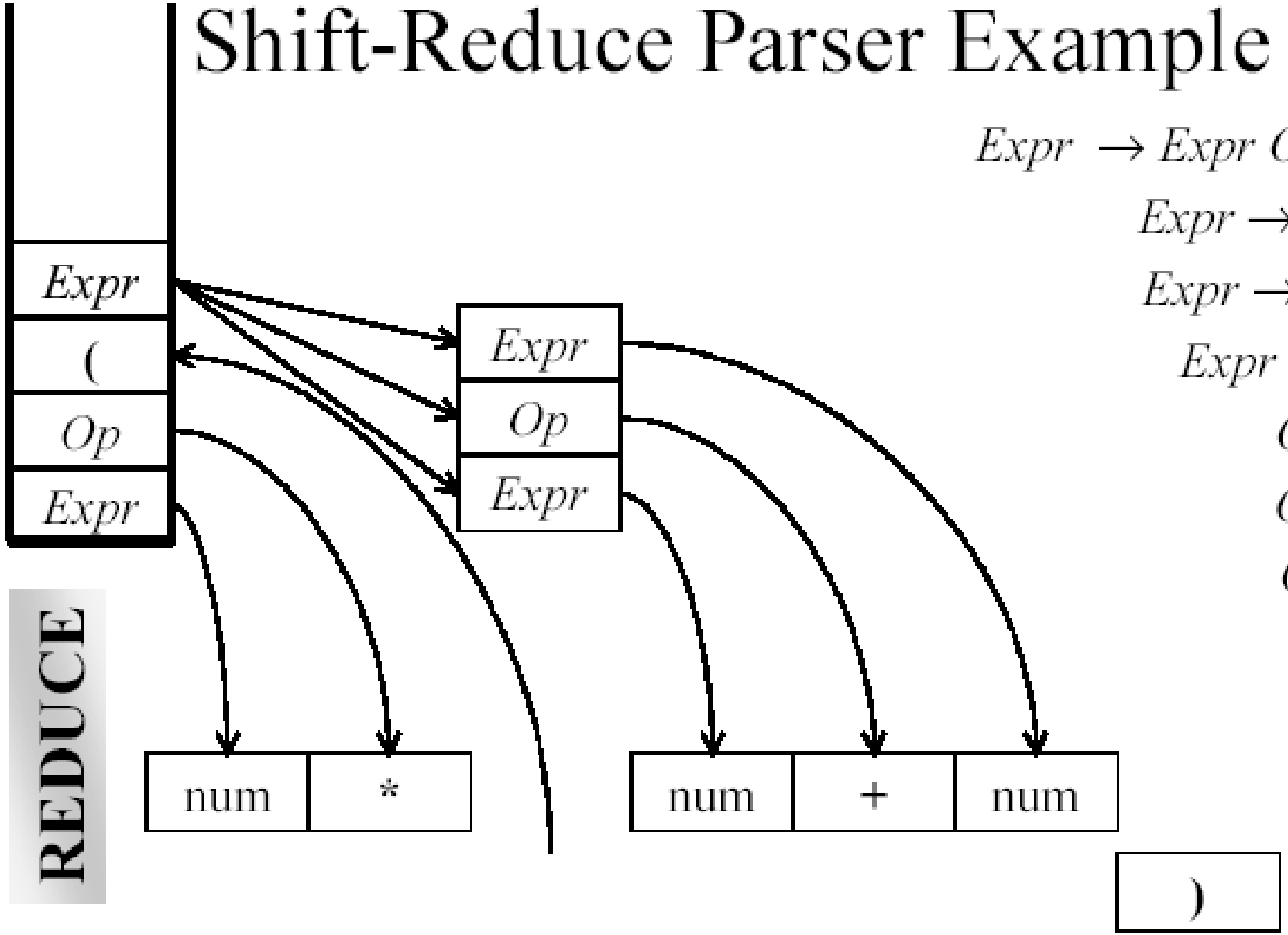
$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

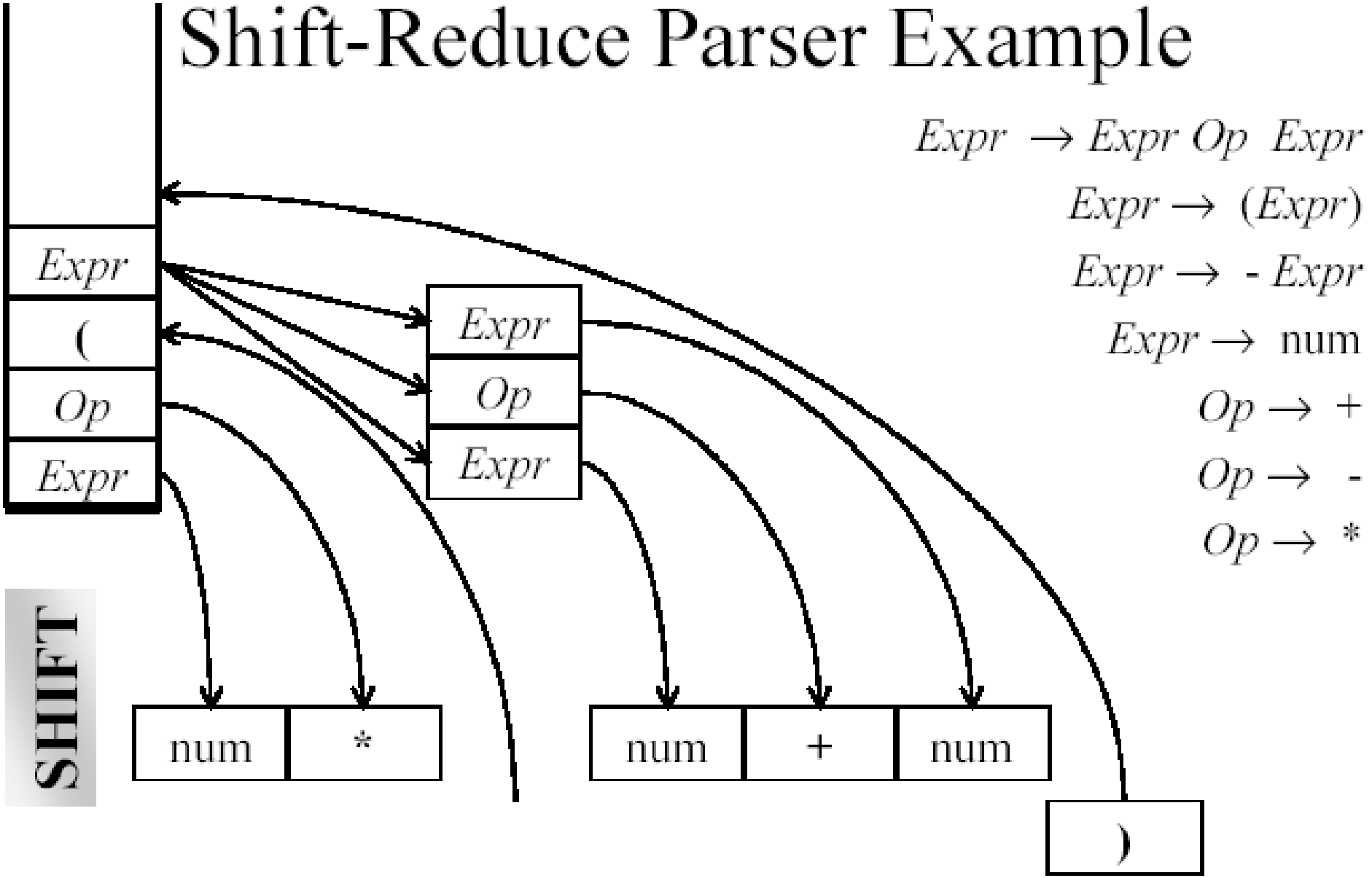
$Op \rightarrow\ +$

$Op \rightarrow\ -$

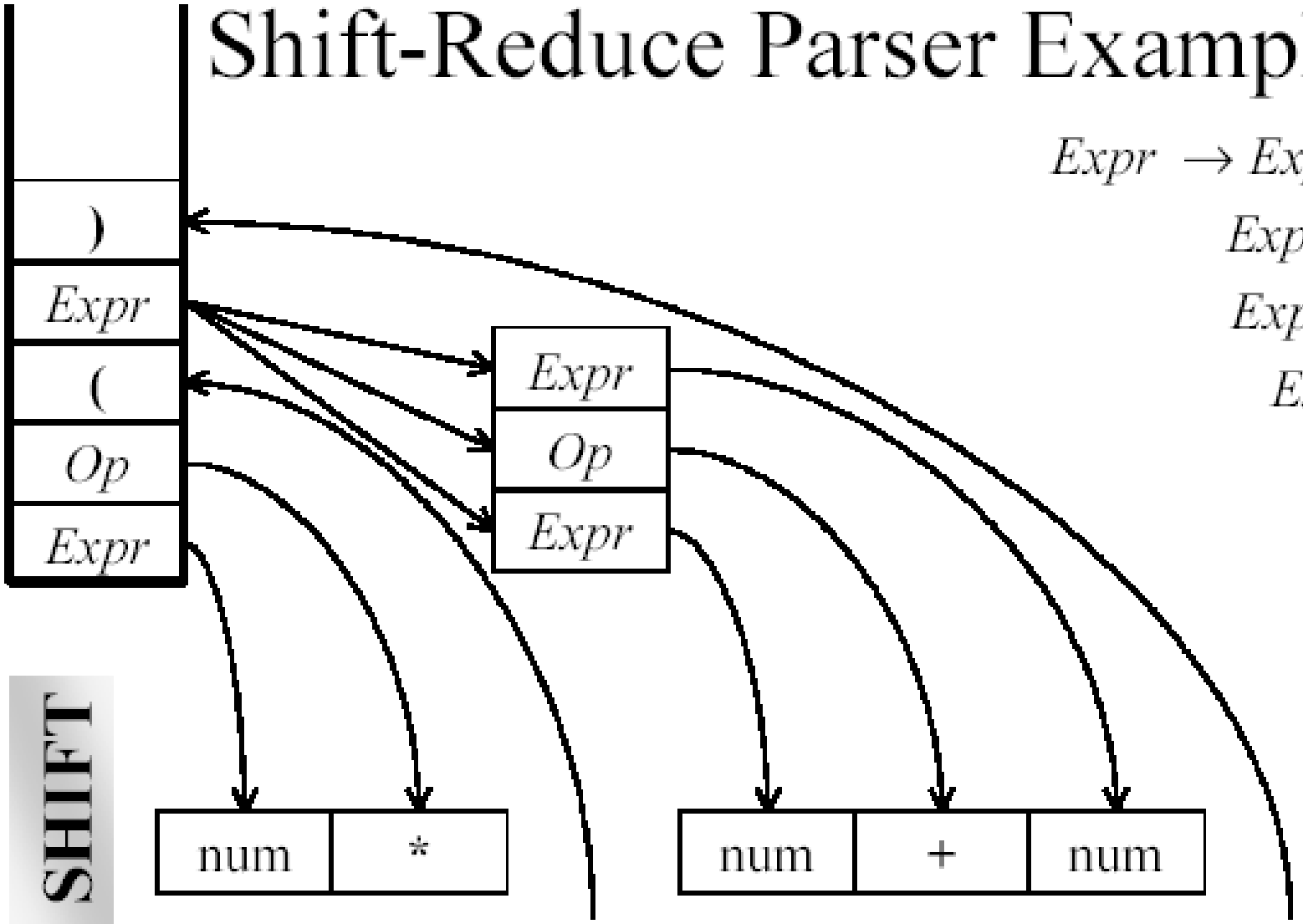
$Op \rightarrow\ *$



Shift-Reduce Parser Example



Shift-Reduce Parser Example



$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow\ (Expr)$

$Expr \rightarrow\ -\ Expr$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ -$

$Op \rightarrow\ *$

Shift-Reduce Parser Example

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow (Expr)$

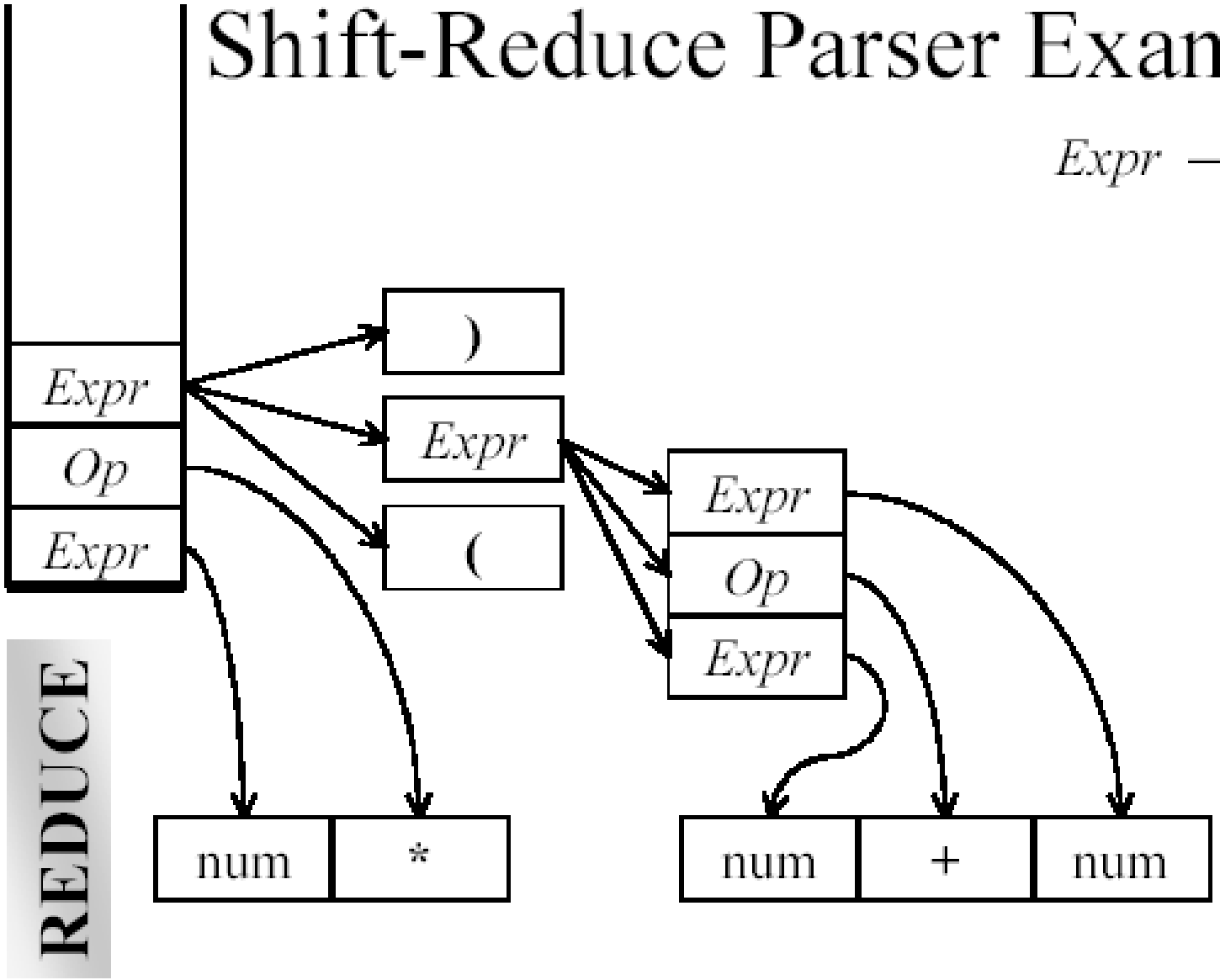
$Expr \rightarrow -\ Expr$

$Expr \rightarrow \text{num}$

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$



Shift-Reduce Parser Example

Expr → *Expr Op Expr*

Expr → (*Expr*)

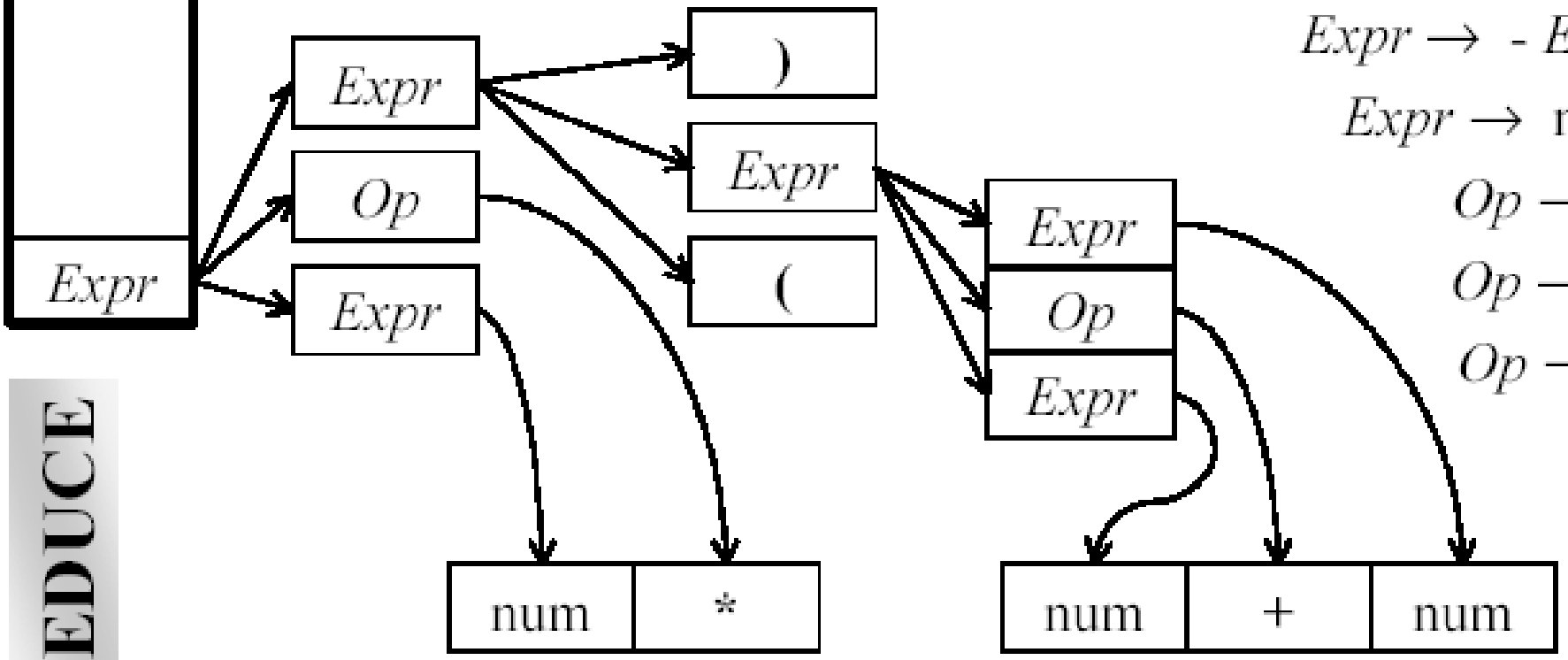
Expr → - *Expr*

Expr → num

Op → +

Op → -

Op → *



REDUCE

Shift-Reduce Parser Example

Expr → *Expr Op Expr*

Expr → (*Expr*)

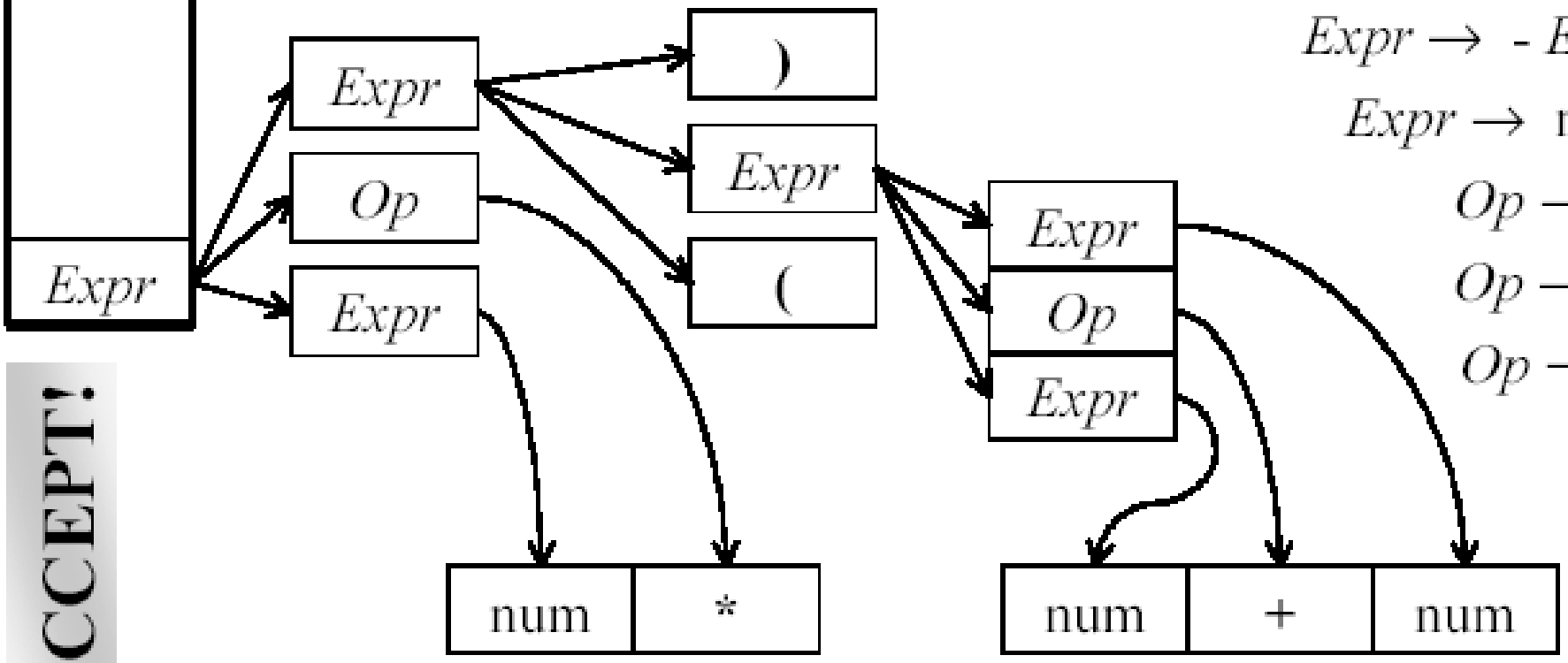
Expr → - *Expr*

Expr → num

Op → +

Op → -

Op → *



ACCEPT!

观 察

- 一个不断移进-归约的过程，从输入串开始，到初始符号结束或者出错结束。
- 如果能成功的话，在过程的每一步都得到一个句型。
- 在过程的每一步，总是做出如下的一个选择：
  - 输入串中的当前输入符号被移进栈，即压入到栈顶，当前输入符号更改为下一个。
  - 如果栈顶部分的符号串刚好跟某个变元A的某个候选式一样，则这部分全部出栈，然后将A压入栈。这个过程即是归约。将来可以给归约附加一些行为或动作如构造语法树、进行语义分析等。

Parsing Stack	Input String	Action
Z_0	num*(num+num)#	shift
Z_0num	*(num+num)#	shift
Z_0*Expr*	*(num+num)#	reduce *Expr* →num
Z_0*Expr**	(num+num)#	shift
Z_0*Expr Op*	(num+num)#	reduce *Op* → *
Z_0*Expr Op*(	num+num)#	shift
Z_0*Expr Op*(num	+num)#	shift
Z_0*Expr Op*(*Expr*	+num)#	reduce *Expr* →num
Z_0*Expr Op*(*Expr*+	num)#	shift
Z_0*Expr Op*(*Expr Op*	num)#	reduce *Op* → +
Z_0*Expr Op*(*Expr Op* num	)#	shift
Z_0*Expr Op*(*Expr Op Expr*	)#	reduce *Expr* →num
Z_0*Expr Op*(*Expr*	)#	red. *Expr*→*Expr Op Expr*
Z_0*Expr Op*(*Expr*)	#	shift
Z_0*Expr Op Expr*	#	reduce *Expr*→(*Expr*)
Z_0*Expr*	#	red. *Expr*→*Expr Op Expr*

栈的变化

- 初始时栈顶为Z_0；
- 移进当前输入符号至栈顶（压入该符号）；
- 当栈顶出现“可归约串”则弹出并将该串的归约变元压入；
- 分析成功时栈顶为初始符号，下面为#；
- 分析不成功时以报错结束。

- 可能的问题：
  - 怎么知道可归约串出现了？如何确定它的归约变元？
  - 如果栈顶有可归约串，是归约还是放弃归约？
  - 如果栈顶没有可归约串，就移进还是断言为失败？
  - 栈上若有可归约串，它们一定都是在栈顶吗？

分析过程中的冲突

- 知道“可归约串”出现、确定出以它为候选式的变元、移进还是失败，会面临多选的情形。
- 对于确定性的过程而言，多选步骤即就是冲突。

- 宏观来看，确定性的自下而上分析过程面临两种冲突：
  - 栈顶部分可能与多个候选式匹配；
  - 归约-归约冲突

  - 也可能即使有匹配的候选式也要放弃归约，进而继续移进直到随后找到其它匹配的候选式。
  - 移进-归约冲突

Conflicts

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow (Expr)$

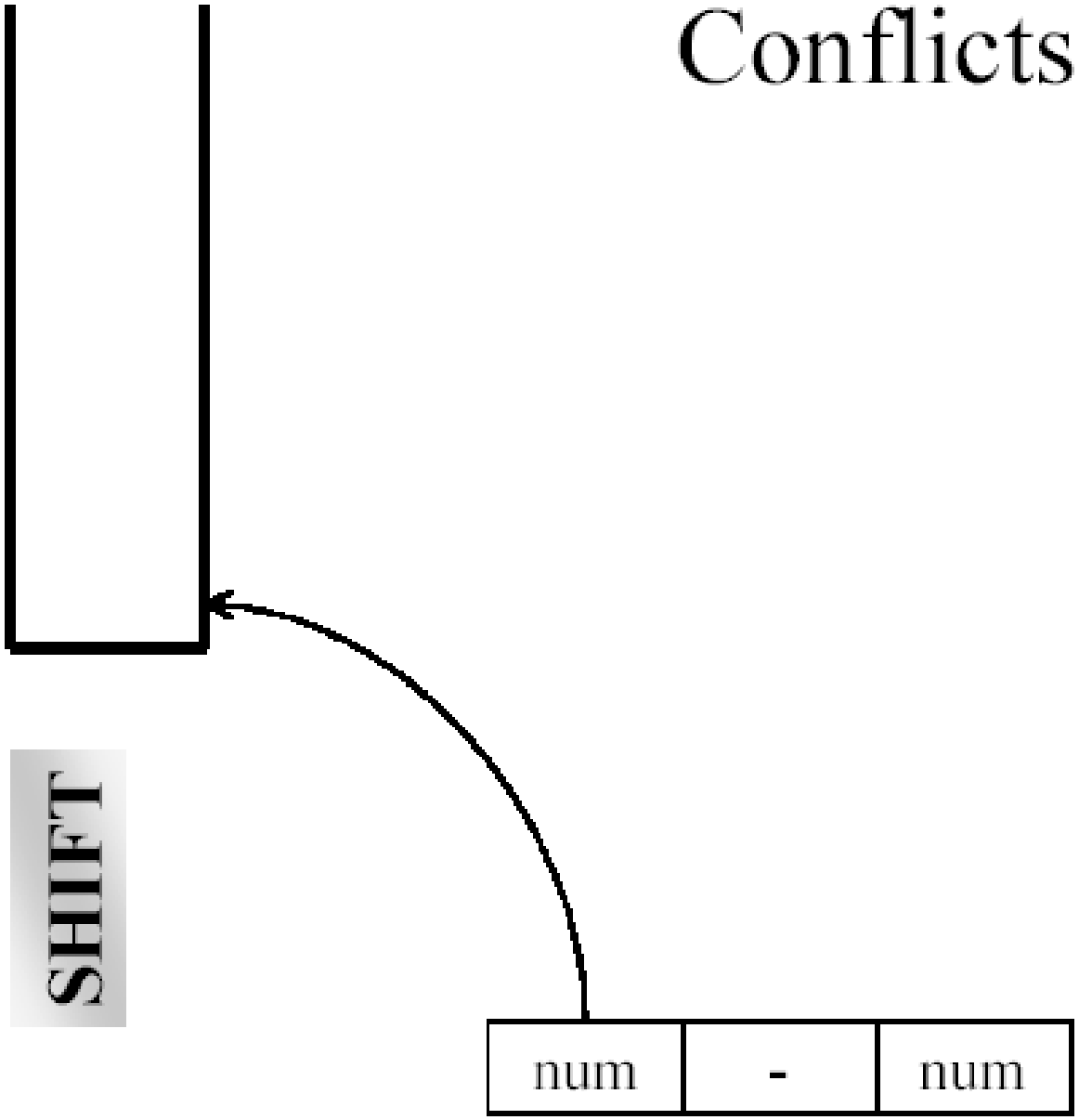
$Expr \rightarrow Expr$ -

$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *



Conflicts

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

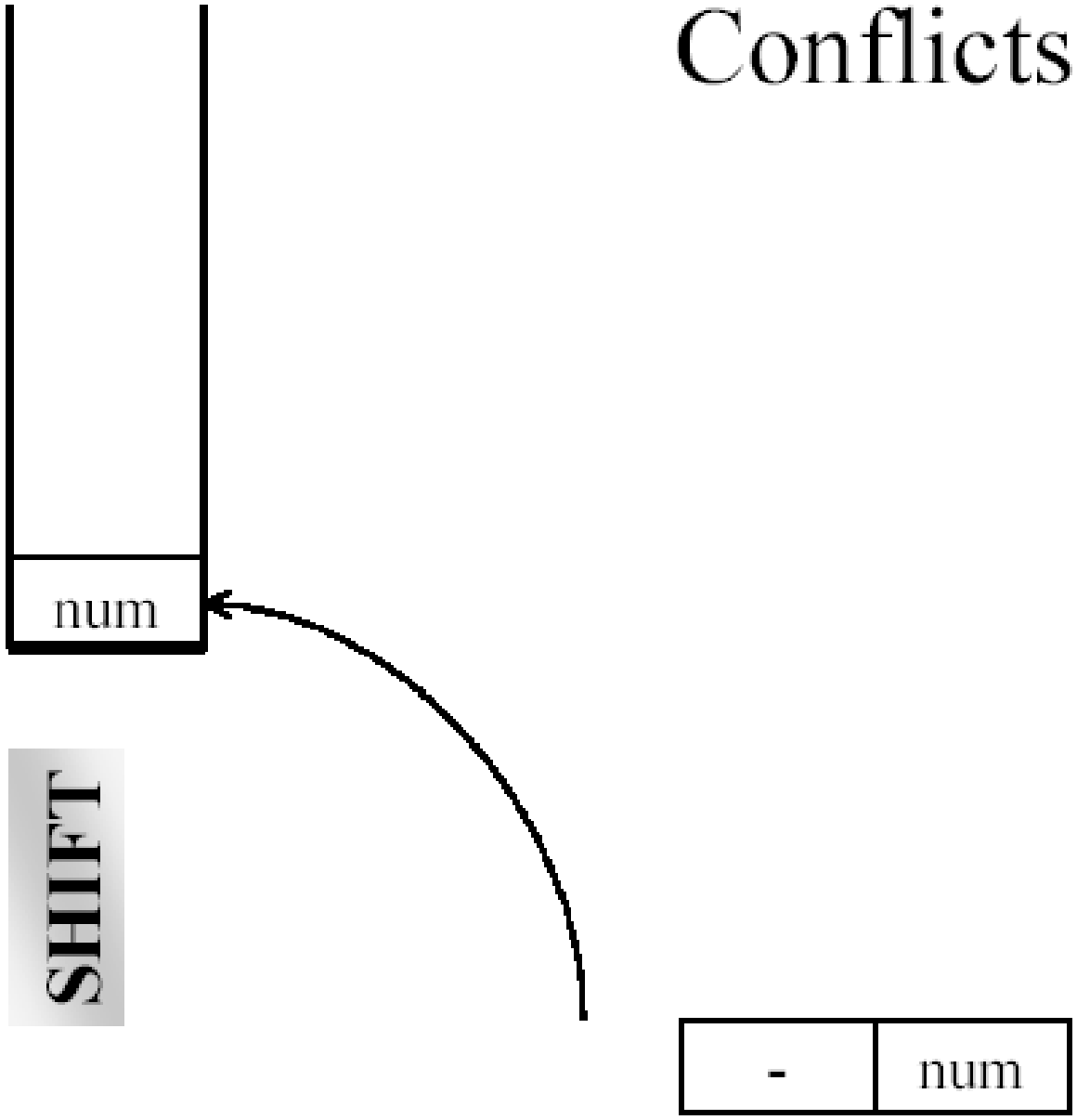
$Expr \rightarrow Expr$ -

$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *



Conflicts

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow (Expr)$

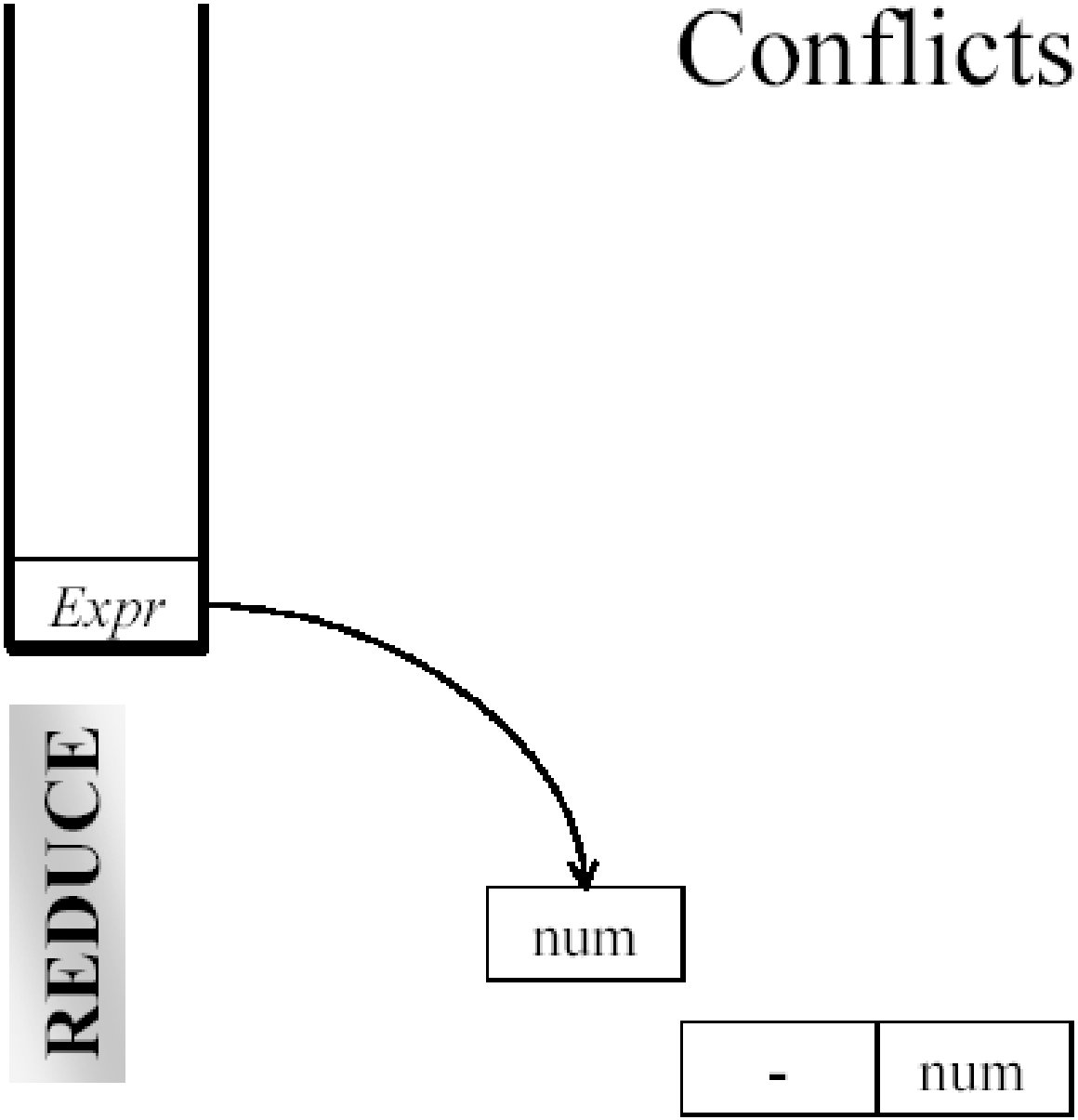
$Expr \rightarrow Expr$ -

$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow$ -

$Op \rightarrow *$



Conflicts

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

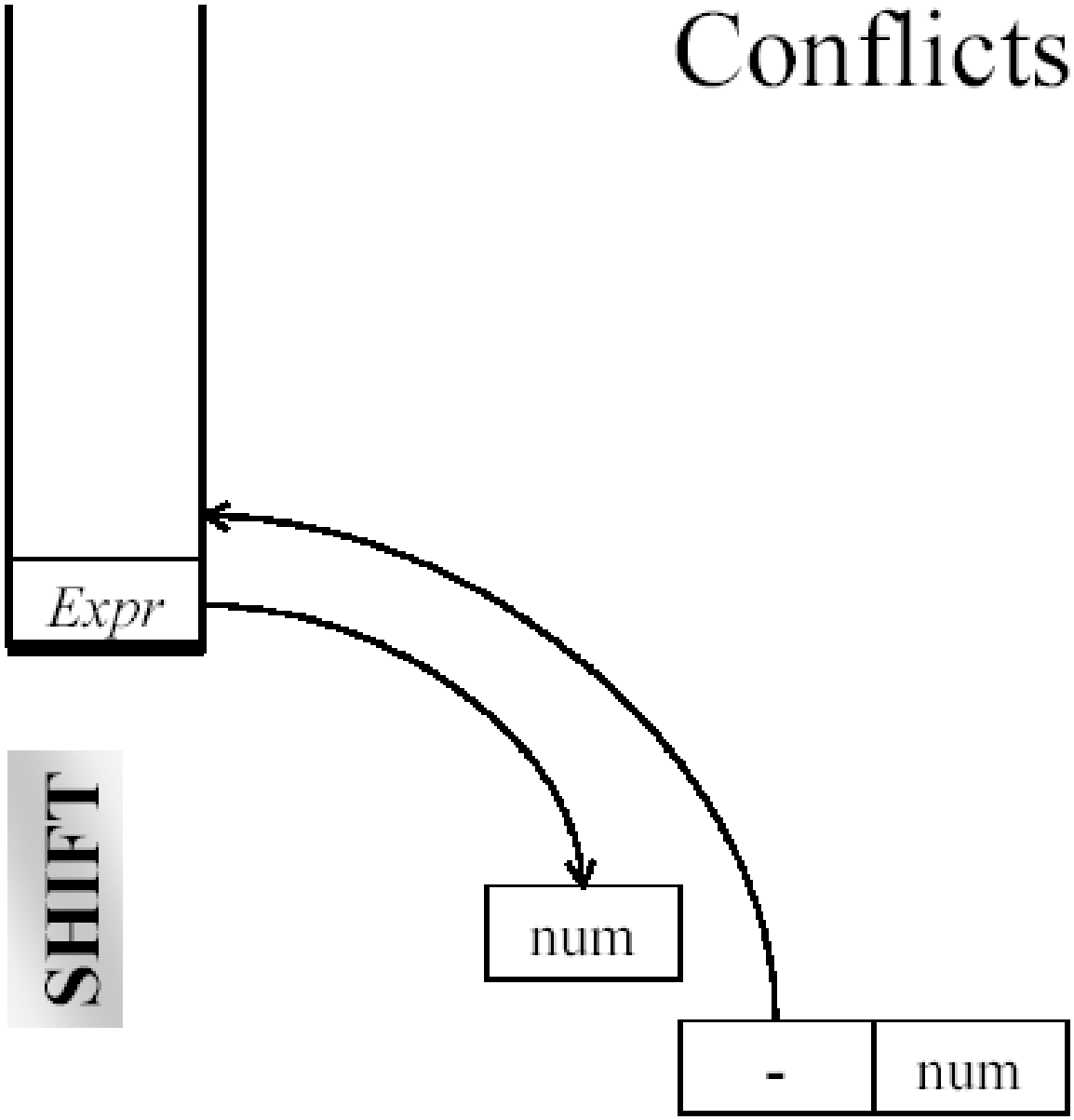
$Expr \rightarrow Expr$ -

$Expr \rightarrow$ num

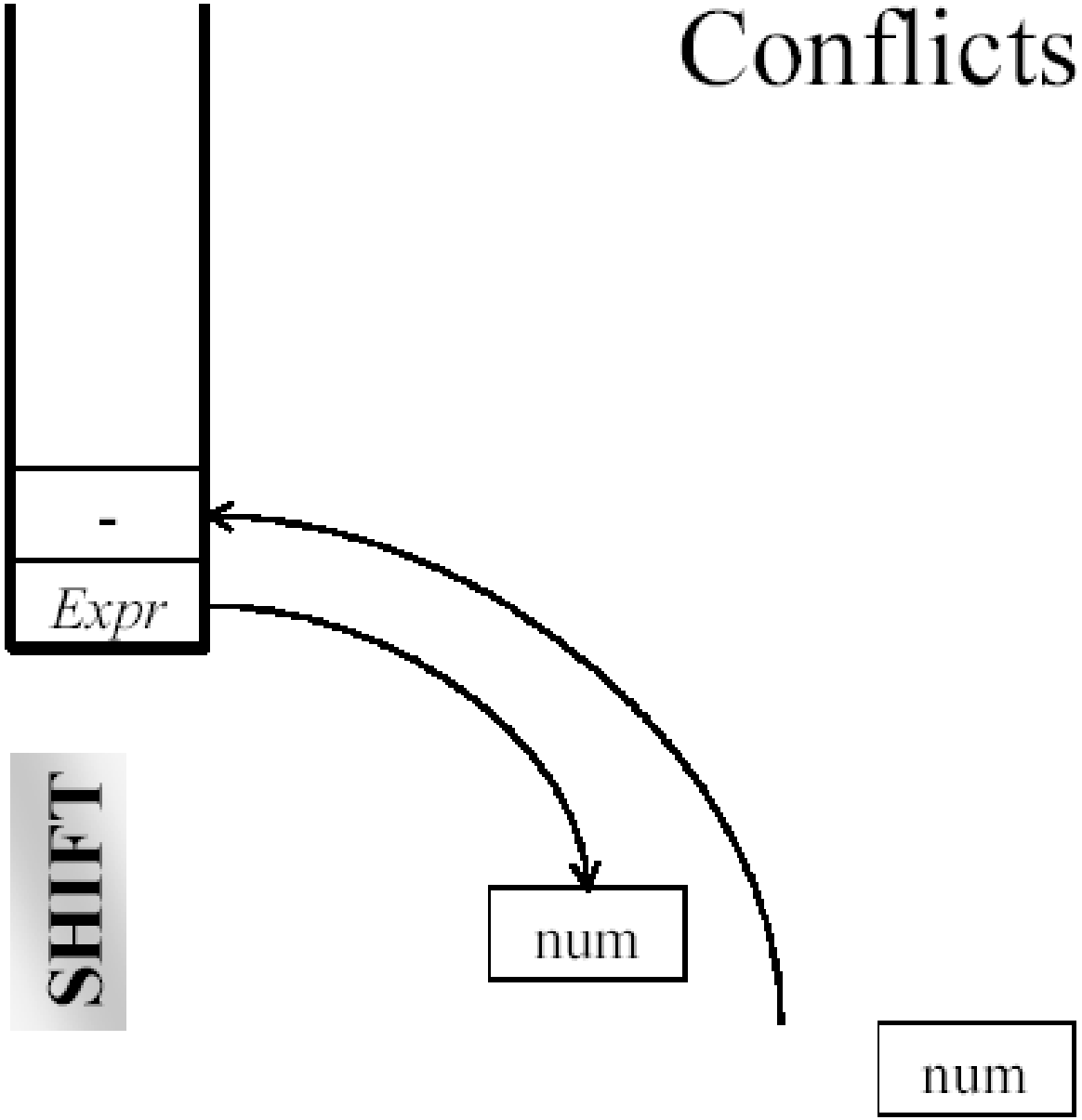
$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *



Conflicts



$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow Expr$ - $\ Expr$

$Expr \rightarrow\ (Expr)$

$Expr \rightarrow\ Expr$ -

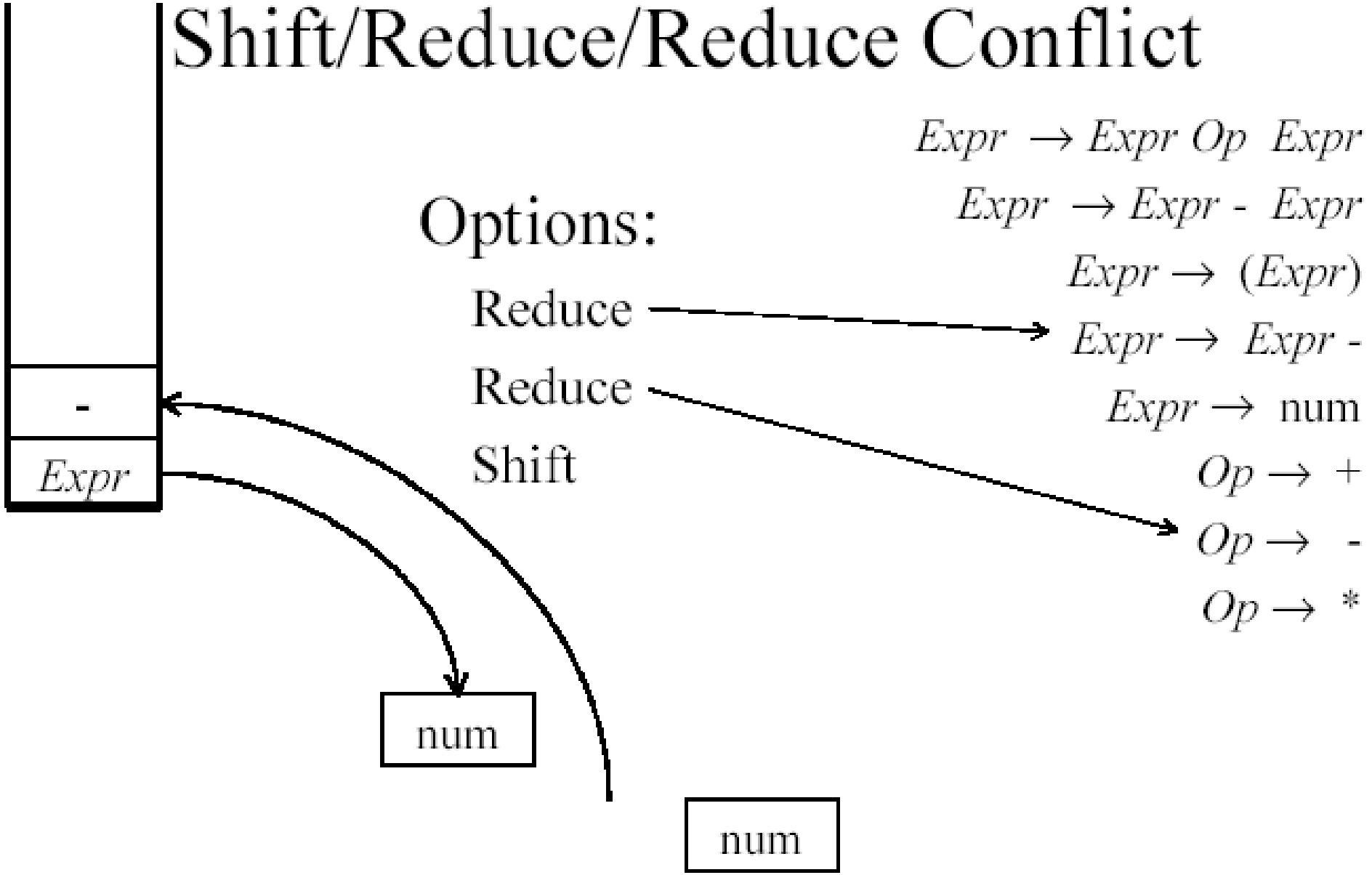
$Expr \rightarrow$ num

$Op \rightarrow\ +$

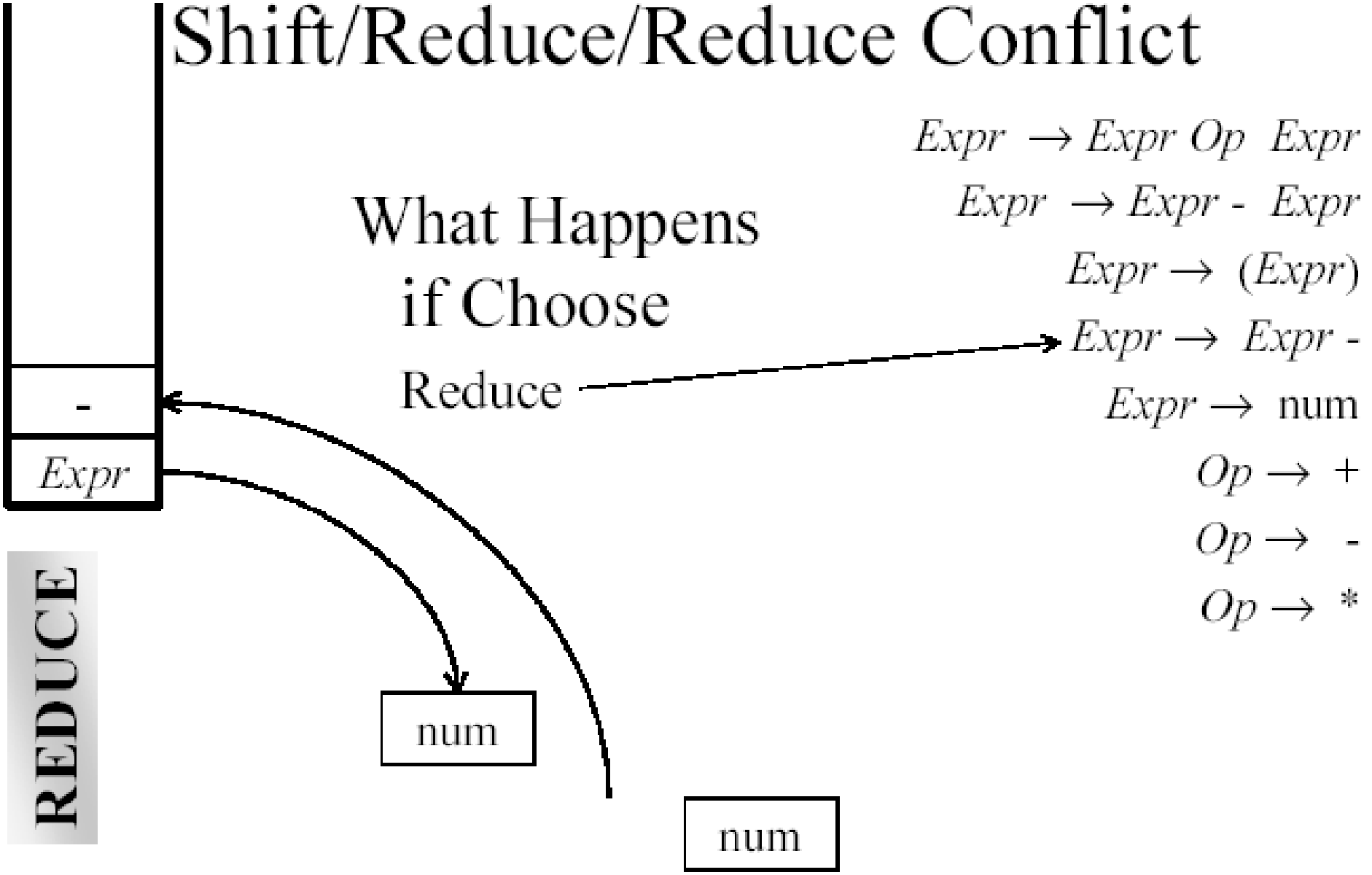
$Op \rightarrow\ $ -

$Op \rightarrow\ *$

Shift/Reduce/Reduce Conflict



Shift/Reduce/Reduce Conflict



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr\ -\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow Expr\ -$

$Expr \rightarrow num$

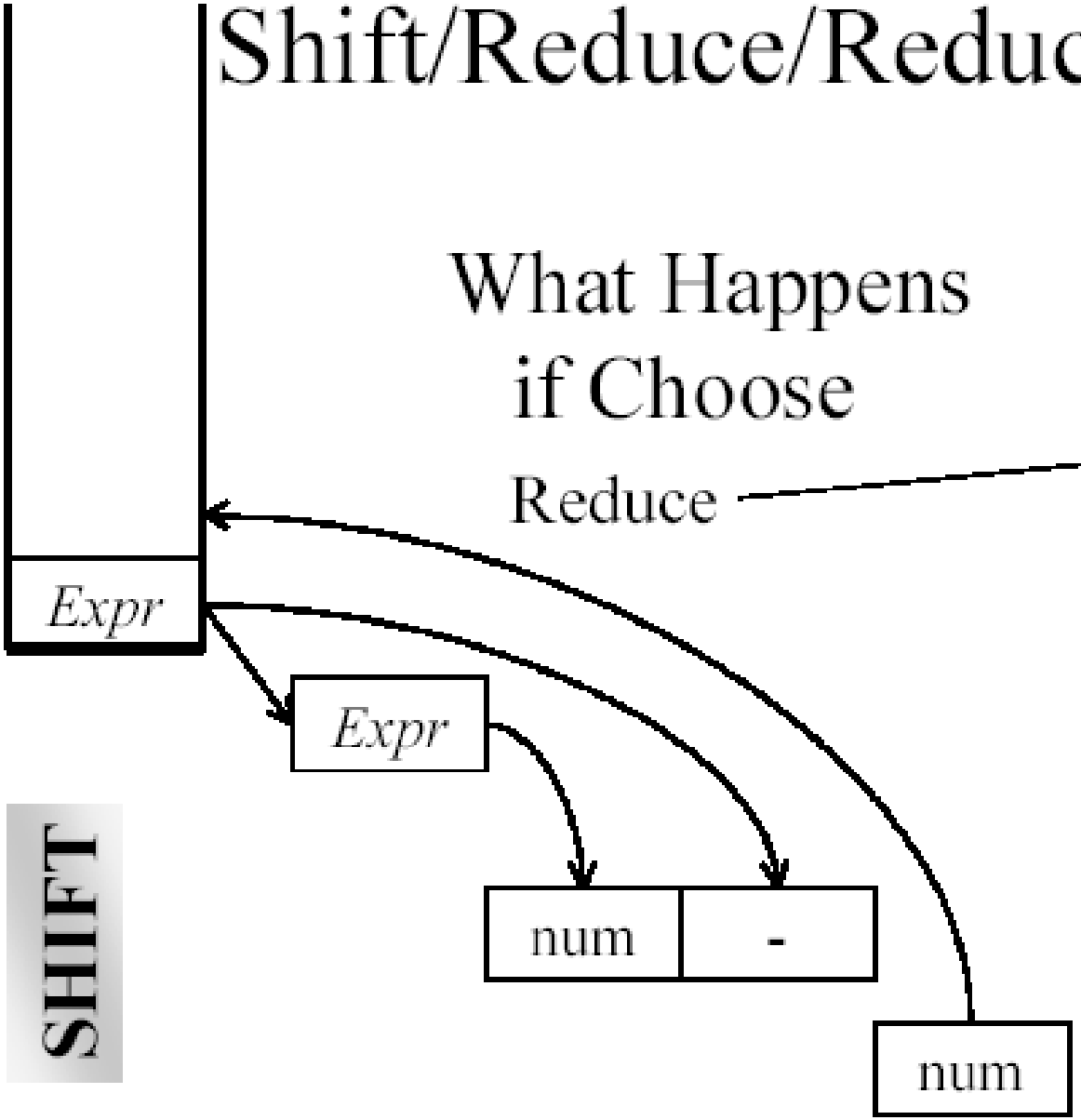
$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

What Happens if Choose

Reduce



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr\ -\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow Expr\ -$

$Expr \rightarrow num$

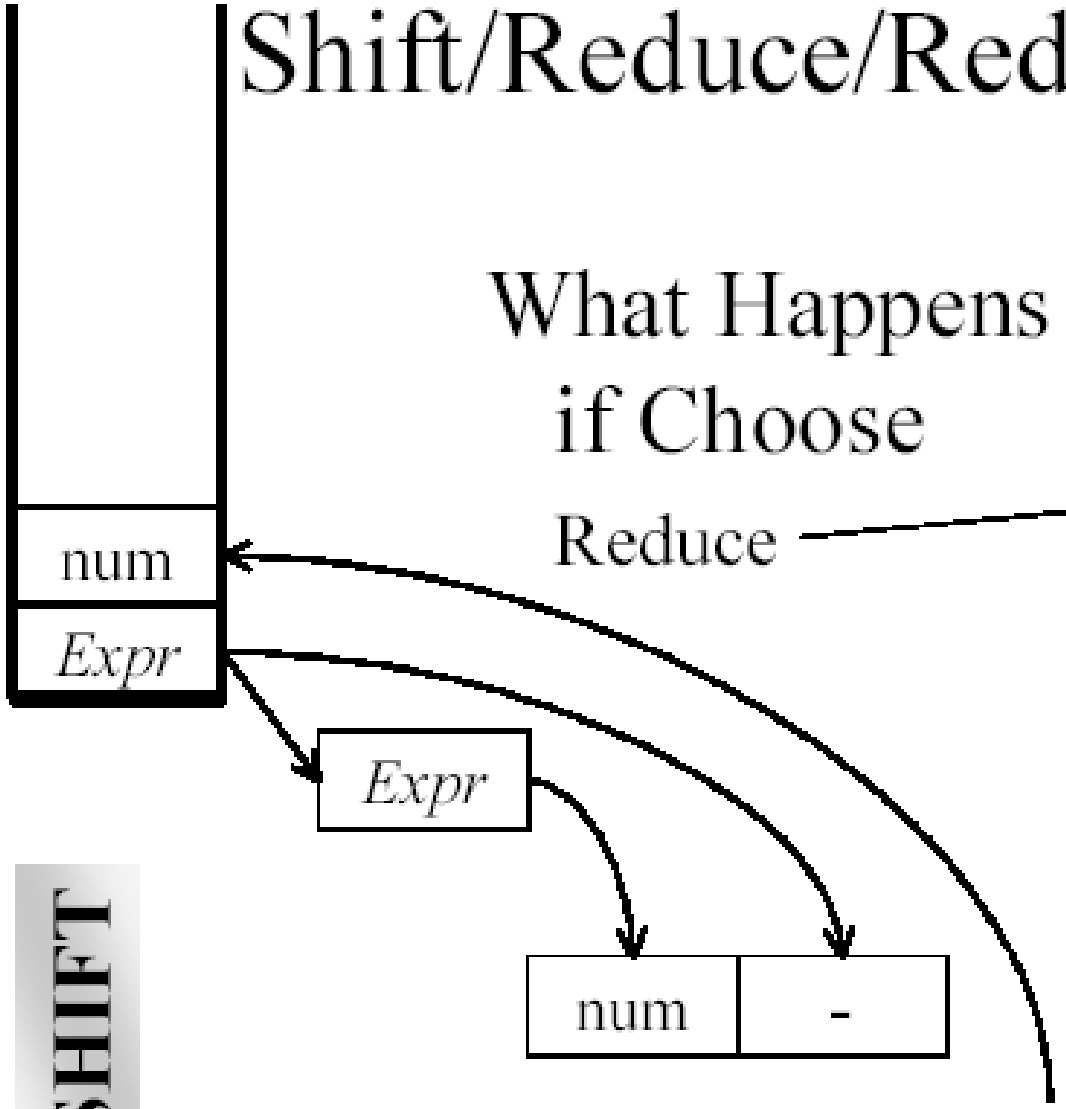
$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

What Happens if Choose

Reduce



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

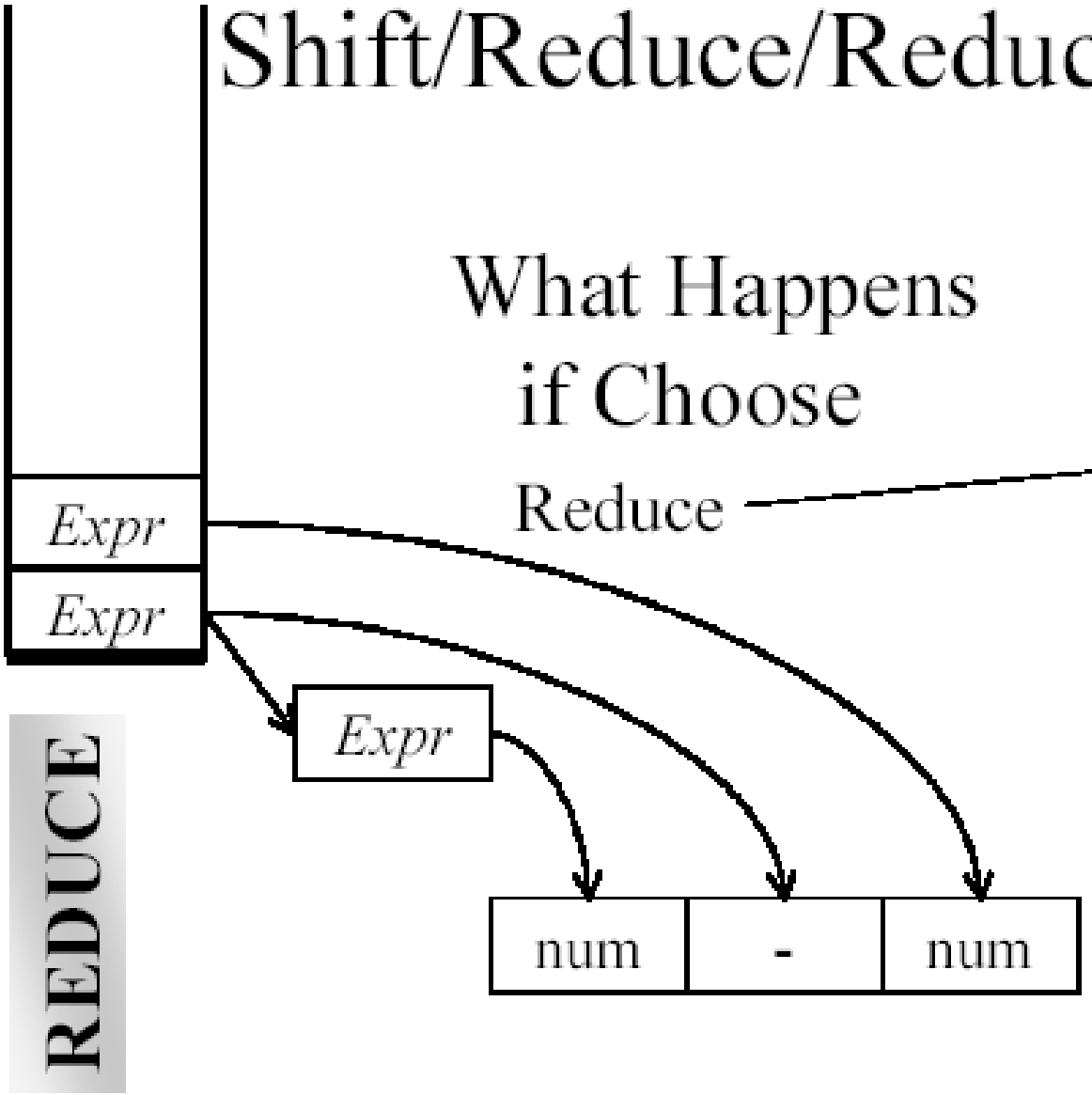
$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *

What Happens if Choose Reduce



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr\ -\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow Expr\ -$

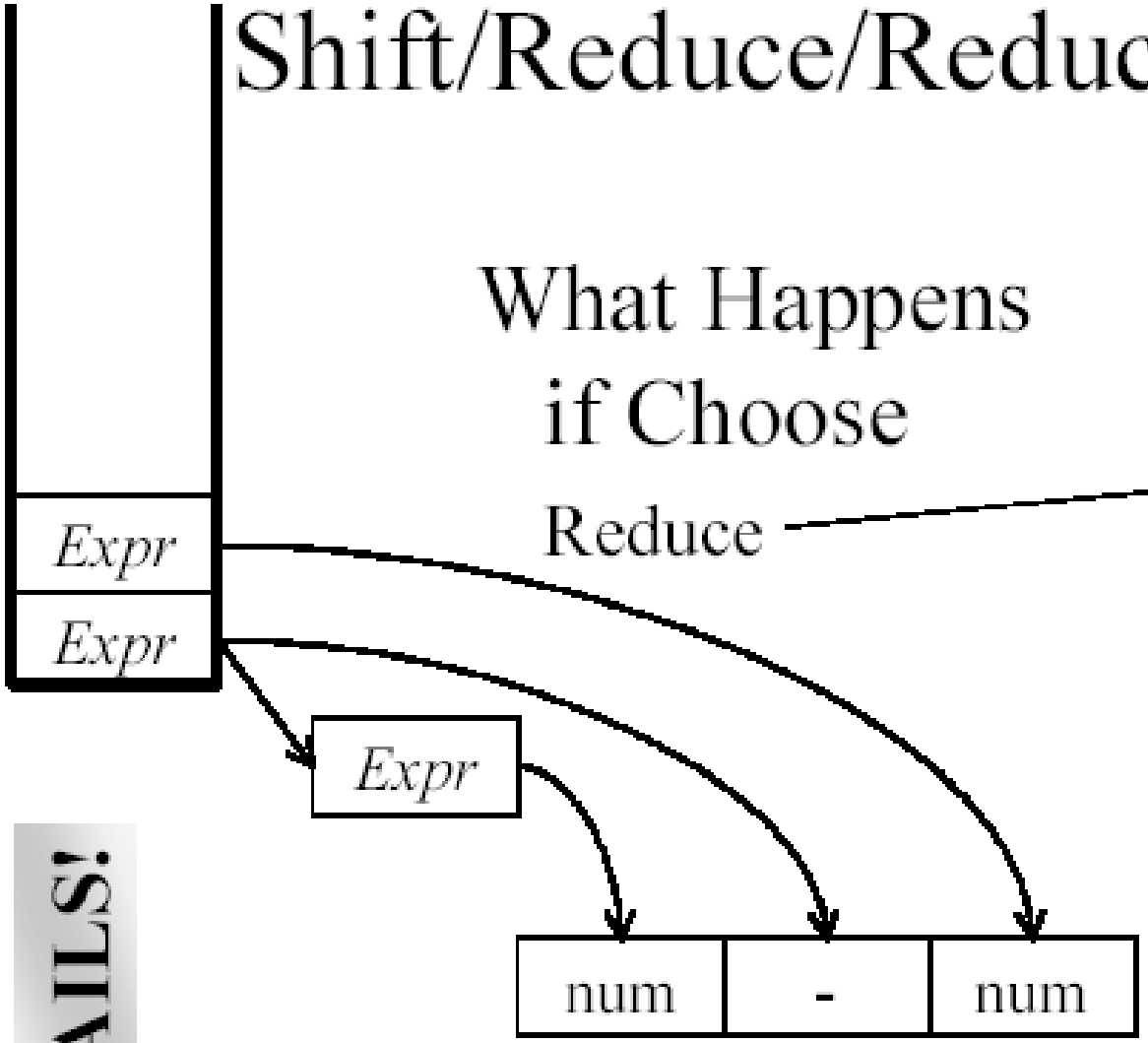
$Expr \rightarrow num$

$Op \rightarrow +$

$Op \rightarrow -$

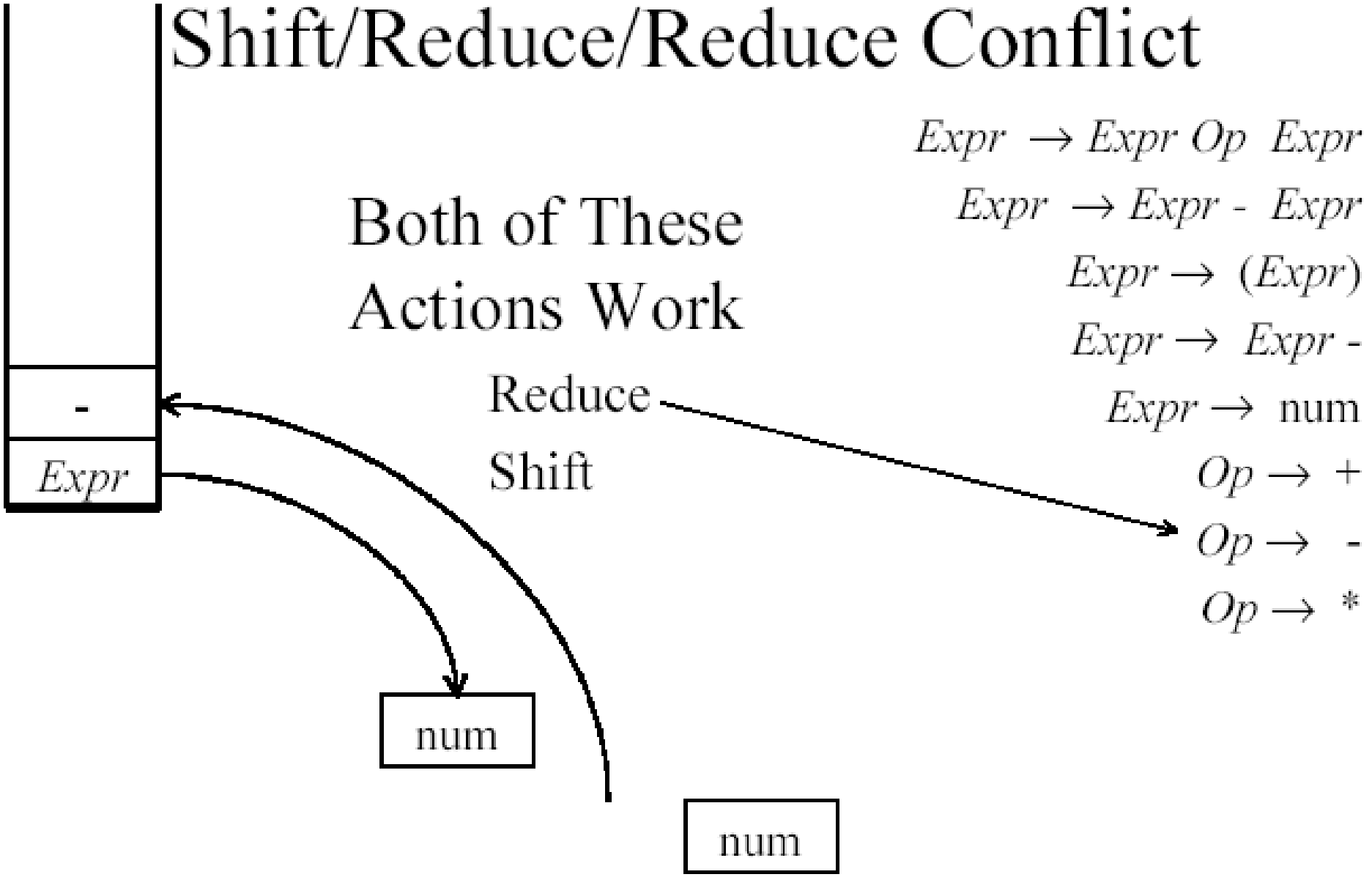
$Op \rightarrow *$

What Happens if Choose Reduce



FAILS!

Shift/Reduce/Reduce Conflict



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

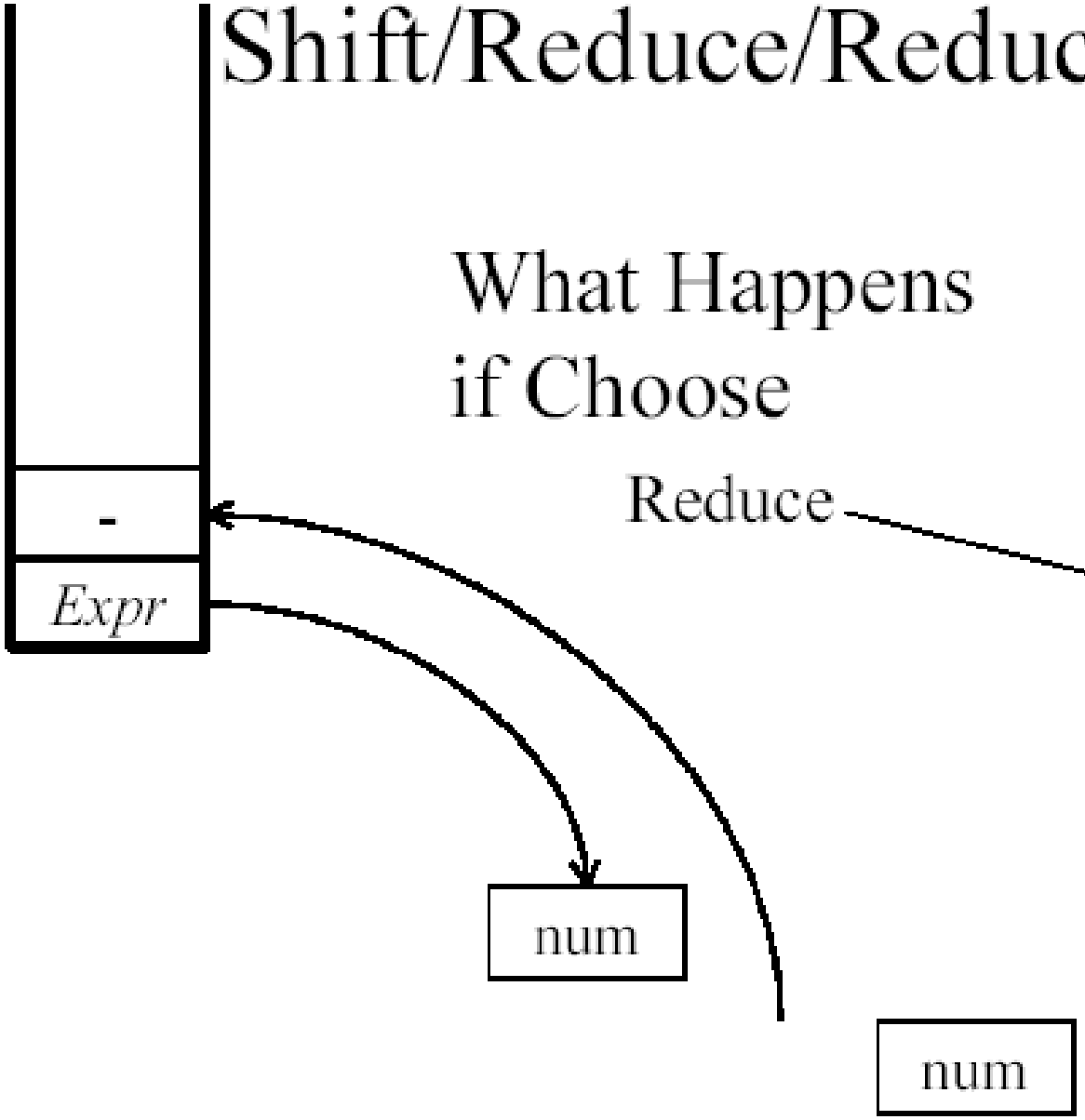
$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *

What Happens if Choose Reduce



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr\ -\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow Expr\ -$

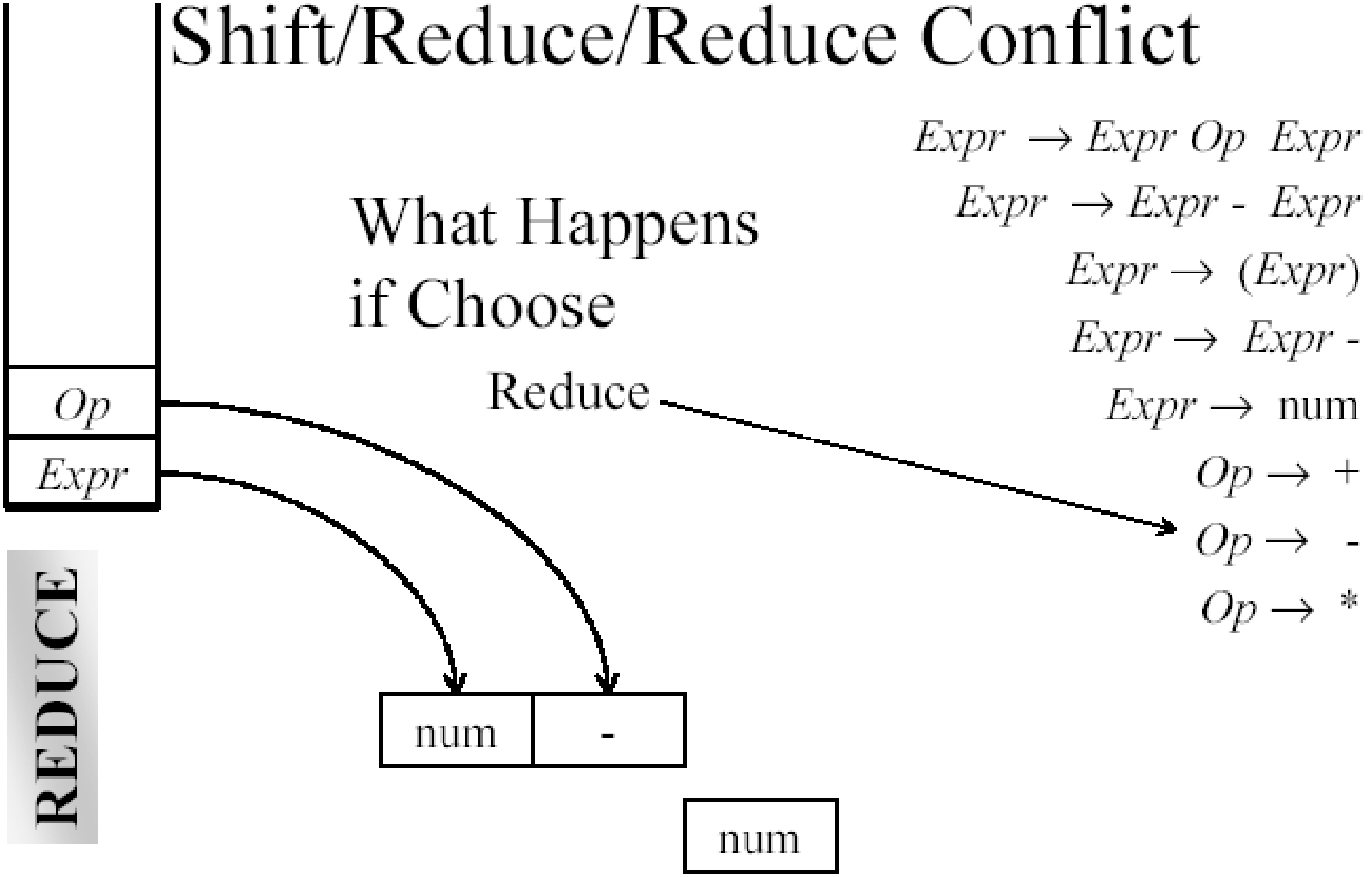
$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

What Happens if Choose



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow Expr\ -\ \ Expr$

$Expr \rightarrow\ (Expr)$

$Expr \rightarrow\ \ Expr\ -$

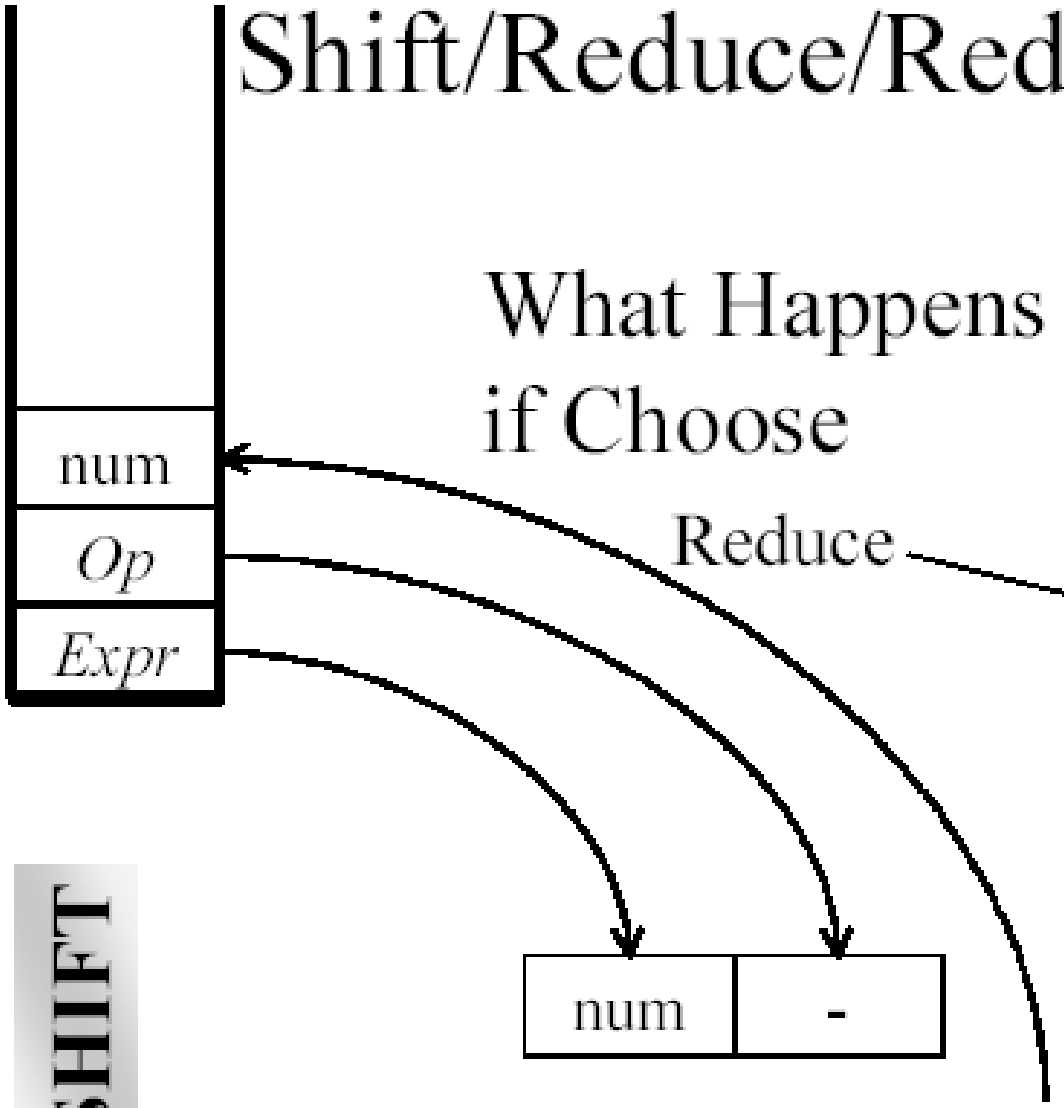
$Expr \rightarrow\ \ \text{num}$

$Op \rightarrow\ +$

$Op \rightarrow\ \ -$

$Op \rightarrow\ *$

What Happens if Choose



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr\ -\ Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow Expr\ -$

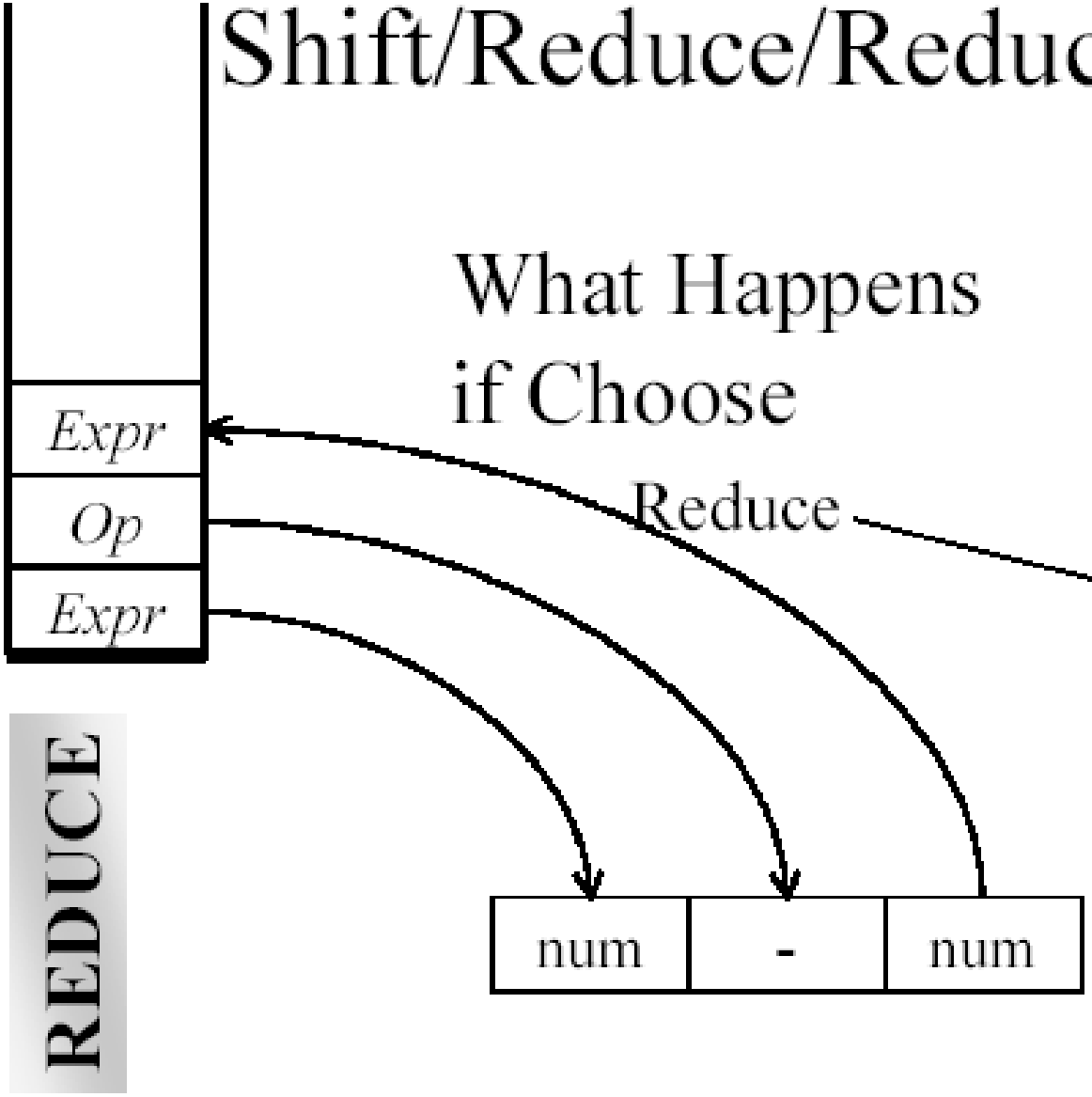
$Expr \rightarrow num$

$Op \rightarrow +$

$Op \rightarrow -$

$Op \rightarrow *$

What Happens if Choose Reduce



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

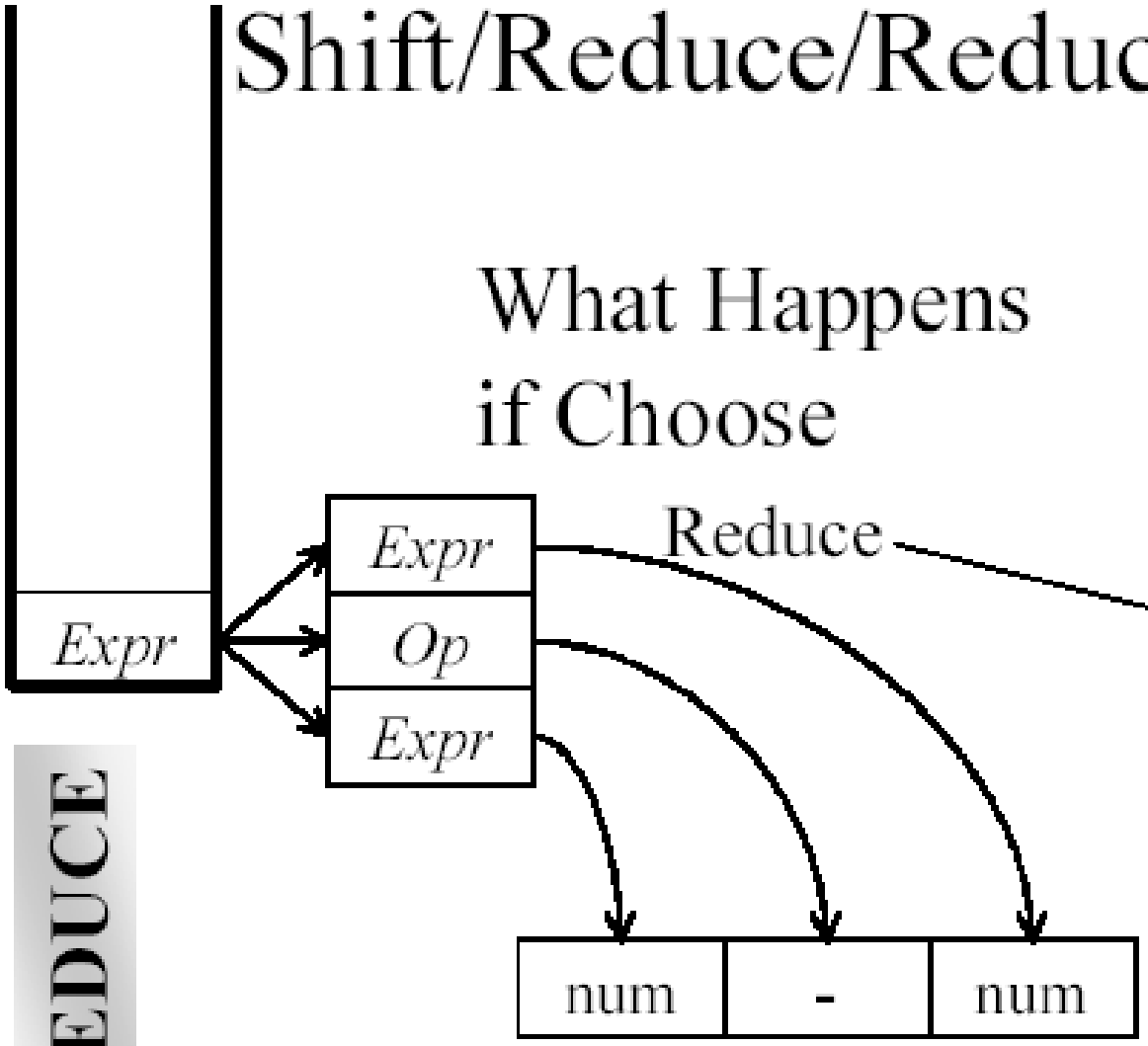
$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *

What Happens if Choose



Shift/Reduce/Reduce Conflict

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

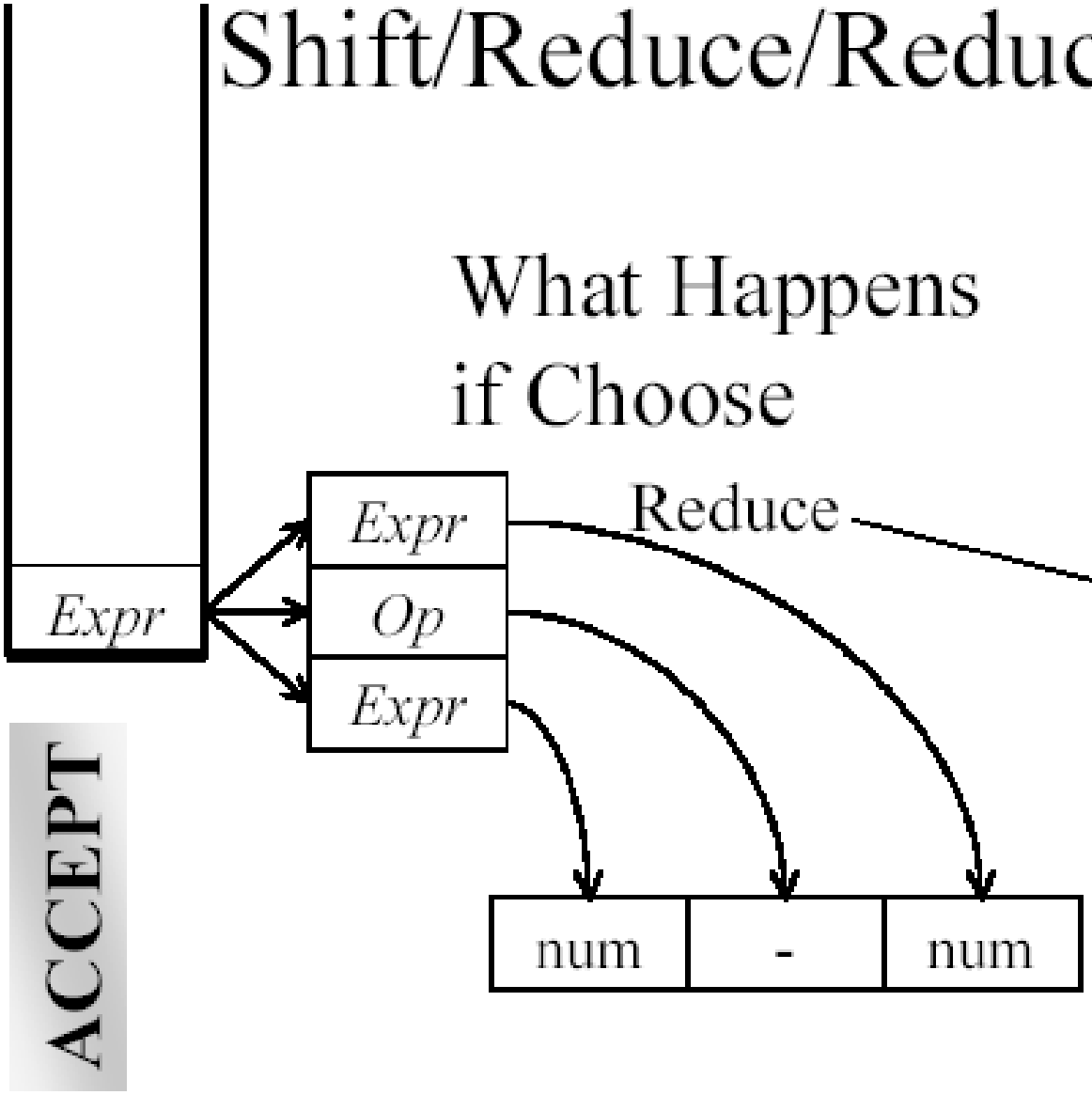
$Expr \rightarrow$ num

$Op \rightarrow$ +

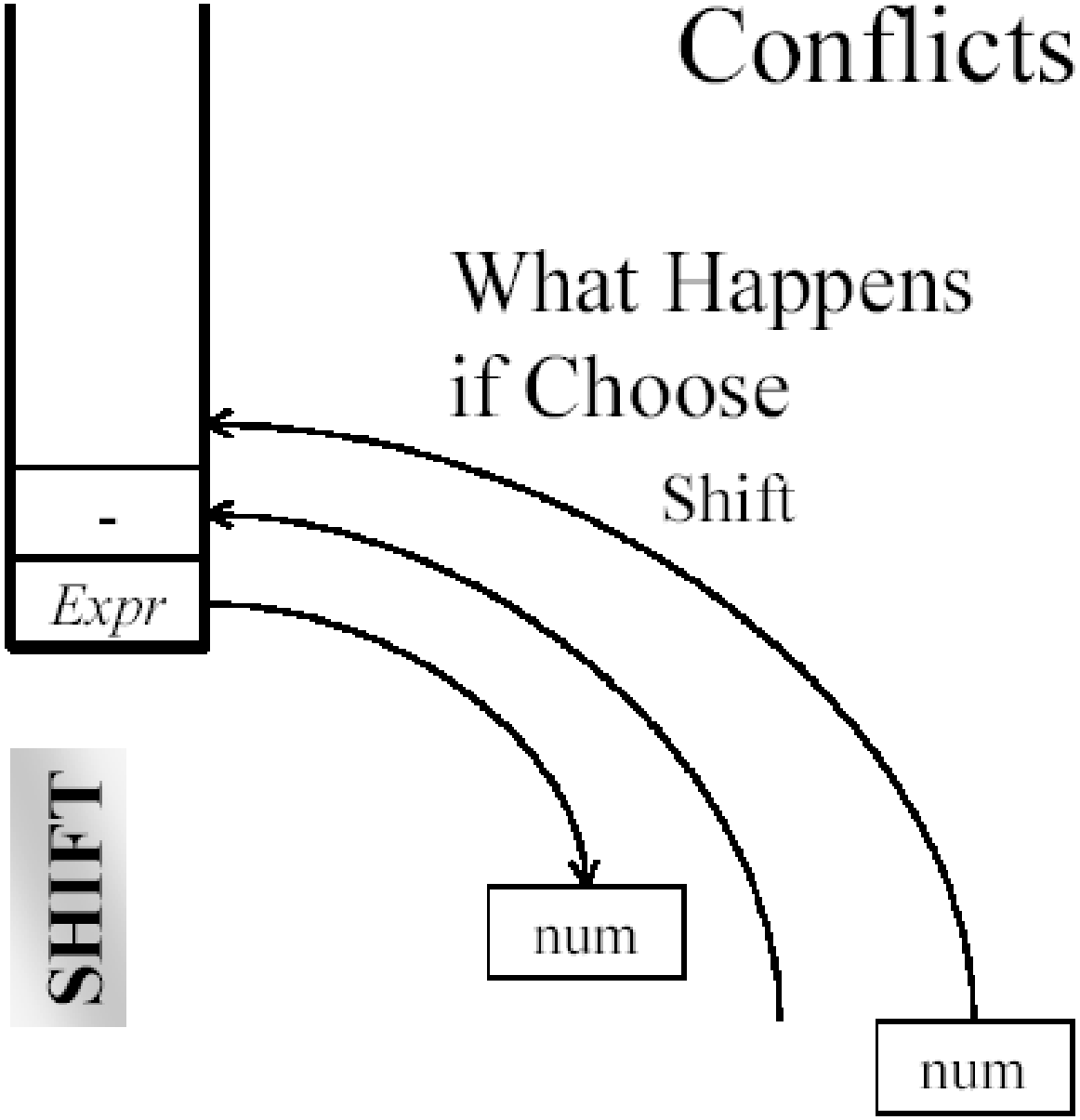
$Op \rightarrow$ -

$Op \rightarrow$ *

What Happens if Choose Reduce



Conflicts



$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow (Expr)$

$Expr \rightarrow Expr$ -

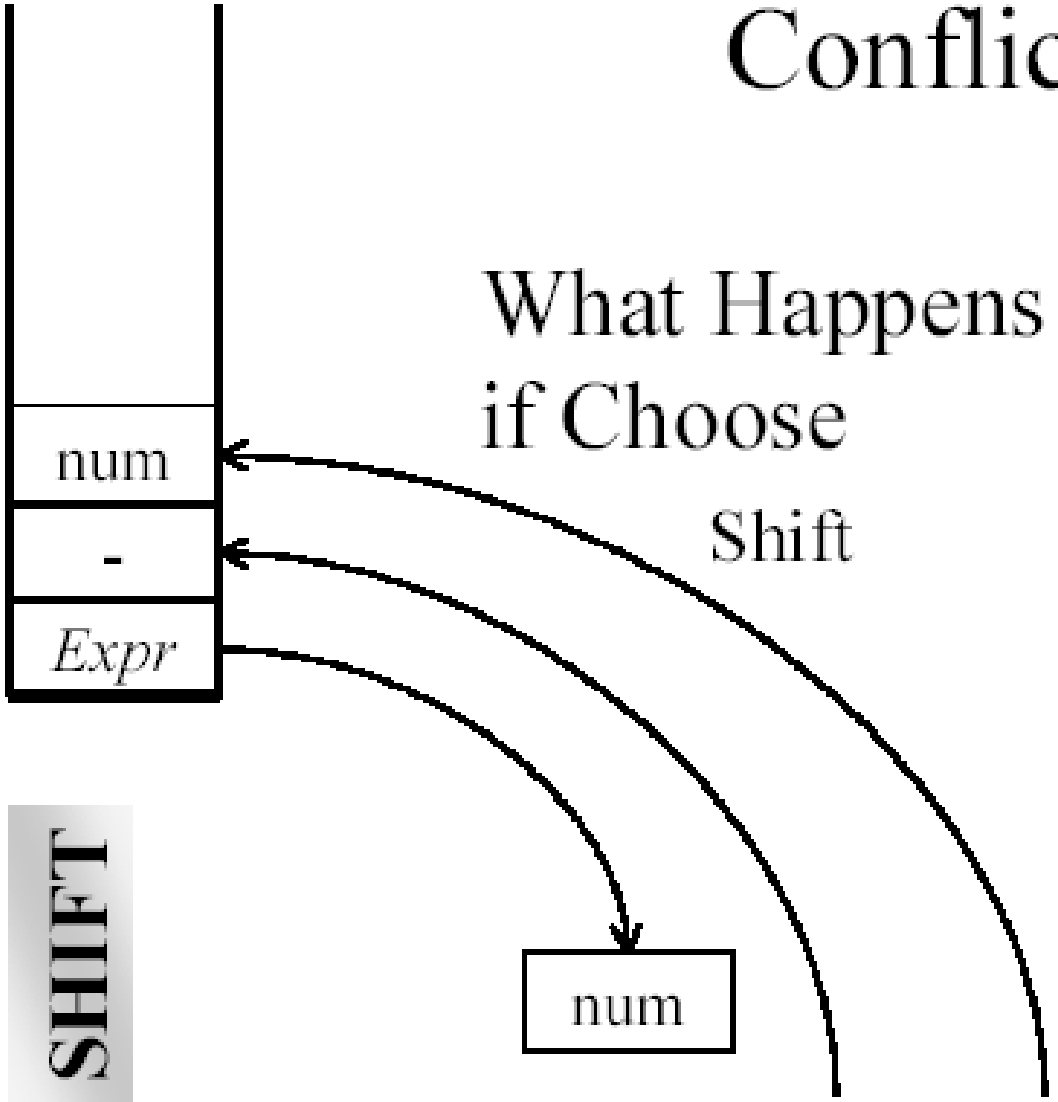
$Expr \rightarrow$ num

$Op \rightarrow +$

$Op \rightarrow$ -

$Op \rightarrow *$

Conflicts



$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

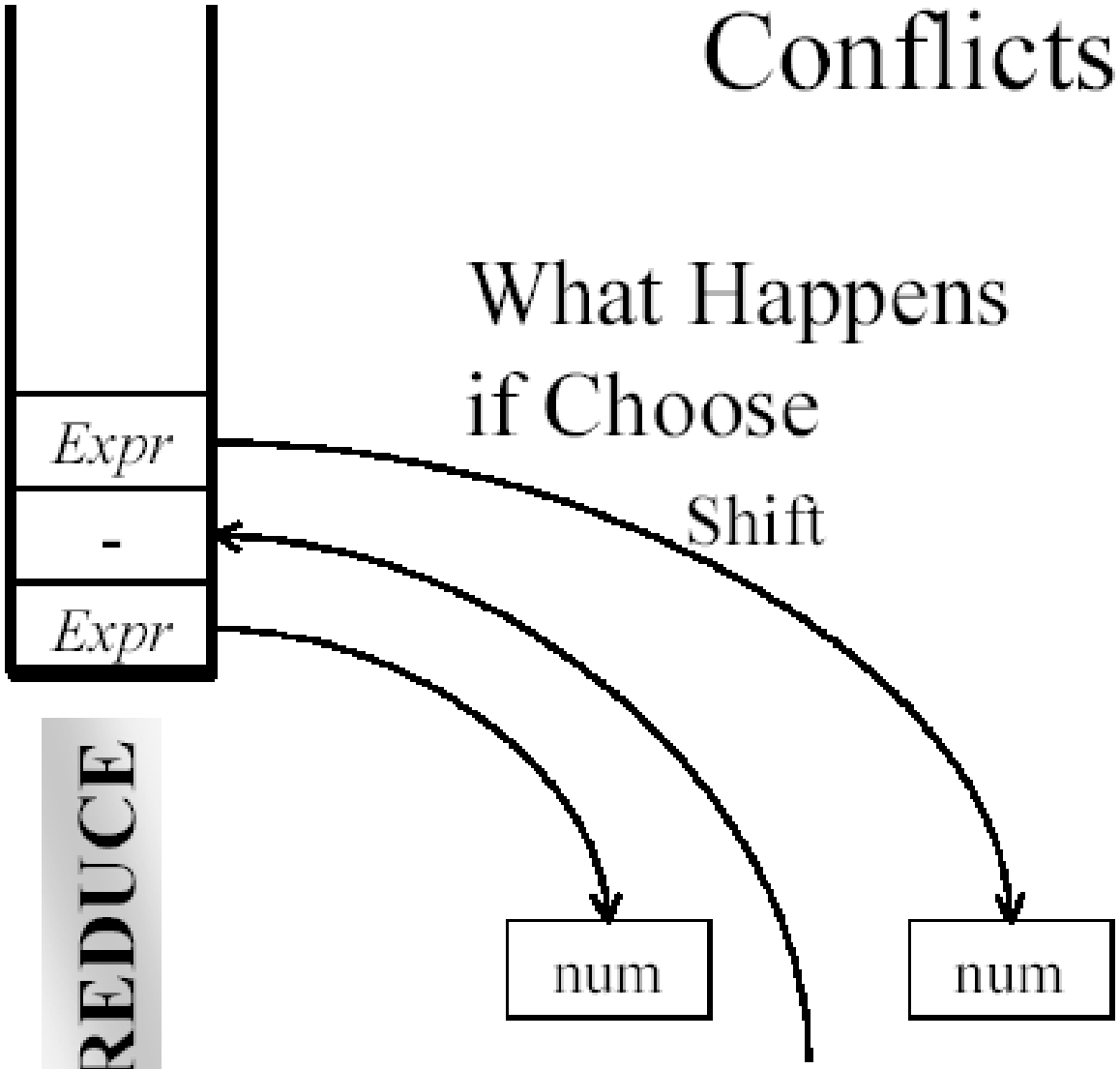
$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *

Conflicts



$Expr \rightarrow Expr\ Op\ \ Expr$

$Expr \rightarrow Expr\ -\ \ Expr$

$Expr \rightarrow\ (Expr)$

$Expr \rightarrow\ \ Expr\ -$

$Expr \rightarrow$ num

$Op \rightarrow\ +$

$Op \rightarrow\ \ -$

$Op \rightarrow\ *$

Conflicts

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

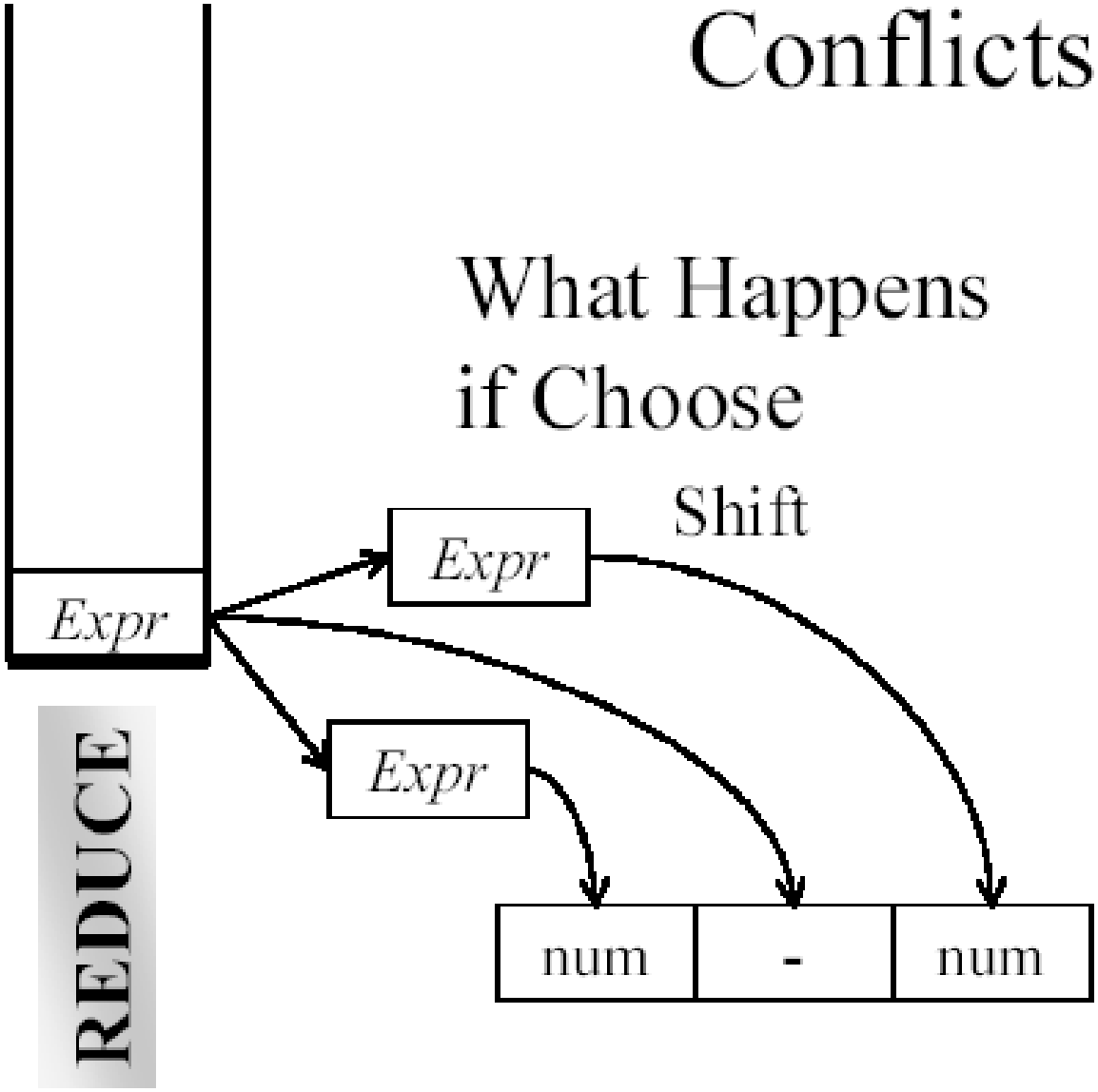
$Expr \rightarrow$ num

$Op \rightarrow$ +

$Op \rightarrow$ -

$Op \rightarrow$ *

What Happens if Choose



Conflicts

$Expr \rightarrow Expr\ Op\ Expr$

$Expr \rightarrow Expr$ - $Expr$

$Expr \rightarrow$ ($Expr$)

$Expr \rightarrow Expr$ -

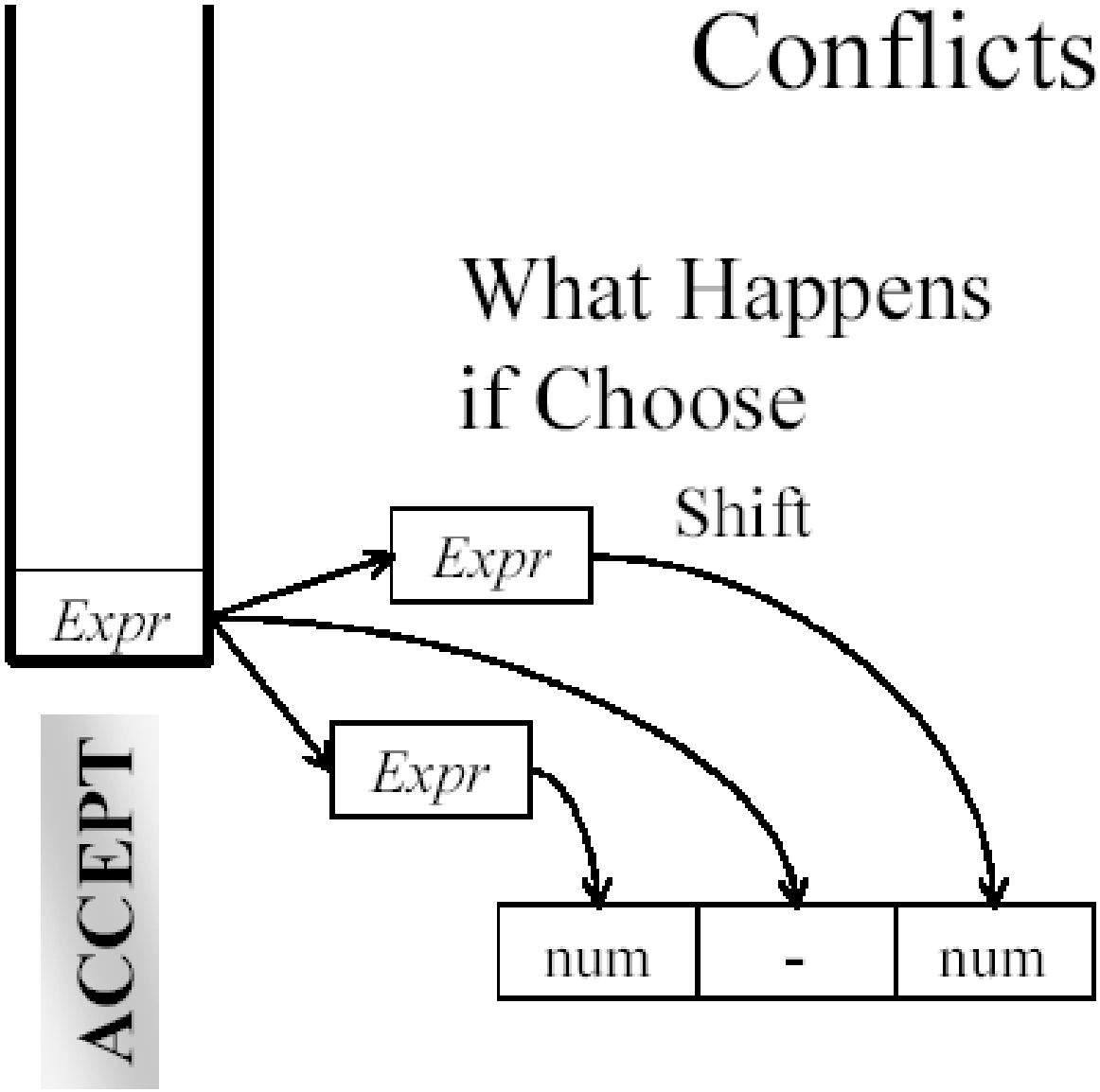
$Expr \rightarrow$ num

$Op \rightarrow$ +

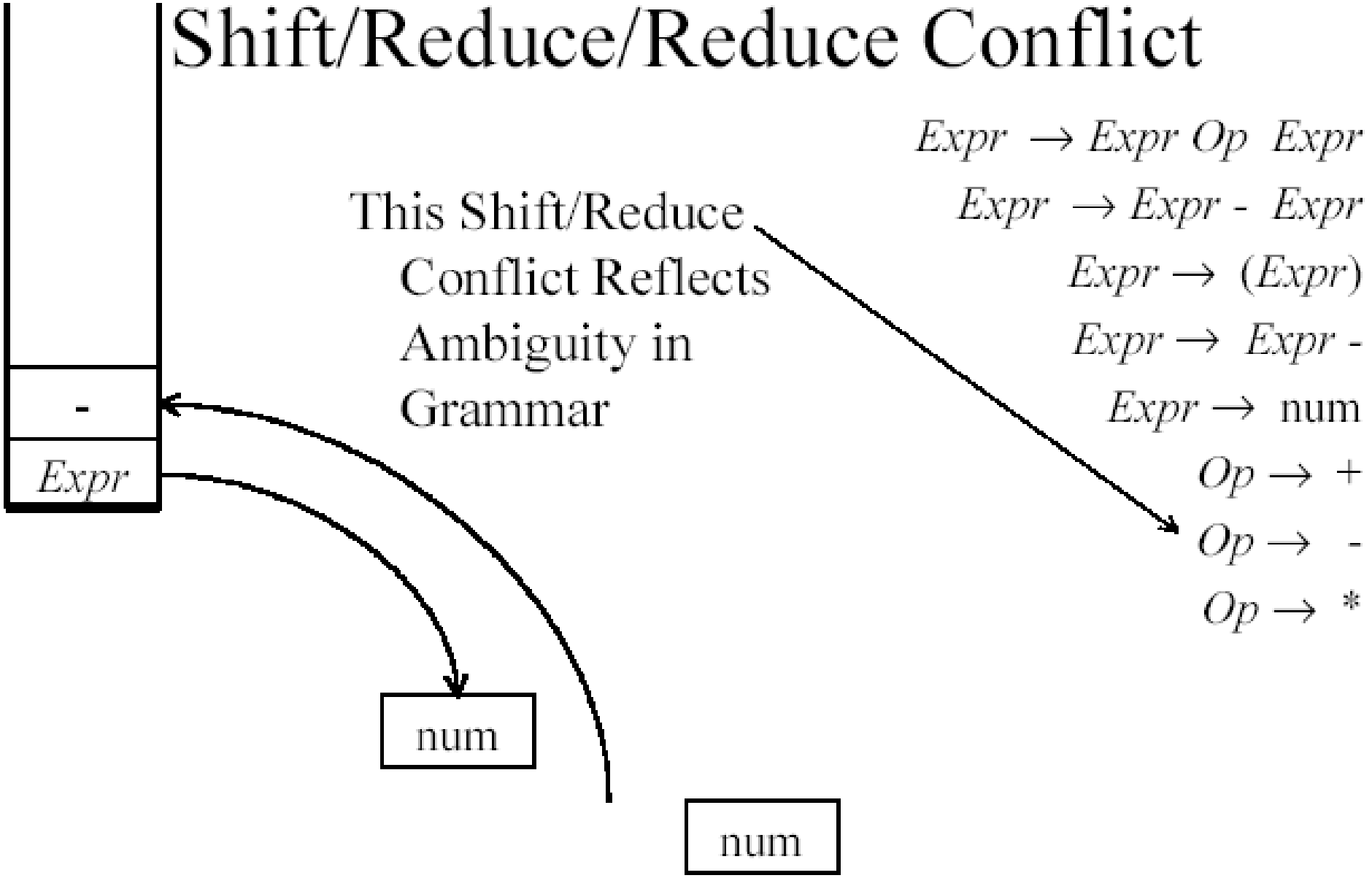
$Op \rightarrow$ -

$Op \rightarrow$ *

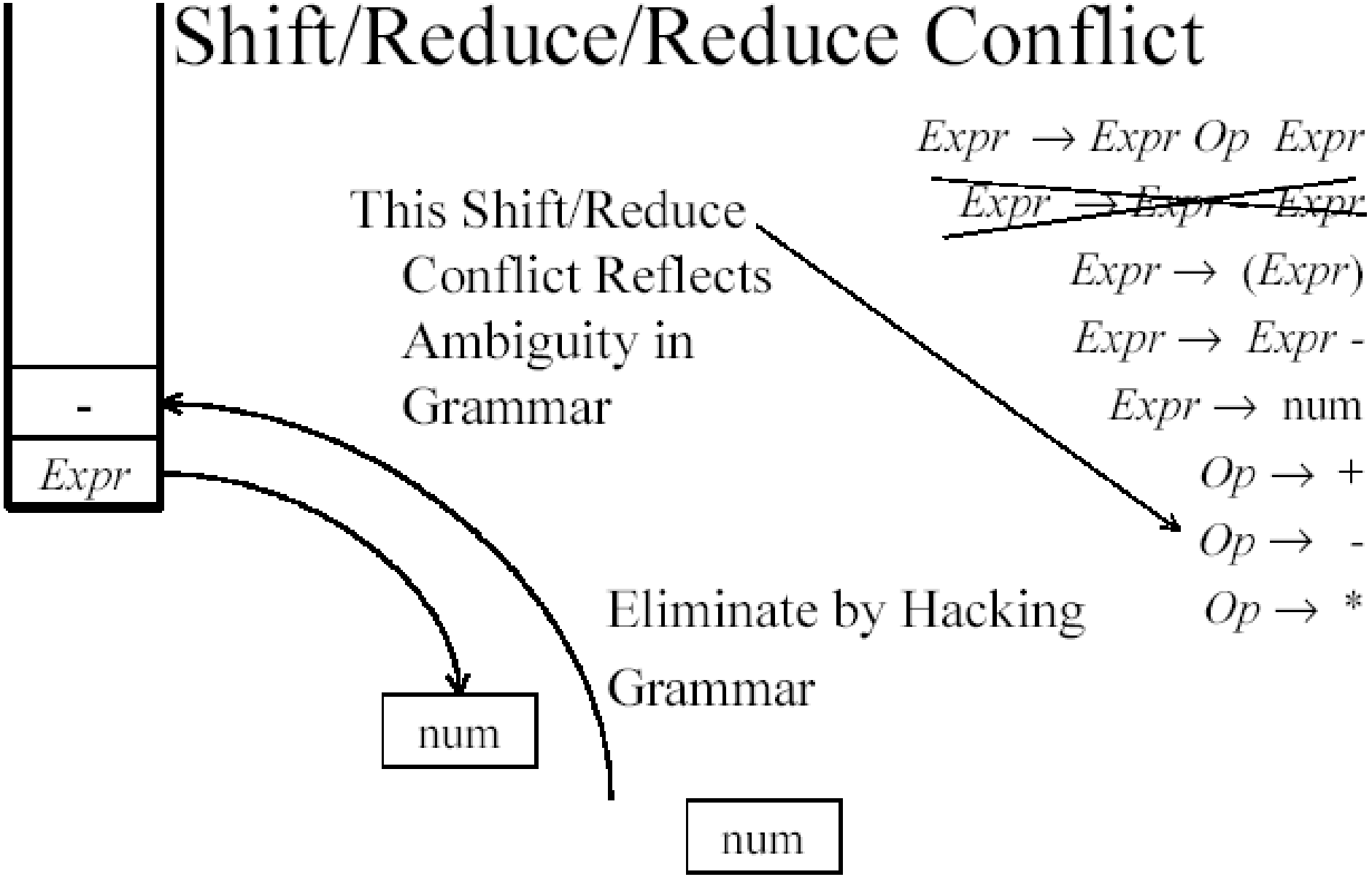
What Happens if Choose Shift



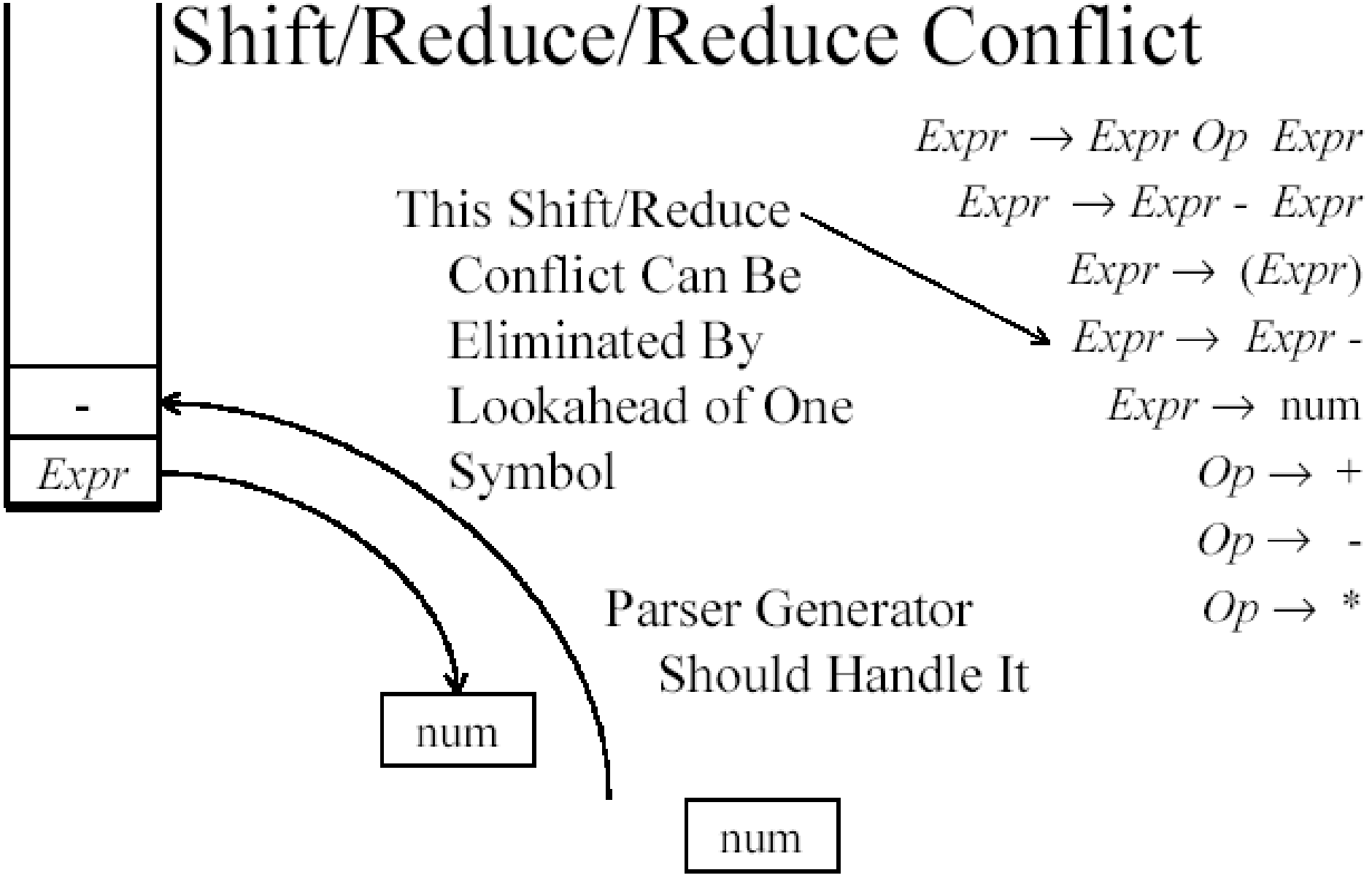
Shift/Reduce/Reduce Conflict



Shift/Reduce/Reduce Conflict



Shift/Reduce/Reduce Conflict



观 察

- “归约-归约”冲突：当前栈顶部分与多个产生式右部匹配：
  - 一个可归约串匹配多个变元的候选式，或者，
  - 多个可归约串匹配多个候选式。
- “移进-归约”冲突：当可归约串出现时可以归约也可以继续移进。

- 造成冲突的原因是什么呢？
  - 只能在文法上找原因。
  - 文法有二义性才造成此类冲突？
  - 与当前输入符号有关？
  - 与当前输入符号及其随后的k-1个符号有关？
  - 通过消除文法二义性、向前查看k个符号，几乎避免了冲突的发生？

8.2.3 规范归约

- 规范推导 = 最右推导、
- 规范句型 = 右句型， 若$S \Rightarrow_{rm}^{*} \beta$那么$\beta$是规范句型

- 规范归约 = 规范推导的逆，用符号 $\mapsto$ 表示。

- 直接最右推导

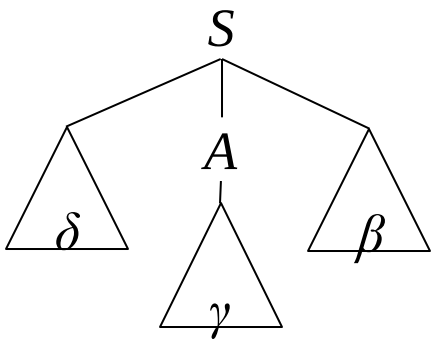
$$\delta Ax \Rightarrow_{rm} \delta\gamma x, (A, \gamma) \in \mathcal{P}$$

的逆是：

$$\delta\gamma x \mapsto \delta Ax, (A, \gamma) \in \mathcal{P}$$

短语与直接短语 *定义可归约串*

- 短语：对于句型$\delta A\beta$若有$A \Rightarrow^{+} \gamma$则称γ是句型$\delta\gamma\beta$的短语，是相对于变元A的短语。
- 直接短语：对于句型$\delta A\beta$若有$A \Rightarrow \gamma$则称γ是句型$\delta\gamma\beta$的直接短语，是相对于$A\rightarrow\gamma$的直接短语。



句型的语法树中每个子树的产物都是一个短语

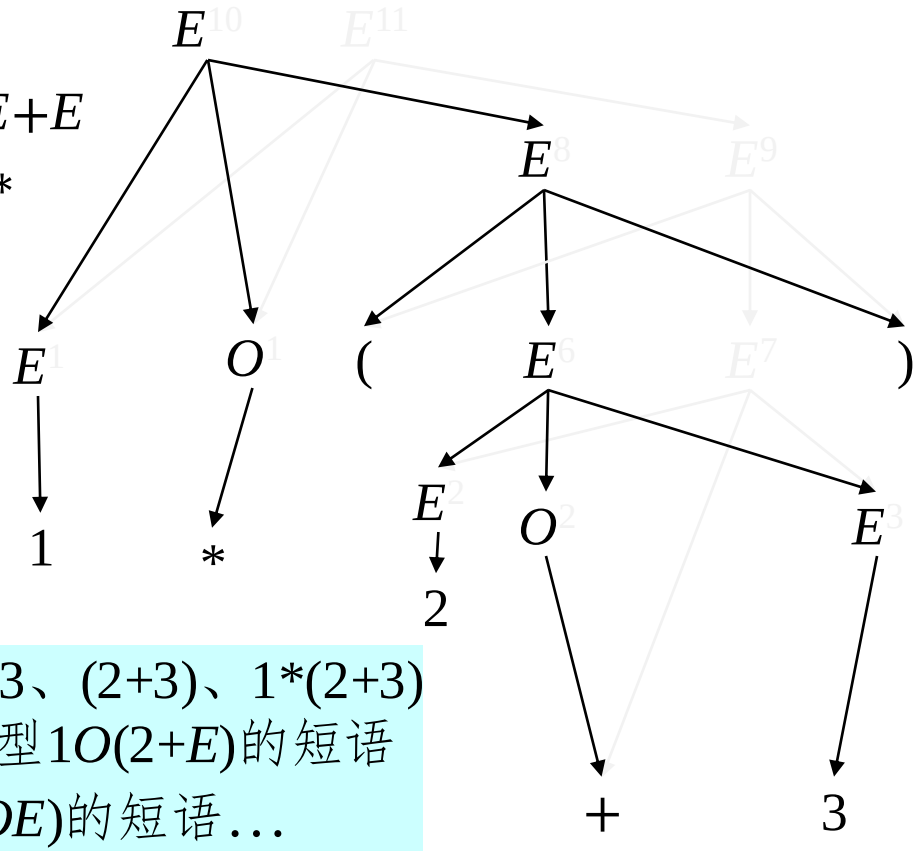
$E \longrightarrow EOE \quad E \longrightarrow E+E$

$E \longrightarrow (E) \quad E \longrightarrow E^{*}$

$E \longrightarrow i \quad O \longrightarrow +$

$O \longrightarrow * \quad U \longrightarrow +$

$E \longrightarrow Ui$



句子1*(2+3)的短语：1、*、2、+、3、2+3、(2+3)、1*(2+3)

1、2、+、2+*E*、(2+*E*)、1*O*(2+*E*)都是句型1*O*(2+*E*)的短语

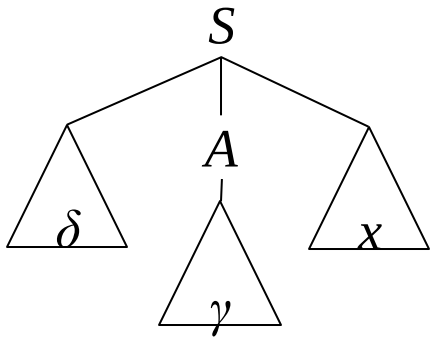
EOE、(*EOE*)、*EO*(*EOE*)都是句型*EO*(*EOE*)的短语…

带下划线的都是直接短语，也都是可归约串。

句柄

- 对于CFG (V, T, P, S)，若有规范句型δAx且其最右推导为 $\delta\gamma x$，则称γ是$\delta\gamma x$的句柄。

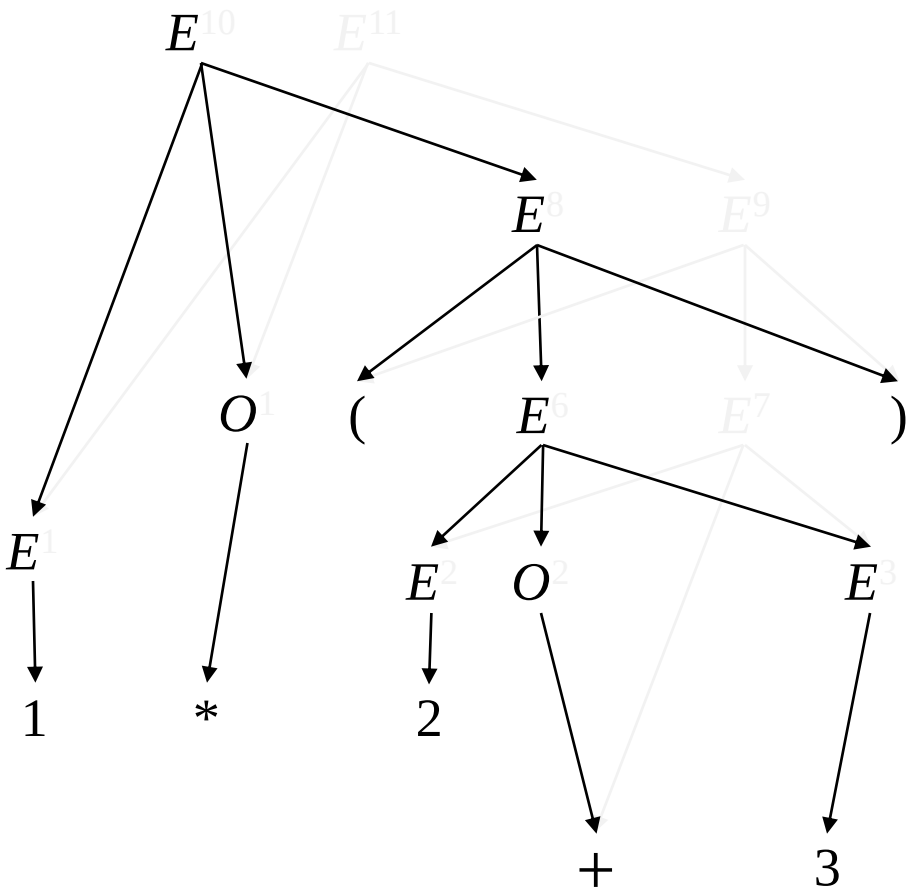
最右推导对应于扩展叶子里最右位置的那个变元

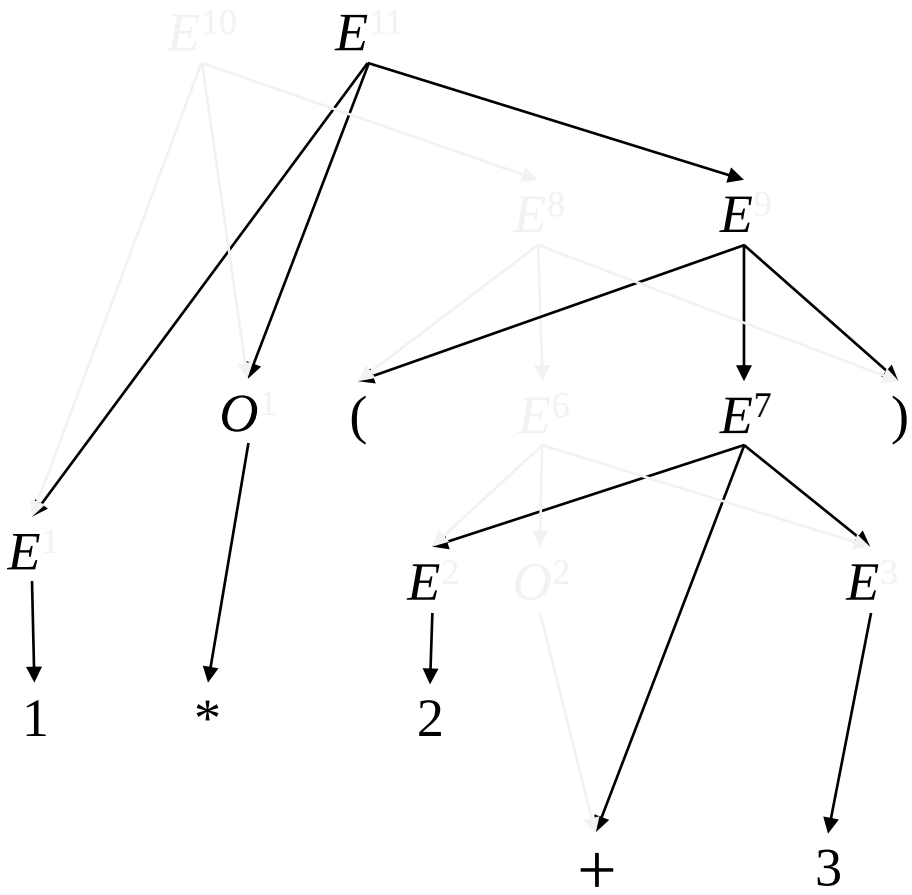


最右推导的逆对应于构建句柄的双亲节点

- 从语法树来看，一个句柄是一个二层子树的产物；
- 若当前树中有多个二层子树，句柄就是最左边的那一个。
- 所以：一个句型中的最左直接短语就是该句型的句柄。
  - 对无二义文法而言每个句型都有唯一一个句柄，这是因为句柄与语法树是一一对应的。

一个句型的句柄是其语法树的最左直接短语





- 把一个语法树的最左直接短语剪枝掉（句柄剪枝）会怎样？
- 得到另一个语法树，继续对其进行句柄剪枝，…，直到仅剩下初始符号。——这就形成了这种规范归约过程。

规范归约（句柄剪枝）示例

$P \to \check{D}\,\check{S}$

$\check{D} \to \varepsilon \mid D\,;\,\check{D}$

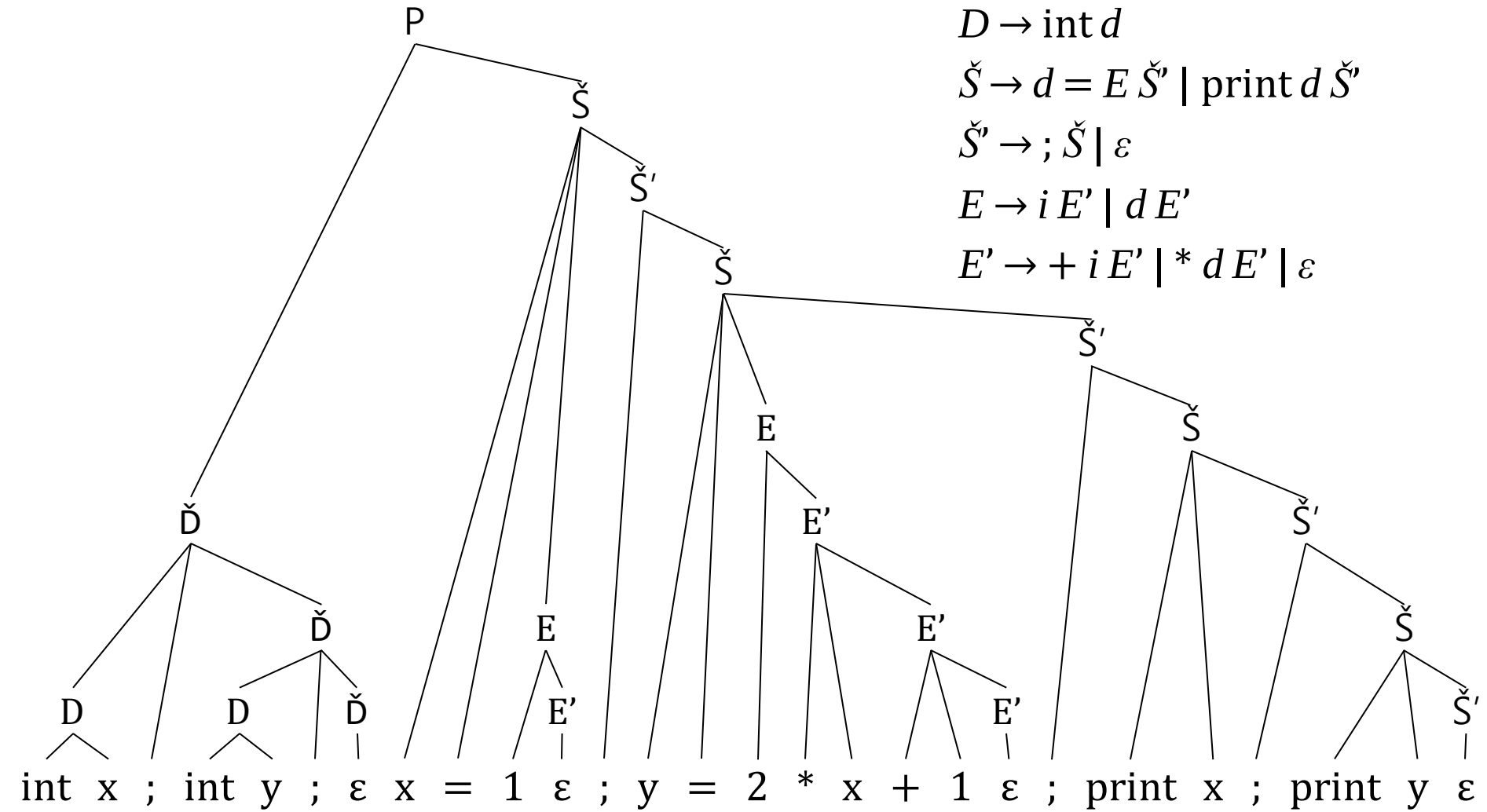
$D \to \mathrm{int}\,d$

$\check{S} \to d = E\,\check{S}' \mid \mathrm{print}\,d\,\check{S}'$

$\check{S}' \to ;\,\check{S} \mid \varepsilon$

$E \to i\,E' \mid d\,E'$

$E' \to +\,i\,E' \mid *\,d\,E' \mid \varepsilon$



规范归约的序列表示

- CFG (V, T, P, S)的几个术语：
- 规范归约是最右推导的逆过程
- 最右推导又被称为规范推导
- 规范句型：从句子的规范推导得出的句型，也叫右句型。

- 对于$\alpha_0 = w$， $\alpha_n = S$，称$\alpha_0,...,\alpha_n$是一个规范归约序列，$n \geqslant 1$，如果满足下面任意一条：
  - $\alpha_{i+1} \Rightarrow_{\mathrm{rm}} \alpha_i$， $i=0,...,n\text{-}1$
  - α_i的句柄γ_i被替换为A_{i+1}得到α_{i+1}，$(A_{i+1},\gamma_i) \in P$，$i=0,...,n\text{-}1$

- 用规范归约算符$\mapsto$表示为$\alpha_0 \mapsto^* \alpha_1 \mapsto^* ... \mapsto^* \alpha_n$
- $\mapsto$既指归约一步（句型有变），也指归约0步但同时剩余串的1前缀被压入栈顶，是移进一步（句型没变）

对规范归约过程的观察

$Z_0w\# \Rightarrow^* \cdots$，$Z_0\alpha\gamma\cdot y\# \Rightarrow Z_0\alpha A\cdot y\#$，$Z_0\beta\cdot az\# \Rightarrow Z_0\beta a\cdot z\#$，$\cdots \Rightarrow^* Z_0S\cdot\#$

其中：句柄γ为A的候选式；β的所有后缀都不是句柄。

•int x; int y; x=1; y=2*x+1; print x; print y ⇨*

int x•; int y; x=1;...	Ďx=E; y=2*xE'•;...
D;int y•;x=1;...	Ďx=E;y=2E'•;...
D;D;•x=1;...	Ďx=E;y=E•;print x;print y
D;D;Ď•x=1;...	Ďx=E;y=E;print x;print y•
D;Ď•x=1;...	Ďx=E;y=E;print x;print yŠ'•
Ď•x=1;...	Ďx=E;y=E;print x;Š•
Ďx=1•; y=2*x+1;...	Ďx=E;y=E;print xŠ'•
Ďx=1E'•; y=2*x+1;...	Ďx=E;y=E;Š•
Ďx=E•; y=2*x+1;...	Ďx=E;y=EŠ'•
Ďx=E; y=2*x+1•;...	Ďx=E;Š•
Ďx=E; y=2*x+1E'•;...	Ďx=EŠ'• ⇨ ĎŠ• ⇨ P•

P→ĎŠ

Ď→ε

Ď→D;Ď

D→int d

Š→d=EŠ'

Š→print dŠ'

Š'→ε

Š'→;Š

E→iE'

E→dE'

E'→+ iE'

E'→* dE'

E'→ε

用“移进-归约”框架模拟规范归约

- 为了叙述方便将“移进-归约”框架记为SRF
- 活前缀：规范句型的前缀，它不包括句柄之后的符号。
  - 最大活前缀：规范句型的一个前缀，它的一个后缀就是句柄。
  - 最大活前缀的任何前缀都是活前缀。

- 模拟规范归约的SRF栈有如下的变化规律：
  - 栈内容永远都是活前缀
  - 当栈内容为最大活前缀时，立即进行归约，即实现一步⇨
  - 当栈内容不是最大活前缀时，进行移进操作，实现一步⇨

模拟规范归约的SRF流程（归约优先）

- 先考虑归约优先（最左归约）这种理想情形，然后处理不理想情形（消解冲突），对应于从LR(0)到SLR(1)。
- 设当前栈内容ρ，剩余串x，待归约串ρx。

1. 初始化步： $\rho=\varepsilon$ ； $x=w$, $ay=x$， a为当前输入符号。
2. 若$\tau(\rho)\cap\mathrm{ran}(\boldsymbol{\mathcal{P}})=\emptyset$ 则将a移进栈（push a），并置$x=y$。
3. 若$\rho=\delta\gamma$且$(A,\gamma)\in\boldsymbol{\mathcal{P}}$则将句柄$\gamma$归约为$A$，并置$\rho=\delta A$。
4. 如果$\rho=\delta A$还是最大活前缀，则如第3步那样继续直到不再有此情形为止。
5. 重复上述第2~4步直到：
   - $\rho=S$且$x=\varepsilon$，则分析成功（接受输入串）。
   - 如果$\rho=S\wedge x=\varepsilon$不能成立，则分析失败（拒绝输入串）。

示例：移进-归约分析过程

步骤	符号栈	剩余串	分析动作
0	Z_0	(())#	shift
1	$Z_0($	())#	shift
2	$Z_0(($	))#	shift
3	$Z_0(()$	)#	reduce:$X\rightarrow()$~~;还是shift?~~
4	$Z_0(X$	)#	shift
5	$Z_0(X)$	#	reduce:$X\rightarrow(X)$~~;还是shift?~~
6	Z_0X	#	acc
~~7~~	~~Z_0S'~~	~~#~~	

$S'\rightarrow X$
$X\rightarrow(X)$
$X\rightarrow()$

文法$G=(V,T,\boldsymbol{\mathcal{P}},S)$的增广文法$G'=(V\cup\{S'\},T,\boldsymbol{\mathcal{P}}\cup\{(S',S)\},S')$

增广文法的初始符号的候选式唯一；

增广文法对移进归约框架保证了归约为原文法的初始符号即成功。

确定化的移进-归约分析过程

- 先考虑不存在冲突的分析过程
  - 不需要查看输入符号就是确定化的LR(0)
  - 需要向前查看一个输入符号就是确定化的SLR(1)、LR(1)
  - 需要向前查看k个输入符号才是确定化的LR(k)
  - 其它，LALR(k)

- 有冲突时需要进行冲突消解
  - 分析引起冲突的原因，给出冲突消解规则
  - 运算符结合性和优先级、悬空ELSE等冲突都可通过冲突消解规则在SLR(1)上得到消解。
- 本章的方法可以满足程序设计语言。

8.3构建规范归约模拟器

- 在讨论了栈的变化规律基础上，基于PDA模型，考虑用NFA控制栈的动作。称为可模拟规范归约的模拟器，记为itemPDA，可进一步确定化为itemDPDA。
- 构建itemDFA：
  - 文法项目与文法项目集
  - 增广文法
  - 活前缀的有效项目
  - 识别活前缀的NFA，即itemNFA，得到由其与SRF组成的itemPDA
  - 得到LR(0)规范簇（识别活前缀的DFA，即itemDFA）
  - 得到由itemDFA和SRF组成的itemDPDA

文法项目集

- **定义8.7** 文法项目集和文法项目。文法G的项目集 itemset$(G)=\{(A,\alpha,\beta)\mid(A,\alpha\beta)\in\mathcal{P}\}$，其中$(A,\alpha,\beta)$是一个文法项目，直观地表示为$A\to\alpha\cdot\beta$，是由产生式规则$A\to\alpha\beta$产生的项目。可以看出，由产生式$A\to\gamma$为文法项目集贡献$|\gamma|+1$个项目

- 例CFG G: S'→X X→(X) X→()

  itemset(G): S'→·X S'→X· X→·(X) X→(·X) X→(X·) X→(X)· X→·() X→(·) X→()·

- 模拟器建模思路：将一个句子的分析过程（SRF的步骤序列）中的每个步骤采用一个一个状态来描述。
- 每一个文法项目都用于构成一个状态。
  - SRF初始化步所对应的状态是[S'→·X]
  - 中间某步骤可能对应状态[X→(·X)]或其他
  - SRF成功步对应状态[S'→X·]

建模规范归约模拟器

- 将SRF+CFG看作一个系统，作为规范归约模拟器，可进一步写程序实现为Parser（语法分析器）。
- 基于PDA对SRF+CFG系统进行建模
  - SRF栈的移进动作表示为状态间的转移标记 $a, \varepsilon/a$
  - SRF栈的归约动作表示为状态间的转移标记 $\varepsilon, \gamma/A$，$(A, \gamma) \in \mathcal{P}$
- **对状态建模的思路：**既兼顾分析过程的历史又展望其未来：
  - 初始状态 $[S' \rightarrow \cdot S]$ 意味着历史为 ε，未来是“消耗掉 w 并归约出 S”，所对应的句型为 $\varepsilon \cdot w$。
  - 接受状态 $[S' \rightarrow S \cdot]$ 意味着历史为“已经消耗掉了 w 并归约出 S”而未来是 ε（没有更多的事情），所对应的句型为 $S \cdot \varepsilon$。
  - 一般状态 $[A \rightarrow \gamma^{\text{past}} \cdot \gamma^{\text{future}}]$ 意味着历史为 $\gamma^{\text{past}} \in \tau(\rho^{\text{current}})$，未来为 $\gamma \in \tau(\rho^{\text{future}})$，所对应的句型为 $\rho^{\text{current}} \cdot ay$，其中 $\gamma = \gamma^{\text{past}} \cdot \gamma^{\text{future}}$，$\rho^{\text{current}} \gamma^{\text{future}} = \rho^{\text{future}}$。若 $\gamma^{\text{past}} = \varepsilon$ 意味着未来是“归约出 A”。$\gamma^{\text{future}} = \varepsilon$ 意味着已经归约出 A 了。ρ 指SRF的栈内容。

建模规范归约模拟器（续）

- 将移动$\vdash$分解为“状态转移”和“SRF栈动作”两部分同步进行，前者在itemNFA上进行，后者在SRF上进行。
  - 状态$[A\to\alpha\cdot c\beta]$ 转移到 $[A\to\alpha c\cdot\beta]$与把c**移进**栈二者同步进行
  - 从$[A\to\alpha\cdot N\beta]$转移到$[A\to\alpha N\cdot\beta]$，前提是消耗剩余串归约出N。将此设为归约目标，并分解为（许愿）能从当前状态开始并移动到栈顶为γ的ID（瞬时描述），其中$(N,\gamma)\in\mathcal{P}$。
  - “从状态为$[A\to\alpha\cdot N\beta]$的ID移动到栈顶为γ的ID”必然要开启归约为N的过程，即从状态为$[N\to\cdot\gamma]$的ID移动到状态为$[N\to\gamma\cdot]$的ID（这是许下的愿望）。
  - 在$[N\to\gamma\cdot]$状态时愿望达成，然后（还愿）先将当前状态回溯至许愿处，然后将栈顶γ替换为N，这就实现了**归约**为N的目标，注意伴随状态转移到$[A\to\alpha N\cdot\beta]$。

示例：构建规范归约模拟器

$\varepsilon\cdot(())$

状态$[S'\to\cdot X]$，目标❶归约出X，许愿①能移动到栈顶为(X)的ID或②能移动到栈顶为$()$的ID。

$\Rightarrow(\cdot())$

状态$[X\to(\cdot X)]$，目标❷归约出X，许愿能移动到栈顶为③(X)或④$()$的ID，或者；
状态为$[X\to(\cdot)]$，积跬步$\Rightarrow$，$\rho^{\text{current}}=($，$\rho^{\text{future}}=()$

$\Rightarrow((\cdot))$

状态$[X\to(\cdot)]$，积跬步$\Rightarrow$，$\rho^{\text{current}}=(($，$\rho^{\text{future}}=(()$

$\Rightarrow(()\cdot)$

状态$[X\to()\cdot]$，愿望④已达成，还愿④。

$\Rightarrow(X\cdot)$

状态$[X\to(X\cdot)]$，积跬步$\Rightarrow^*$，达成目标❷。$\rho^{\text{current}}=(X$，$\rho^{\text{future}}=(X)$。注：正走在实现愿望①的路上

$\Rightarrow(X)\cdot\varepsilon$

状态$[X\to(X)\cdot]$，愿望①已达成，还愿①。

$\Rightarrow X\cdot\varepsilon$

状态$[S'\to X\cdot]$，积跬步$\Rightarrow^*$，达成目标❶，$\rho^{\text{current}}=X$，$\rho^{\text{future}}=X$

示例：得出itemNFA

$\varepsilon\cdot(())$

$[S'\to\cdot X]$；❶X；①(X)；②$()$

$\vDash(\cdot())$

$[X\to(\cdot X)]$；❷X；③(X)；④$()$。
$[X\to(\cdot)]$，积跬步$\rho^{\text{current}}=($，$\rho^{\text{future}}=()$

$\vDash((\cdot))$

$[X\to(\cdot)]$，积跬步$\rho^{\text{current}}=(($，$\rho^{\text{future}}=(()$

$\vDash(()\cdot)$

$[X\to()\cdot]$，愿望④已达成，还愿④。

$\vDash(X\cdot)$

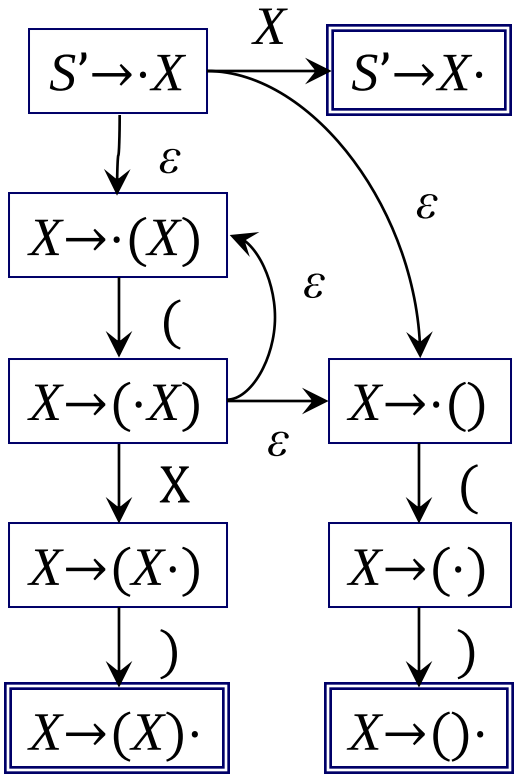
$[X\to(X\cdot)]$，积跬步达成目标❷。
$\rho^{\text{current}}=(X$，$\rho^{\text{future}}=(X)$

$\vDash(X)\cdot\varepsilon$

$[X\to(X)\cdot]$；愿望①已达成，还愿①。

$\vDash X\cdot\varepsilon$

$[S'\to X\cdot]$，积跬步达成总目标❶，
$\rho^{\text{current}}=X$，$\rho^{\text{future}}=X$



示例：得出itmeDFA

$\varepsilon\cdot(())$

状态0； ❶ X； ① (X)； ② $()$

$\Rightarrow (\cdot())$

2含$X\to(\cdot X)$； ❷ X； ③(X)； ④$()$ 。
2含$X\to(\cdot)$，$\rho^{\text{current}}=($， $\rho^{\text{future}}=()$

$\Rightarrow ((\cdot))$

2含$X\to(\cdot X)$； X； ⑤(X)； ⑥$()$ 。
2含$X\to(\cdot)$，$\rho^{\text{current}}=(($， $\rho^{\text{future}}=(()$

$\Rightarrow (()\cdot)$

4$[X\to()\cdot]$， 愿望④已达成还愿④ 。

$\Rightarrow (X\cdot)$

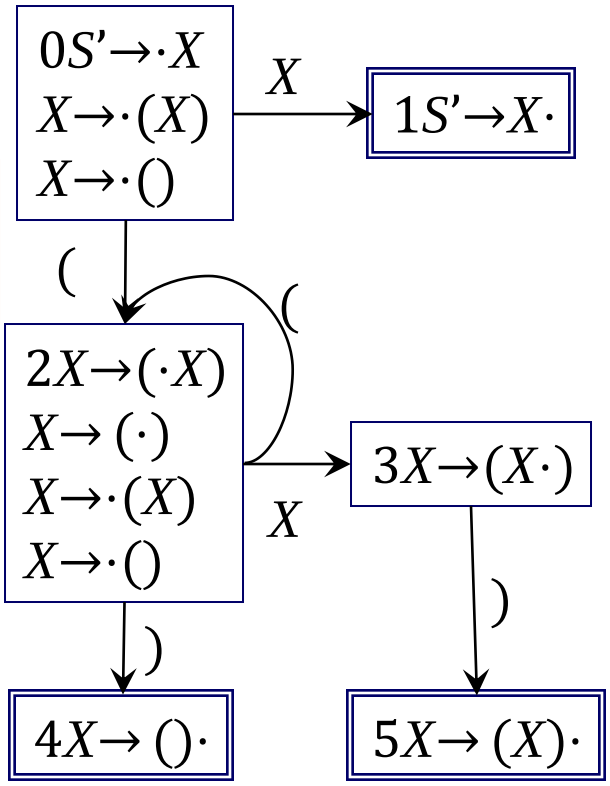
3$[X\to(X\cdot)]$， 积跬步达成目标❷ 。
$\rho^{\text{current}}=(X$， $\rho^{\text{future}}=(X)$

$\Rightarrow (X)\cdot\varepsilon$

5$[X\to(X)\cdot]$； 愿望①已达成还愿①。

$\Rightarrow X\cdot\varepsilon$

1$[S'\to X\cdot]$， 积跬步达成总目标❶ ，
$\rho^{\text{current}}=X$， $\rho^{\text{future}}=X$

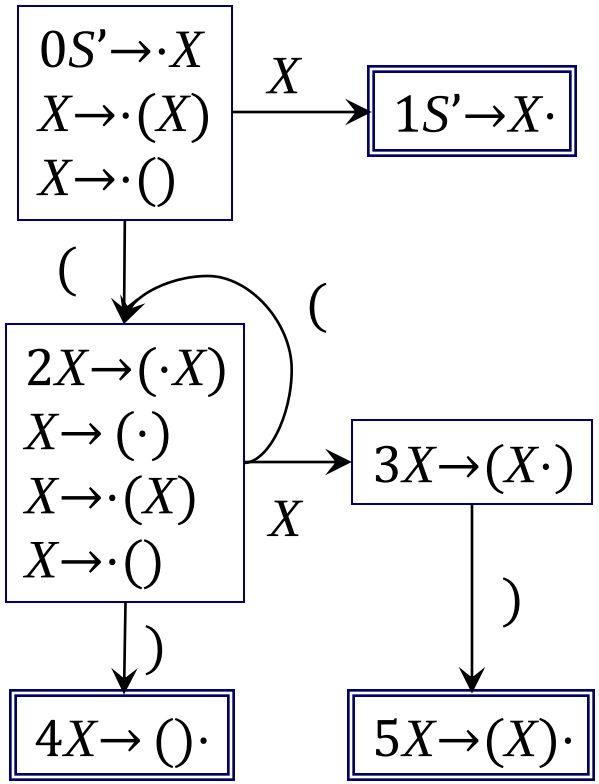


4还愿④：从4退到2再到2栈做$()/X$，接着$\upsilon(2,X)=3$

5还愿①：从5退到3再退到2再退到0，栈做$(X)/X$，接着$\upsilon(0,X)=1$

一种确定性的分析过程

状态栈	符号栈	剩余串	动作
0	#	(())#	shift
02	#(	())#	shift
022	#((	))#	shift
0224	#(()	)#	reduce X→()
023	#(X	)#	shift
0235	#(X)	#	reduce X→(X)
01	#X	#	acc



[X→γ•]时，双栈同步弹出|γ|个符号，然后，符号栈压入X，状态找顶s替换为υ(s, X)s，即压入υ(s, X)。

文法项目的常用别名

- G的项目集itemset(G)：
  - $S' \to \bullet X$　　初始项目、待约项目
  - $S' \to X\bullet$　　接受项目、归约项目、完全项目
  - $X \to \bullet(X)$　　初始项目、移进项目
  - $X \to (\bullet X)$　　待约项目
  - $X \to (X\bullet)$　　移进项目
  - $X \to (X)\bullet$　　归约项目、完全项目
  - $X \to \bullet()$　　初始项目、移进项目
  - $X \to (\bullet)$　　移进项目
  - $X \to ()\bullet$　　归约项目、完全项目

CFG G:　$S \to X$　$X \to (X)$　$X \to ()$

进一步认识：文法项目作为状态的意义

- 从**待约项目**$N\to\alpha\cdot A\beta$定下目标“消耗剩余串前缀归约出A”，并许下愿望“能移动到栈顶为γ这样的ID”。这里，A是点右变元，γ是其候选式。
  - 例：待约项目$S'\to\cdot X$和$X\to(\cdot X)$的点右变元都是X。
- 从**初始项目**（作为状态）开启实现愿望之路（积跬步）
  - 最大的愿望是栈顶为S'的唯一候选式。
  - 在后面还愿时，该起点会被回溯到并直至许愿点。
  - 初始项目$S'\to\cdot X$、$X\to\cdot(X)$和$X\to\cdot()$作为状态都是愿望之路的起点。
- 从**完全项目**知道愿望已经达成，即一条愿望之路已完成，SRF栈顶为γ，并立即还愿，先回退到许愿时所处状态$[N\to\alpha\cdot A\beta]$，同时将栈顶γ替换为A，然后（积跬步）状态转移到$[N\to\alpha A\cdot\beta]$
  - 例:完全项目$S'\to X\cdot$、$X\to(X)\cdot$和$X\to()\cdot$分别是各愿望之路的终点
- 从**移进项目**看到是走在愿望之路上（积跬步）
  - 移进项目$X\to\cdot(X)$、$X\to(X\cdot)$、$X\to\cdot()$和$X\to(\cdot)$都是通过SRF移进而形成一步规范归约$\Mapsto$

itemNFA状态转移与SRF栈内容变化对应

- 设置两类状态，移进状态和归约状态。
- 在移进状态下，SRF栈中不是最大活前缀。是否允许移进当前输入符号呢？问itemNFA（或itemDFA）：
  - 在 $[A\rightarrow\alpha\bullet c\beta]$状态，如果tok=$c$那么移进，否则出错。

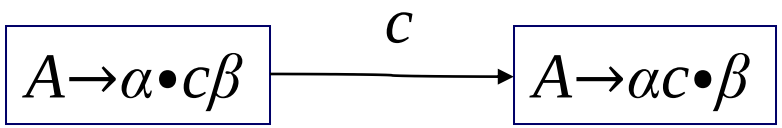
- 在归约状态下，SRF栈中为最大活前缀。用哪个产生式归约呢？问itemNFA（或itemDFA）：
  - 设句柄为γ，通过当前状态里的项目来指明用$A\rightarrow\gamma$归约。
  - 在 $[A\rightarrow\gamma\bullet]$状态，栈顶句柄$\gamma$归约为$A$。

基于文法穷尽所有状态及状态转移关系

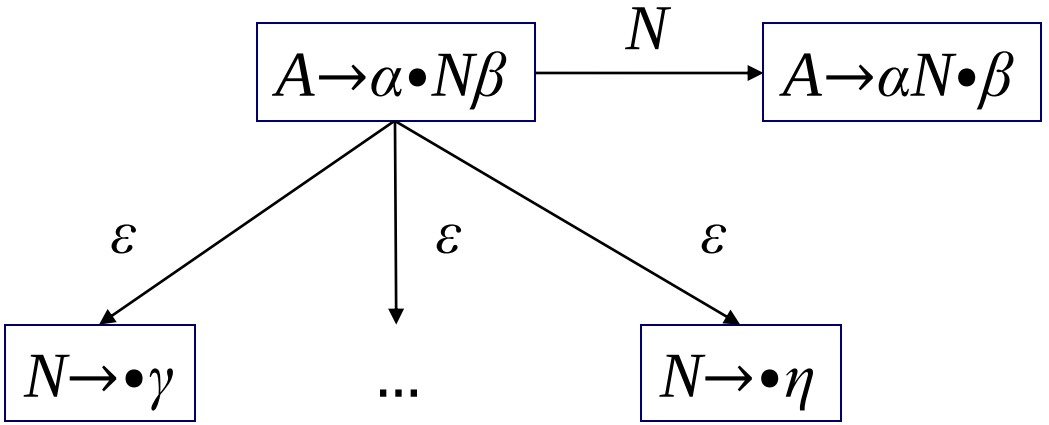
- 使用itemNFA实现穷尽所有情形。
- 已知CFG$(V, T, \mathcal{P}, S)$它的增广文法的项目集为itemset(G')。
- 该集合就是itemNFA的状态集合，
- $V \cup T$是字母表，$[S' \rightarrow \bullet S]$是初始状态，$\{[A \rightarrow \gamma \bullet] | (A, \gamma) \in \mathcal{P}\}$为接受状态集合。
- 转移函数：

itemNFA的转移函数

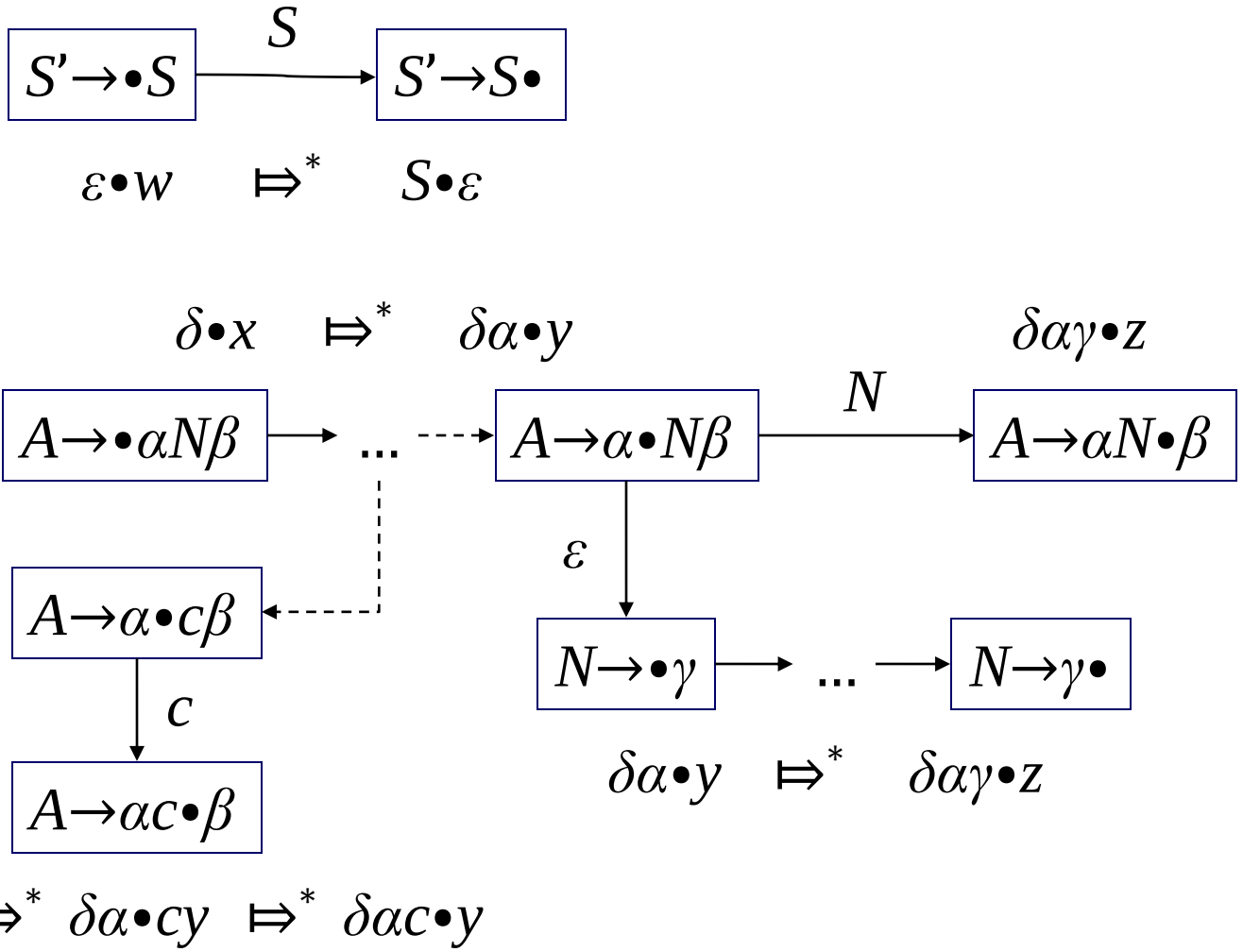
- $v([A\to\alpha\bullet c\beta], c) = \{[A\to\alpha c\bullet\beta]\}$ 对应于规范归约
  $\ldots\alpha\bullet c\ldots \Rightarrow \ldots\alpha c\bullet\ldots$



- $v([A\to\alpha\bullet N\beta], \varepsilon) \supseteq \bigcup(N,\gamma)\in\mathcal{P}\cdot\{[N\to\bullet\gamma]\}$
- $v([A\to\alpha\bullet N\beta], N) \supseteq \{[A\to\alpha N\bullet\beta]\}$
- 对应于规范归约
  $\delta\alpha\bullet y_k z_k \Rightarrow^* \delta\alpha N\bullet z_k$ 若有以下中仅第k个计算是合法的
  $\delta\alpha\bullet y_1 z_1 \Rightarrow^* \delta\alpha\gamma\bullet z_1$
  ...
  $\delta\alpha\bullet y_m z_m \Rightarrow^* \delta\alpha\eta\bullet z_m$



itemNFA示例：移动与规范规约对应



活前缀的可用项目与有效项目

- 对于itemPDA，SRF栈内容ρ始终是规范句型ρx的活前缀。
- 项目$A\to\gamma^{\text{past}}\cdot\gamma^{\text{future}}$被称为是活前缀$\rho=\delta\gamma^{\text{past}}$的可用项目。
- 可用项目的意义在于表示这个活前缀ρ是在$[A\to\gamma^{\text{past}}\cdot\gamma^{\text{future}}]$状态可能存在的，这个状态是愿望之路（目标$A$、许愿积跬步$\gamma$现栈顶）上的一个节点，不管该愿望是否达成。
- 若可用项目的愿望之路出现在一条成功的规范归约路径中，就表明它是ρ的有效项目。ρ在$[A\to\gamma^{\text{past}}\cdot\gamma^{\text{future}}]$状态肯定存在
- 项目$A\to\gamma^{\text{past}}\cdot\gamma^{\text{future}}$的愿望之路：
  - 起点：目标是“归约为A”；愿望是“积跬步到γ成为栈顶”；起点状态是$[A\to\cdot\gamma]$。终点状态是$[A\to\gamma\cdot]$
- 愿望之路到规范归约步骤有一一对应关系：

$$\delta\cdot\ldots x\mapsto^{*}\rho\cdot x=\delta\gamma^{\text{past}}\cdot yz\mapsto^{*}\delta\gamma\cdot z\mapsto\delta A\cdot z$$

- 愿望之路出现在一条成功的规范归约路径里：

$$w\mapsto^{*}\rho\cdot x=\delta\gamma^{\text{past}}\cdot yz\mapsto^{*}\delta\gamma\cdot z\mapsto\delta A\cdot z\mapsto^{*}S\cdot\varepsilon$$

例：活前缀的有效项目

$Z_0 \bullet (())\#$ 0, 2, 6

$\vdash Z_0(\bullet())\#$ 3, 2, 6, 7

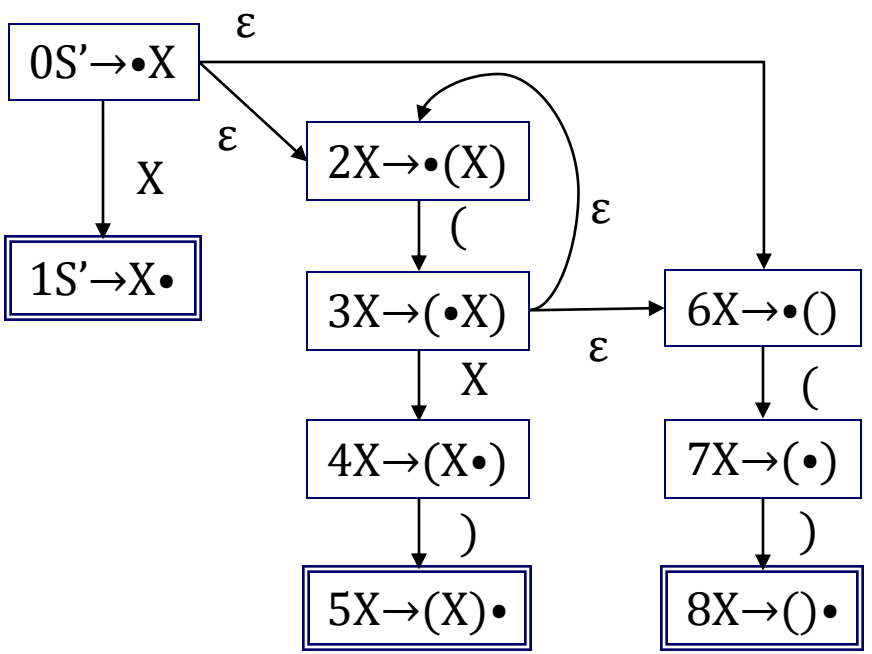
$\vdash Z_0((\bullet))\#$ 3, 2, 6, 7

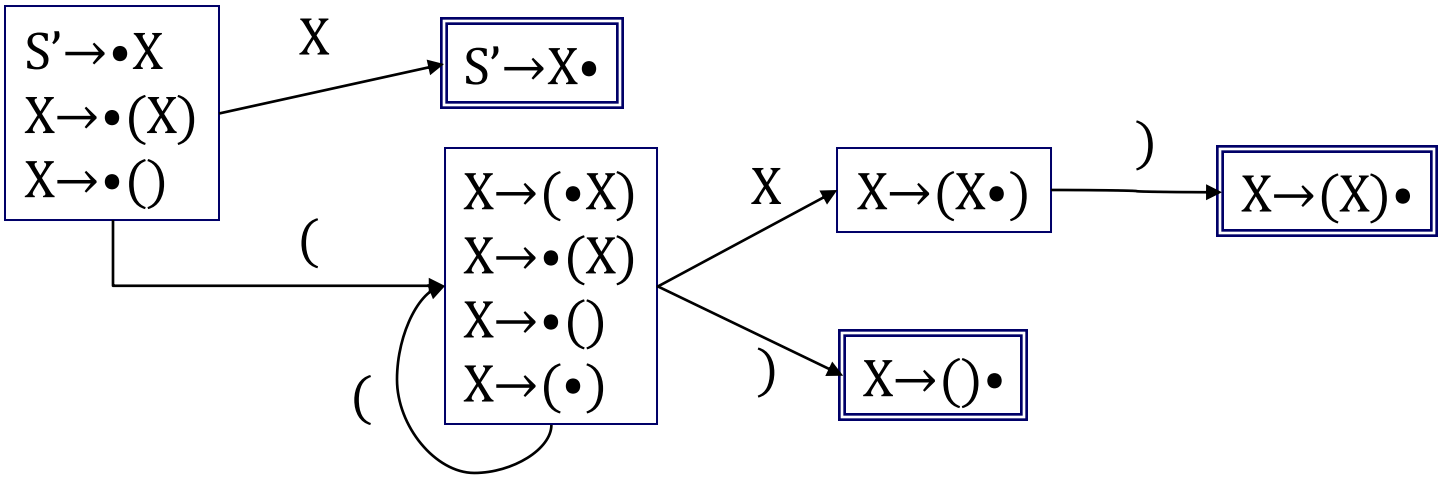
$\vdash Z_0(()\bullet)\#$ 8

$\vdash Z_0(X\bullet)\#$ 4

$\vdash Z_0(X)\bullet\#$ 5

$\vdash Z_0X\bullet\#$ 1





CFG:

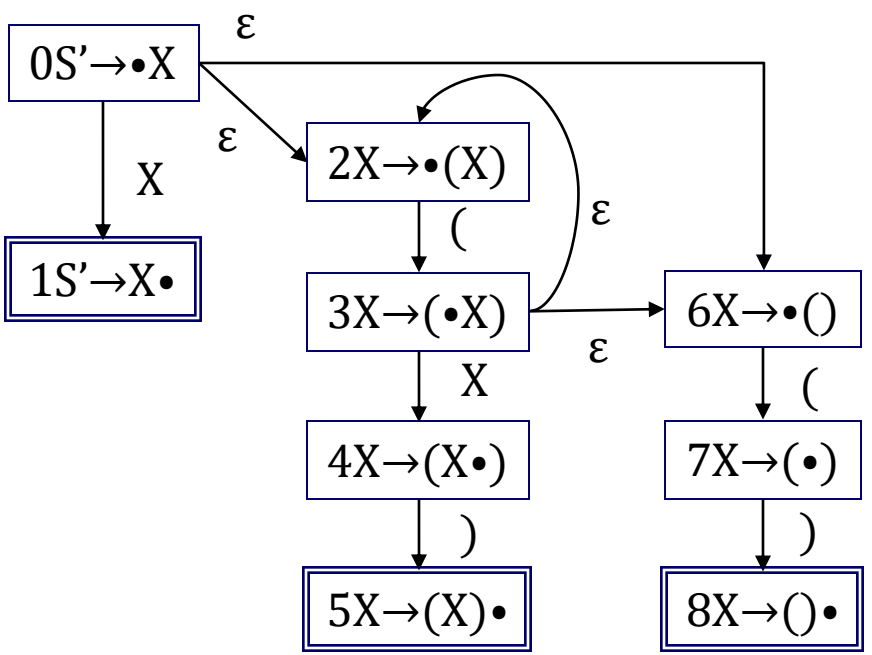
S'→X

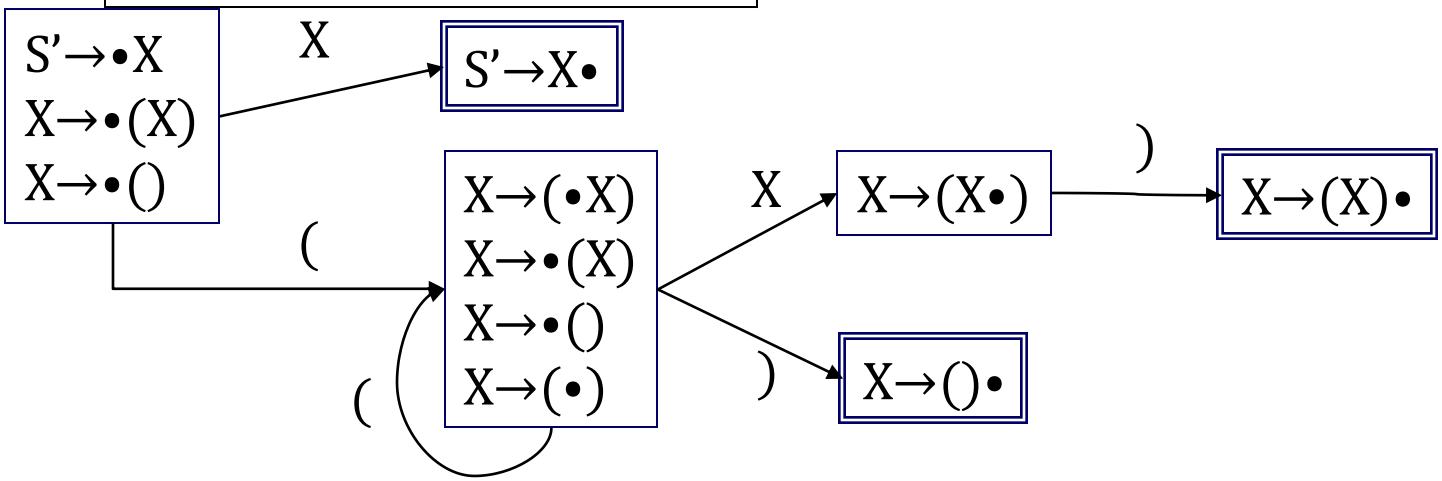
X→(X)

X→()

例：活前缀的有效项目

$Z_0\bullet((()))\#$	0, 2, 6
$\Rightarrow Z_0(\bullet(()))\#$	3, 2, 6, 7
$\Rightarrow Z_0((\bullet()))\#$	3, 2, 6, 7
$\Rightarrow Z_0(((\bullet)))\#$	3, 2, 6, 7
$\Rightarrow Z_0((()\bullet))\#$	8
$\Rightarrow Z_0((X\bullet))\#$	4
$\Rightarrow Z_0((X)\bullet)\#$	5
$\Rightarrow Z_0(X\bullet)\#$	4
$\Rightarrow Z_0(X)\bullet\#$	5
$\Rightarrow Z_0X\bullet\#$	1





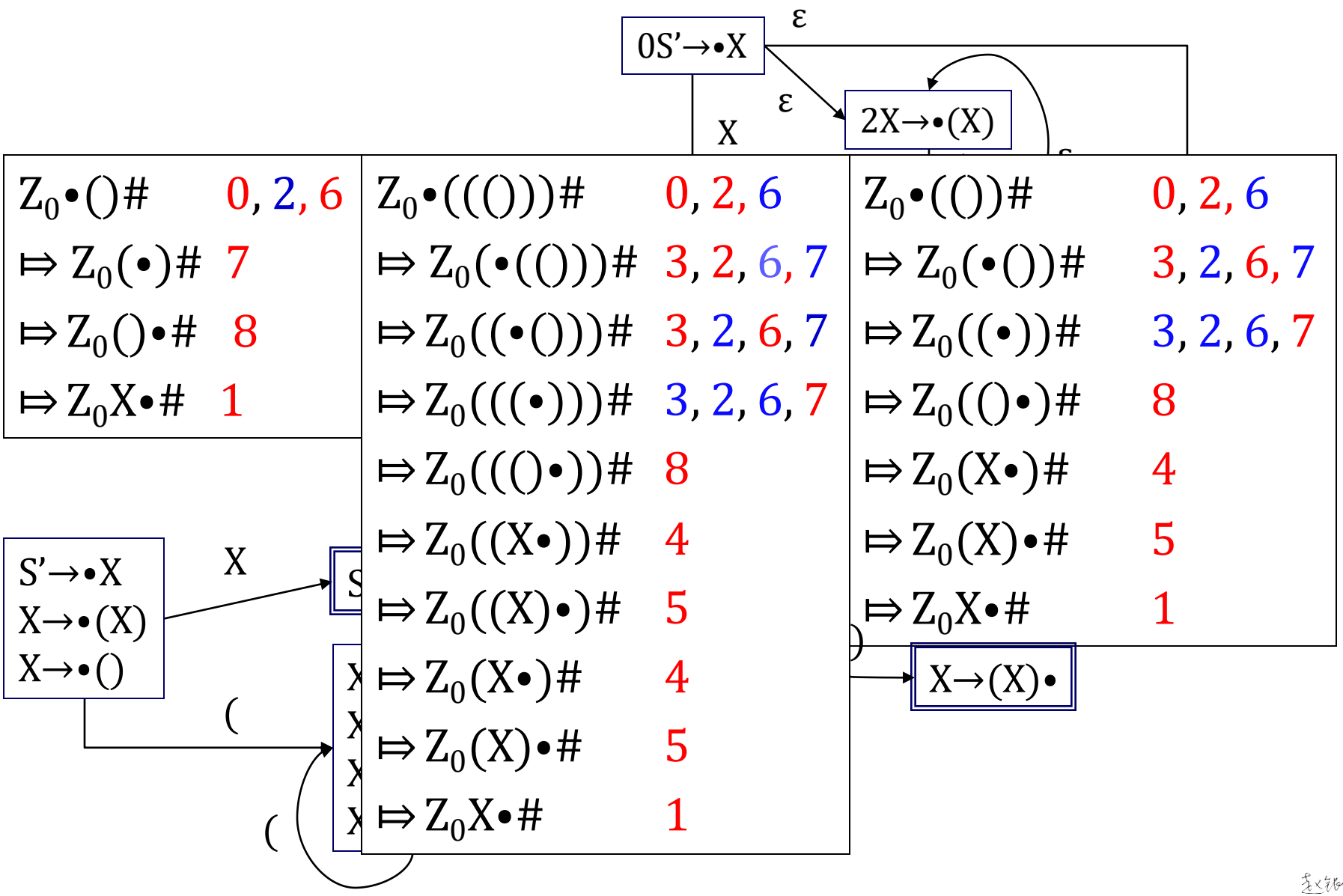
CFG:

$S'\rightarrow X$

$X\rightarrow(X)$

$X\rightarrow()$

例：活前缀的有效项目


0S'→•X
ε
ε
2X→•(X)
X
S'→•X
X→•(X)
X→•()
X
(
(
)
X→(X)•
Z0•()# 0, 2, 6
⊨ Z0(•)# 7
⊨ Z0()•# 8
⊨ Z0X•# 1
Z0•((()))# 0, 2, 6
⊨ Z0(•(()))# 3, 2, 6, 7
⊨ Z0((•()))# 3, 2, 6, 7
⊨ Z0(((•)))# 3, 2, 6, 7
⊨ Z0(((()•))# 8
⊨ Z0((X•))# 4
⊨ Z0((X)•)# 5
⊨ Z0(X•)# 4
⊨ Z0(X)•# 5
⊨ Z0X•# 1
Z0•(()) # 0, 2, 6
⊨ Z0(•())# 3, 2, 6, 7
⊨ Z0((•))# 3, 2, 6, 7
⊨ Z0(()•)# 8
⊨ Z0(X•)# 4
⊨ Z0(X)•# 5
⊨ Z0X•# 1

有效项目集与规范簇

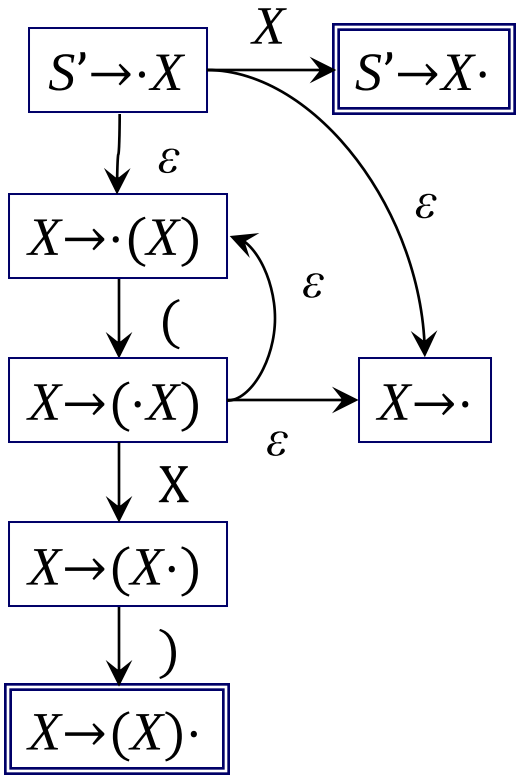
- 一个活前缀的所有有效项目构成的集合被称为识别该活前缀的有效项目集。
- 文法G的所有的有效项目集构成的集合称为该文法的LR(0)项目集规范簇，简称规范簇或LR(0)规范簇。
- 考察$L(G)$的每个元素（句子）的规范推导，收集到所有活前缀；再基于itemset(G)文法确定每个活前缀的有效项目集，这些项目集构成的集合就是该文法的LR(0)规范簇。
- 任意一个活前缀都有唯一一个有效项目集；
- 规范归约过程中，活前缀ρ增长为活前缀ρ'，其中$|\rho'|=|\rho|+1$，是因为ρ的有效项目集到ρ'的有效项目集有状态转移关系；
- itemNFA和itemDFA都给出了有效项目集间的转移关系。
- itemDFA的状态集合就是LR(0)项目集规范簇。

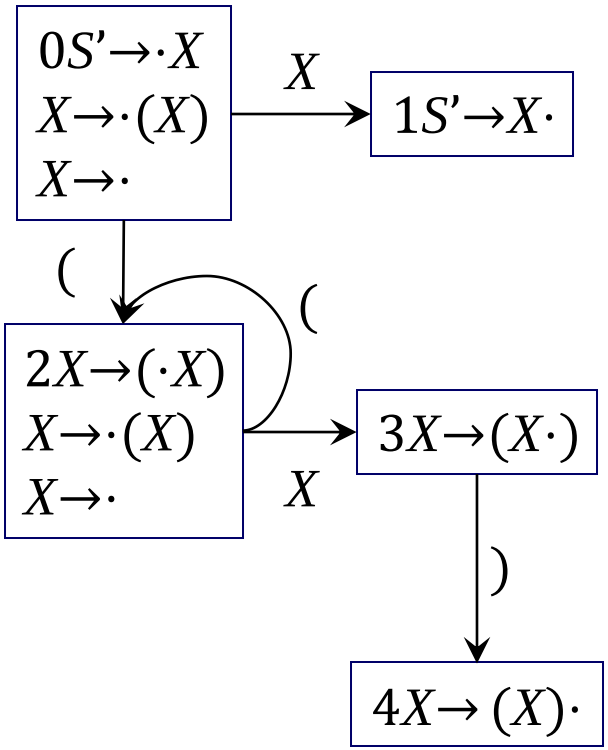
直接构造itemDFA

- 给定CFG $G=(\mathbb{V}, T, \boldsymbol{\mathcal{P}}, S)$，构造itemDFA$(Q, \Sigma, \upsilon, q_0, F)$。
- itemDFA 的状态是项目集合
- 初始状态$q_0=\omega([S'\rightarrow\bullet S])$
  - 定义项目集q的ε闭包$\omega(q)$如下：
    如果$[A\rightarrow\alpha\bullet N\beta]\in\omega(q)$，那么$[N\rightarrow\bullet\gamma]\in\omega(q)$，
    其中$(A, \alpha N\beta), (N, \gamma)\in\boldsymbol{\mathcal{P}}$。
  - 直观地，itemNFA中，$[A\rightarrow\alpha\bullet N\beta]$到$[N\rightarrow\bullet\gamma]$有$\varepsilon$转移。
- **转移函数**：对于$q\in Q$，令$p=\upsilon(q, X)$，$X\in\mathbb{V}\cup T$，那么，
  - $p=\omega(\{[A\rightarrow\alpha X\bullet\eta]\mid[A\rightarrow\alpha\bullet X\eta]\in q\})$，
  - $p\in Q$，
  - 若p仅含完全项目，那么$p\in F$。

例：$X\to(X)$ $X\to\varepsilon$

构建itemNFA、itemDFA都使用增广文法。
建议直接构建itemDFA;
也可先构建itemNFA，再确定化为itemDFA。





注意，ε产生式的项目只有1个，其愿望之路的起点即终点，长度为0

构造itemDFA算法

输入：CFG $G=(V, T, P, S)$

输出：识别活前缀的DFA$(Q, V\cup T, \upsilon, q_0, F)$。

$Q=\emptyset$; $F=\emptyset$; $\upsilon=\emptyset$;

将$q_0=\omega([S'\rightarrow\bullet S])$加入$Q$;

while (Q中存在未标记元素q) {

  标记q;

  for ($X\in V\cup T$) {$p=\emptyset$;

    for ($s\in q$ && $s==[A\rightarrow\alpha\bullet X\eta]$) 将$[A\rightarrow\alpha X\bullet\eta]$加入$p$;

    if($p!=\emptyset$){ $p=\omega(p)$; 将p加入Q;

      将$((q, X), p)$加入υ; } //即令$\upsilon(q, X)=p$

      若p含完全项目，那么将p加入F; }

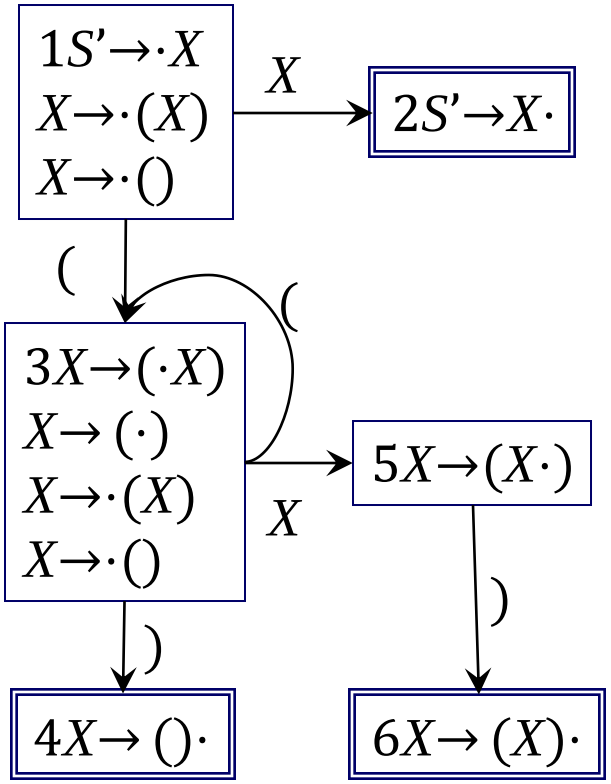
}

从itemDFA构造LR(0)分析表

- LR(0)分析表由ACTION[]和GOTO[]两个表组成：
- ACTION[m, a]=? 其中m为状态，a为当前输入符号
  - sn：若itemDFA中有尾m头n权a的弧；
  - acc：m状态包含有接受项目；
  - rk：m状态含编号k的产生式的完全项目（非接受项目）。
- GOTO[m, A]=?
  - n：若itemDFA中有尾m头n权A 的弧。

- LR(0)分析表看起来是itemDFA的转移表表示，其中状态被排序并使用索引值替代状态引用，另外对终结符列添加了额外信息。
- 但是与用DFA判定输入串的运行方式不同，注意这里输入串是终结符串。（相关理论：项目PDA）

例：LR(0)分析表



	ACTION			GOTO
状态	(	)	#	X
1	s3			2
2			acc	
3	s3	s4		5
4	r3	r3	r3	
5		s6		
6	r2	r2	r2	

$1S'\rightarrow X$

$2X\rightarrow(X)$

$3X\rightarrow()$

- 给定CFG；
- 先求出itemDFA；
- 再填ACTION表和GOTO表；

基于LR(0)分析表的分析过程

	ACTION			GOTO
状态	(	)	#	X
1	s3			2
2			acc	
3	s3	s4		5
4	r3	r3	r3	
5		s6		
6	r2	r2	r2	

ACTION[m,a]：

- sn：bi-push(n, a)
- rn：按产生式$nA\rightarrow\alpha$归约：bi-pop($|\alpha|$);bi-top(m', X] bi-push((GOTO[m', A], A)。
- acc：分析成功。

状态栈	符号栈	剩余串	动作
1	#	(())#	shift
13	#(	())#	shift
133	#((	))#	shift
1334	#(()	)#	reduce X→()
135	#(X	)#	shift
1356	#(X)	#	reduce X→(X)
12	#X	#	acc

$1S'\rightarrow X$

$2X\rightarrow(X)$

$3X\rightarrow()$

基于分析表的LR(0)分析算法

▻ 输入：分析表，*w*，双栈空栈。 输出：分析成功与否。

```
bi-stack(1,#);//双栈初始化分别为初始状态1和符号#
scan();          //读入当前符号赋给全局变量tok
while(1){
   bi-top(m, c);          //将双栈栈顶符号分别赋给m和c
   if (ACTION[m, tok]==sn)bi-push(n, tok);
   else if(ACTION[m, tok]==rk){
         bi-pop(body-size(k)); //弹出产生式k的体长那么多
         A=head(k);              //产生式的头，即左部变元
         bi-top(m', c);   bi-push(GOTO[m',A], A);
    }else if(ACTION[m, tok]==acc)break;   //接受
    else error();          //其它情况都属于错误
}
```

5.4 SLR(1)分析

- LR(0)分析要求基于LR(0)文法进行，
- LR(0)文法对它的LR(0)规范簇的每一个有效项目集都要求没有冲突，即：
  - 含有一个完全项目，就不能再含有任何别的项目。

- 那么，SLR(1)分析的思想？
- 查看当前输入符号来消解LR(0)规范簇中的冲突：
  - 移进-归约冲突；
  - 归约-归约冲突。
- 满足以上条件的文法是SLR(1)文法

不是SLR(0)文法?

1 $S' \to X$

2 $X \to (X)$

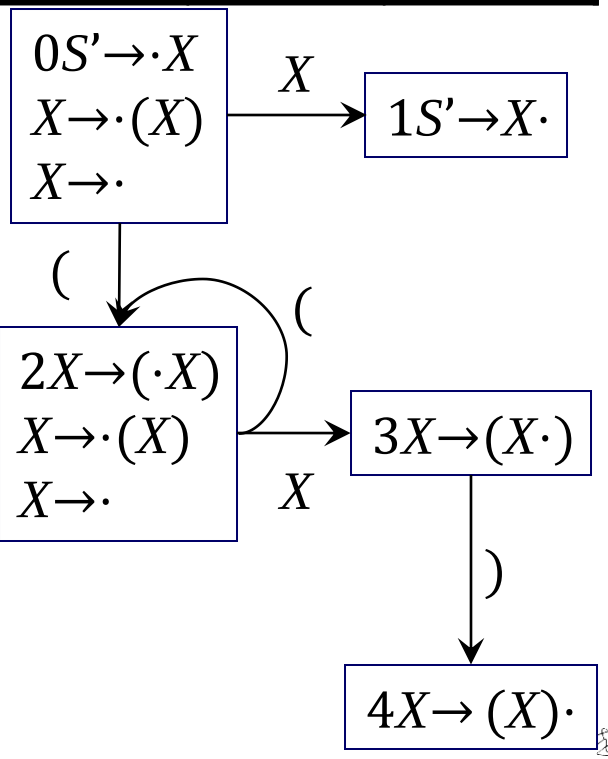
3 $X \to \varepsilon$

FIRST(X)={(, ε, #}

FOLLOW(X)={#,)}

	ACTION			GOTO
状态	(	)	#	X
0	s2/~~r3~~	r3	r3	1
1			acc	
2	s2/~~r3~~	r3	r3	3
3		s4		
4		r2	r2	

状态栈	符号栈	剩余串	动作
0	Z_0	(())#	shift
02	Z_0(	())#	shift
022	Z_0((	))#	reduce X→ε
0223	Z_0((X	))#	shift
02234	Z_0((X)	)#	reduce X→(X)
023	Z_0(X	)#	shift
0234	Z_0(X)	#	reduce X→(X)
01	Z_0X	#	acc



有冲突的例子很常见

- 对LR(0)文法若$\tau(\rho)\cap \mathrm{ran}(\boldsymbol{\mathcal{P}})=\emptyset$则移进否则归约。

$$S\to En \quad S\to n \quad S\to \varepsilon \quad E\to n+$$

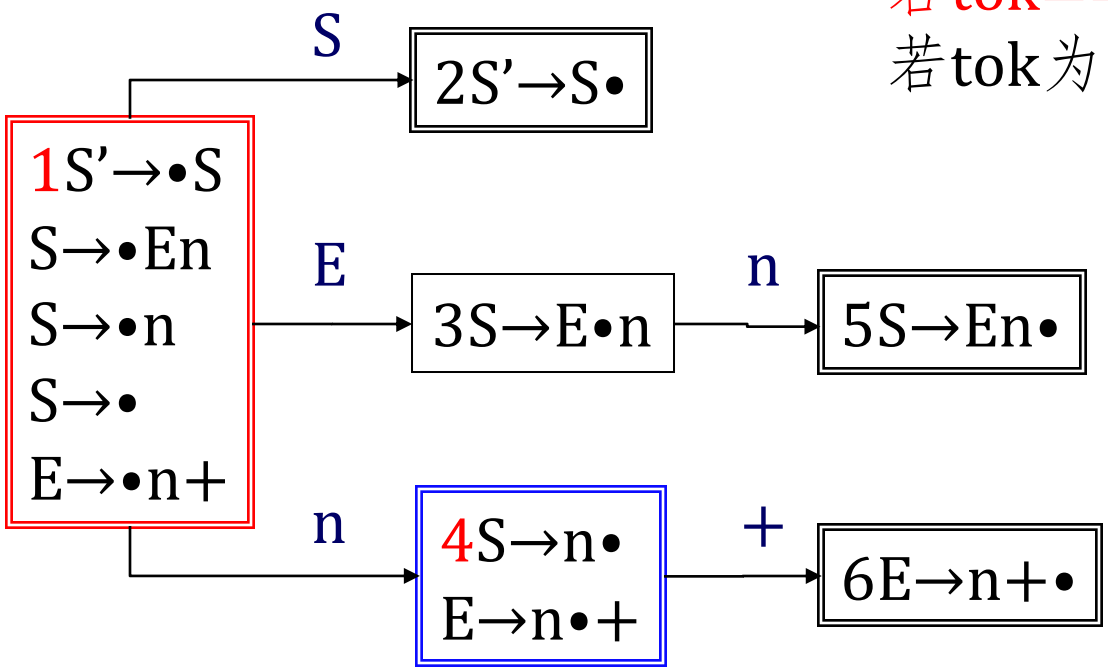
#•n+n#
⇨ #•n+n#　　//移进归约冲突；选移进
⇨ #n•+n#　　//移进归约冲突、归约归约冲突；选移进
⇨ #n+•n#　　//移进归约冲突；选归约$E\to n+$
⇨ #E•n#　　//移进归约冲突；选移进
⇨ #En•#　　//归约归约冲突；选归约$S\to En$
⇨ #S•#　　//分析成功

冲突消解思路

S'→S

S→En|n|ε

E→n+



状态1移进归约冲突。

冲突消解思路：

若tok∈FOLLOW(S)则归约;

若tok==n则移进(推迟消解);

若tok为其它则出错。

状态4移进归约冲突。

消解：

若tok==#则归约;

若tok==+则移进;

若tok为其它则出错。

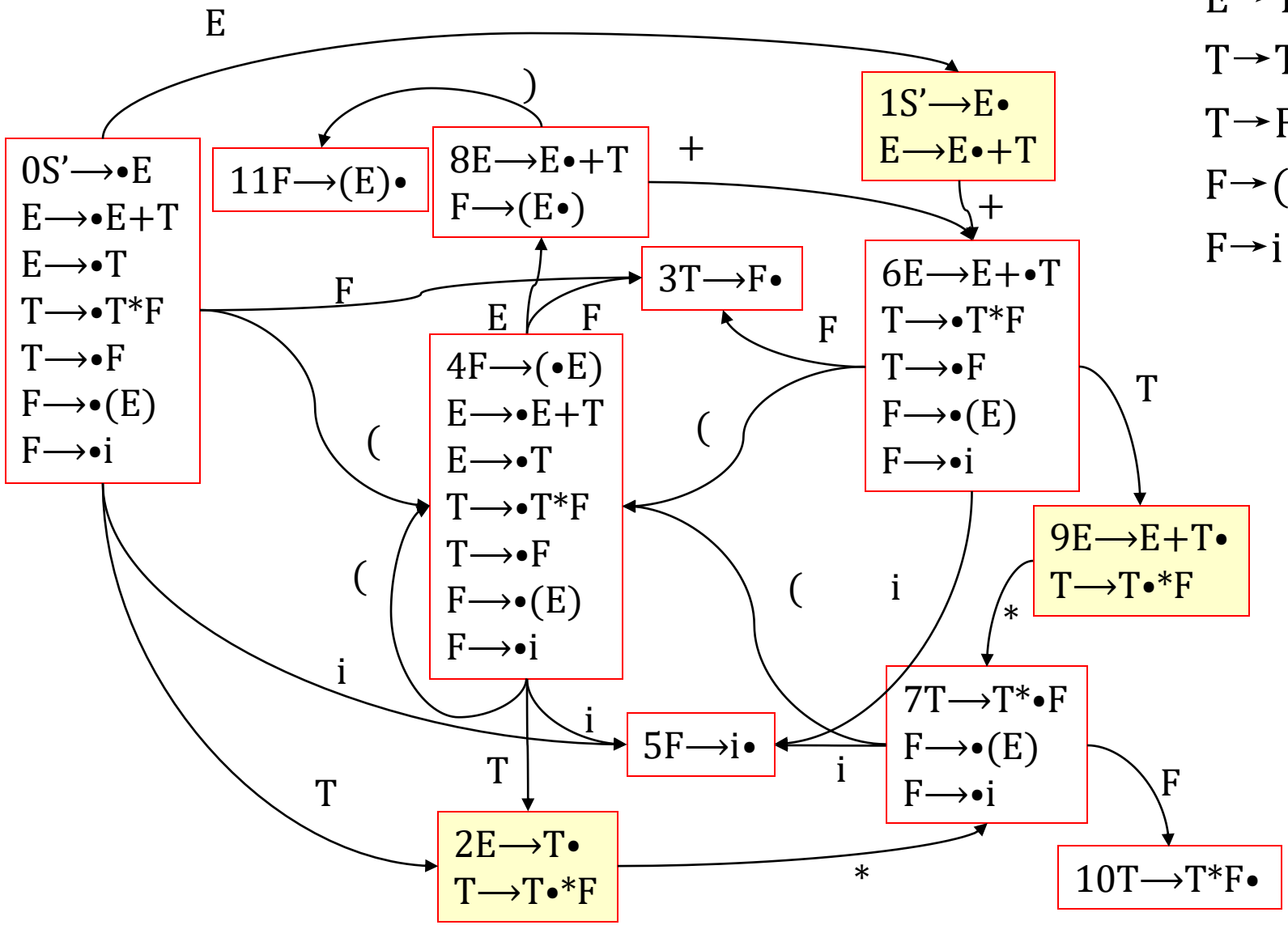
以上都属于SLR(1)消解方法，因此，这种文法是SLR(1)文法。

举例：SLR(1)冲突消解规则

- itemDFA的当前状态包含 [A→γ•]
  - 如果当前输入符号a∈FOLLOW(A)，那么用A→γ归约。
- itemDFA的当前状态包含{X→α•cβ, A→α•}且c∉FOLLOW(A)，
  - 若a=c则移进；
  - 若a∈FOLLOW(A)则用A→α归约；
- itemDFA当前状态包含{X→α•cβ, A→α•, B→α•}且 c∉FOLLOW(A)， c∉FOLLOW(B)， FOLLOW(A)∩FOLLOW(B)=∅，
  - 若a=c则移进；
  - 若a∈FOLLOW(A)则用A→α归约；
  - 若a∈FOLLOW(B)则用B→α归约；
- ……

例：构造SLR(1)分析表

S→E
E→E+T
E→T
T→T*F
T→F
F→(E)
F→i



例：冲突消解与填表

0: S→E
1: E→E+T
2: E→T
3: T→T*F
4: T→F
5: F→(E)
6: F→i

- 规范集0没有冲突，直接填表
- ACTION[0,(]=s5；ACTION[0,i]=s4
- GOTO[0,E]=1；GOTO[0,T]=2；GOTO[0,F]=3
- 规范集1移进-归约冲突
- +∉FOLLOW(S)，ACTION[1,+]=s6，
- FOLLOW(S)={#}，ACTION[1,#]=acc
- 规范集2移进-归约冲突
- FOLLOW(E)={#,+,)}，
  ACITON[2,#]=ACTION[2,+]=ACTION[2,)]=r2；
- ACTION[2,*]=s7
- 规范集3无冲突
- FOLLOW(T)={+,*,),#}，
  ACTION[3,+]=ACTION[3,*]=ACTION[3,)]=ACTION[3,#]=r4

例：冲突消解与填表

0: S→E
1: E→E+T
2: E→T
3: T→T*F
4: T→F
5: F→(E)
6: F→i

- 项目集4没有冲突，直接填表
- ACTION[4,(]=s4；ACTION[4,i]=s5
- GOTO[4,E]=8；GOTO[4,T]=2；GOTO[4,F]=3
- 项目集5无冲突
- FOLLOW(F)={+,*,),#}，
  ACTION[5,+]=ACTION[5,*]=ACTION[5,)]=ACTION[5,#]=r6
- 项目集6无冲突
- ACTION[6,(]=s4；ACTION[6,i]=s5
- GOTO[4,T]=9；GOTO[4,F]=3
- 项目集7无冲突
- ACTION[7,(]=s4；ACTION[7,i]=s5
- GOTO[4,F]=10

例：冲突消解与填表

0: S→E
1: E→E+T
2: E→T
3: T→T*F
4: T→F
5: F→(E)
6: F→i

- 项目集8没有冲突，直接填表
- ACTION[8,)]=s11；ACTION[4,+]=s6
- GOTO[4,E]=8；GOTO[4,T]=2；GOTO[4,F]=3
- 项目集9移进-归约冲突
- FOLLOW(E)={+,),#}， ACTION[9,*]=s7
- ACTION[9,+]=ACTION[9,)]=ACTION[9,#]=r1
- 项目集10无冲突
- FOLLOW(T)={+,*,),#}，ACTION[10,+]=ACTION[10,*]=ACTION[10,)]=ACTION[10,#]=r3
- 项目集11无冲突
- FOLLOW(T)={+,*,),#}，ACTION[11,+]=ACTION[11,*]=ACTION[11,)]=ACTION[11,#]=r5

SLR(1)消解简单冲突的通用规则

- 假定LR(0)规范簇的某个规范集中含有m个移进项目和n个归约项目：
  - $X_i \rightarrow \alpha \bullet c_i \beta_i$, $i=1,...,m$，
  - $A_i \rightarrow \alpha \bullet$, $i=1,...,n$。
- 该规范集满足：
  - 所有移进项目的点右终结符组成的集合C与任意完全项目的左部变元的FOLLOW集都不相交；
  - 任意两个完全项目的左部变元的FOLLOW集都不相交。
- 当前状态为该规范集、当前输入符号为a时，分析过程为：
  - 若$a \in C$那么移进；否则，
  - 若$a \in FOLLOW(A_k)$，$k=1,...,$或n，则用产生式$A_k \rightarrow \alpha$进行归约；否则，
  - 其它情况报错。

如何在SLR(1)中容忍其它冲突

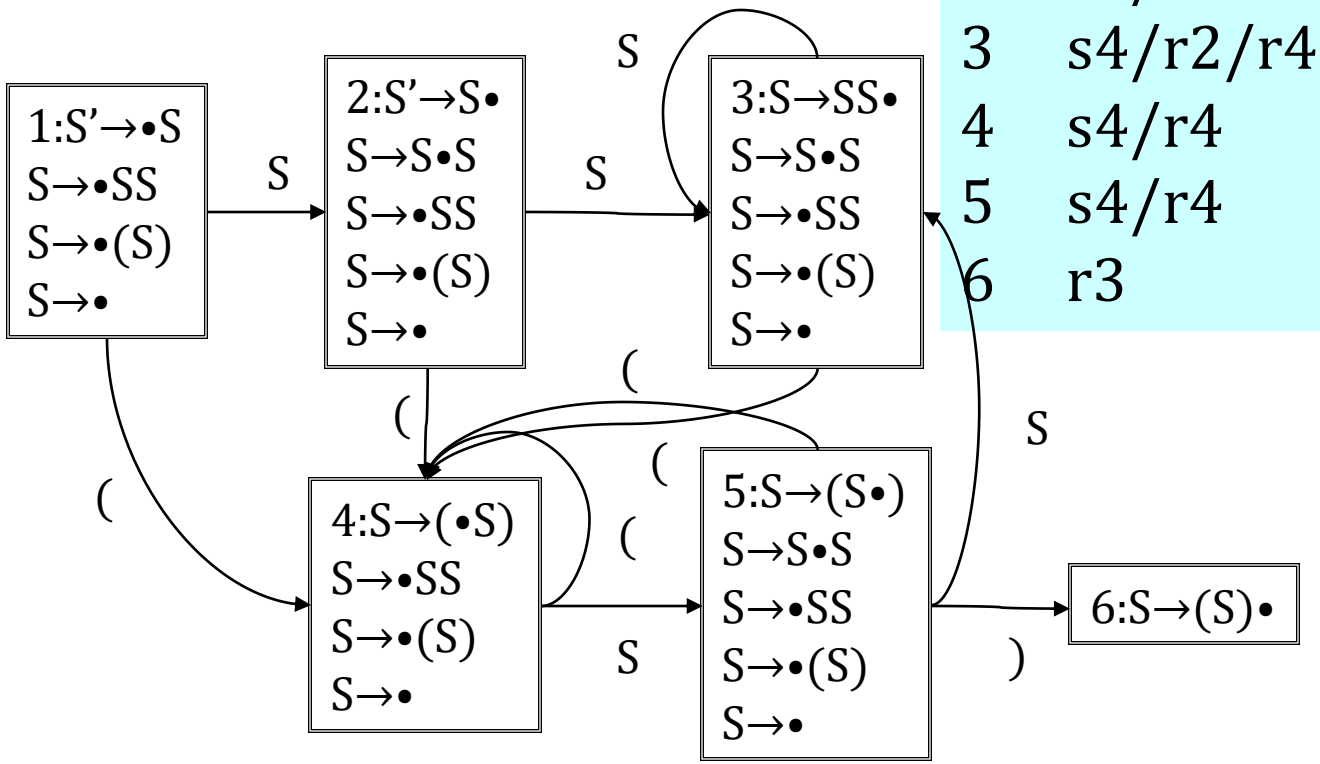
- 定理：任何有二义性的文法都不是LR文法。
- 证明略。

- 二义性文法带来许多好处：
  - 规范簇更小；
  - 表示更简单；
  - 分析过程更简短等。

- 那么，考虑容忍二义性的途径：
  - 让更多的冲突消解规则起作用；
  - 通过修剪文法推迟归约（因为暂时难以解决）。

例：SLR(1)中其它冲突

- S'→S
- S→SS|(S)|ε



	(	)	#	S
1	s4/r4	r4	r4	2
2	s4/r4	r4	acc	3
3	s4/r2/r4	r2/r4	r2/r4	3
4	s4/r4	r4	r4	5
5	s4/r4	s6/r4	r4	3
6	r3	r3	r3	

1:S'→S
2:S→SS
3:S→(S)
4:S→ ε

S'	{(,ε}	{#}
S	{(,ε}	{(,),#}

是文法二义性导致的冲突

1	#	()()#	s4
14	#(	)()#	r4
145	#(S	)()#	s6
1456	#(S)	()#	r3
12	#S	()#	s4
124	#S(	)()#	r4
1245	#S(S	)()#	s6
12456	#S(S)	()#	r3
123	#SS	()#	r2
12	#S	()#	s4
124	#S(	)#	r4
1245	#S(S	)#	s6
12456	#S(S)	#	r3
123	#SS	#	r2
12	#S	#	acc

	(	)	#	S
1	s4/r4	r4	r4	2
2	s4/r4	r4	acc	3
3	s4/r2/r4	r2/r4	r2/r4	3
4	s4/r4	r4	r4	5
5	s4/r4	s6/r4	r4	3
6	r3	r3	r3	

1:S'→S
2:S→SS
3:S→(S)
4:S→ε

为什么S→SS在程序设计语言中少见?

- 靠LR(1)解决，或，
- 文法中避免同一符号并列出现

1	#	(()))#	s4
14	#(	()))#	s4
144	#((	))#	s4
1444	#(((	))#	r4
14445	#(((S	))#	s4
144456	#(((S)	)#	r3
1445	#((S	)#	s6
12456	#((S)	)#	r3
123	#(S	)#	r2

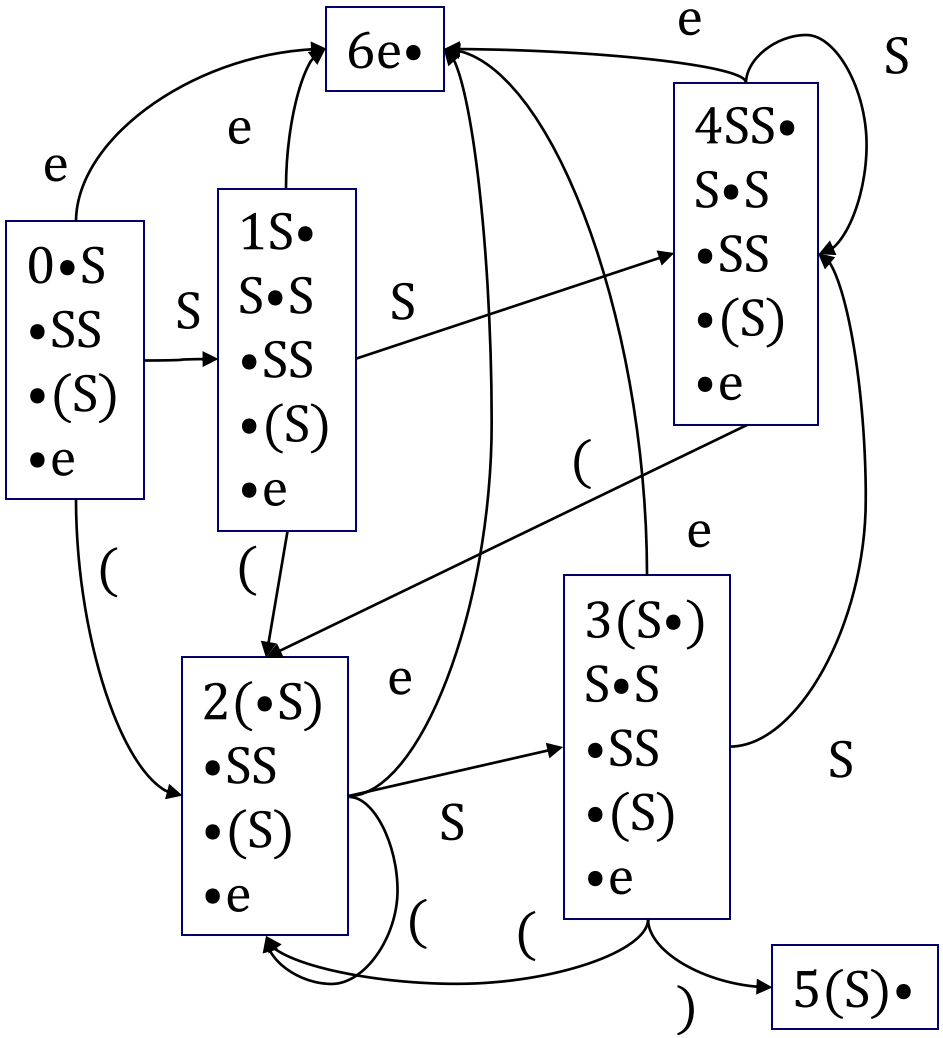
	(	)	#	S
1	s4/r4	r4	r4	2
2	s4/r4	r4	acc	3
3	s4/r2/r4	r2/r4	r2/r4	3
4	s4/r4	r4	r4	5
5	s4/r4	s6/r4	r4	3
6	r3	r3	r3	

1:S'→S
2:S→SS
3:S→(S)
4:S→ε

文法$G_{(e)}$

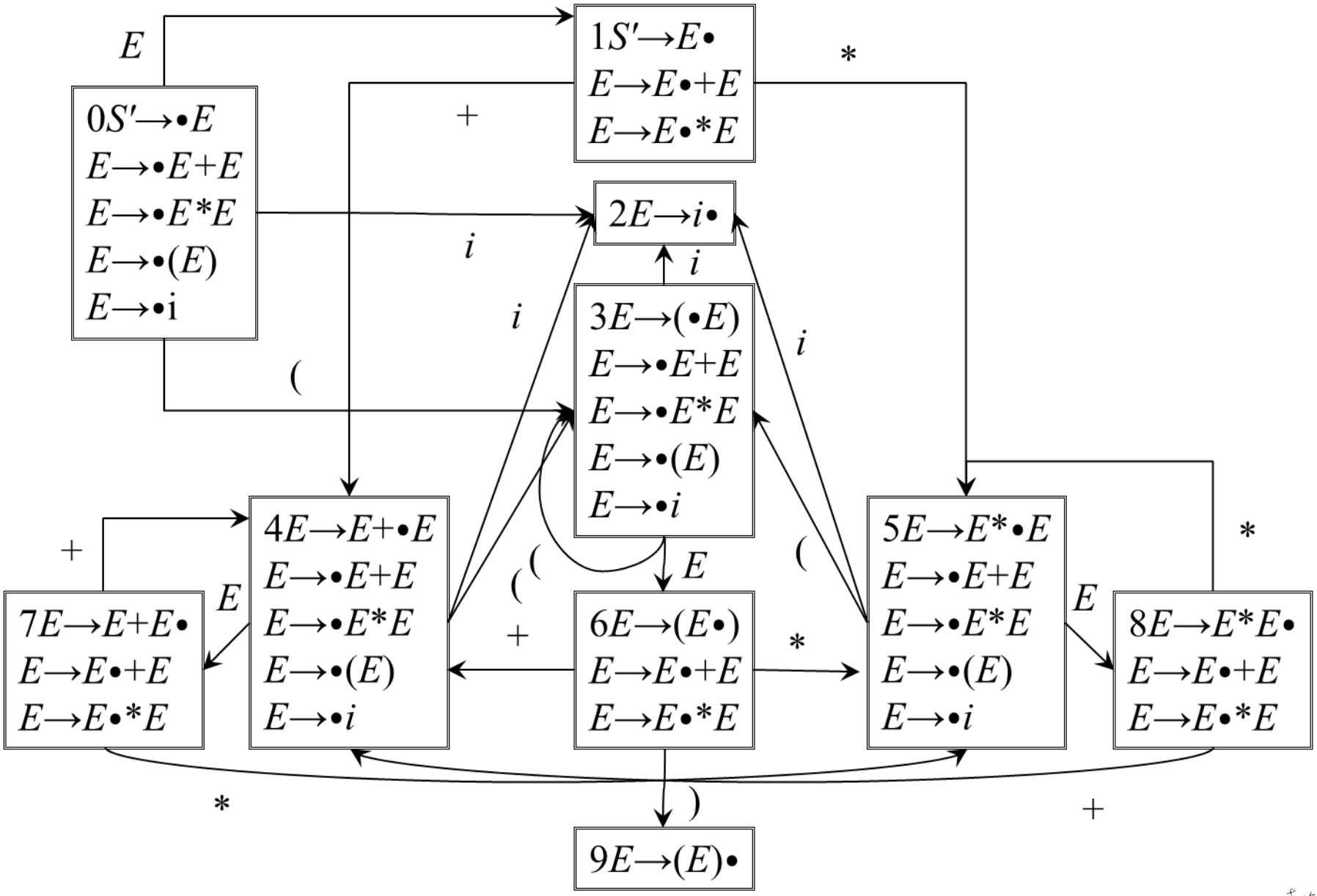
S'→S
S→SS
S→(S)
S→e

	e	(	)	#	S
0	s6	s2			1
1	s6	s2		acc	4
2	s6	s2			3
3	s6	s2	s5		4
4	s6	s2/r2	r2	r2	4
5		r3	r3	r3	
6	r4	r4	r4	r4	



通过计算FOLLOW(S) = {#,), (, e}，虽然SLR(1)默认冲突消解规则失效，但是容易添加额外的规则，就是有关左、右结合性之规则。具体来说，如果消解为归约，就表示句型$\alpha SS \cdot (y$归约为$\alpha S \cdot (y$，这是从左往右的运算次序。如果消解为移进，就表示从句型$\alpha SS \cdot (y$到$\alpha SS(\cdot y$，其中活前缀由αSS变为$\alpha SS($，这是从右往左的运算次序。

例：E⟶E+E|E*E|(E)|i


0S′⟶•E
E⟶•E+E
E⟶•E*E
E⟶•(E)
E⟶•i
1S′⟶E•
E⟶E•+E
E⟶E•*E
2E⟶i•
3E⟶(•E)
E⟶•E+E
E⟶•E*E
E⟶•(E)
E⟶•i
4E⟶E+•E
E⟶•E+E
E⟶•E*E
E⟶•(E)
E⟶•i
5E⟶E*•E
E⟶•E+E
E⟶•E*E
E⟶•(E)
E⟶•i
6E⟶(E•)
E⟶E•+E
E⟶E•*E
7E⟶E+E•
E⟶E•+E
E⟶E•*E
8E⟶E*E•
E⟶E•+E
E⟶E•*E
9E⟶(E)•
E
i
(
+
*
)

利用优先次序规则来消解冲突

- 状态1移进-归约冲突。 用SLR(1)规则来消解。
- E’→E•　　E→E•+E | E•*E
- FOLLOW(E’)={#}， tok为#时归约，为+或*时移进。

- 状态7移进-归约冲突。 用优先次序规则来消解。
- E→E+E• | E•+E | E•*E
- FOLLOW(E)={#,+,*,)}，tok为+、)、#时都归约，为*时移进
- 等价于这个状态中没有第二个项目。第二个项目不起作用
- 状态8移进-归约冲突。 用优先次序规则来消解。
- E→E*E• | E•+E | E•*E
- tok∈FOLLOW(E)时归约。
- 等价于这个状态中没有第二、三个项目。

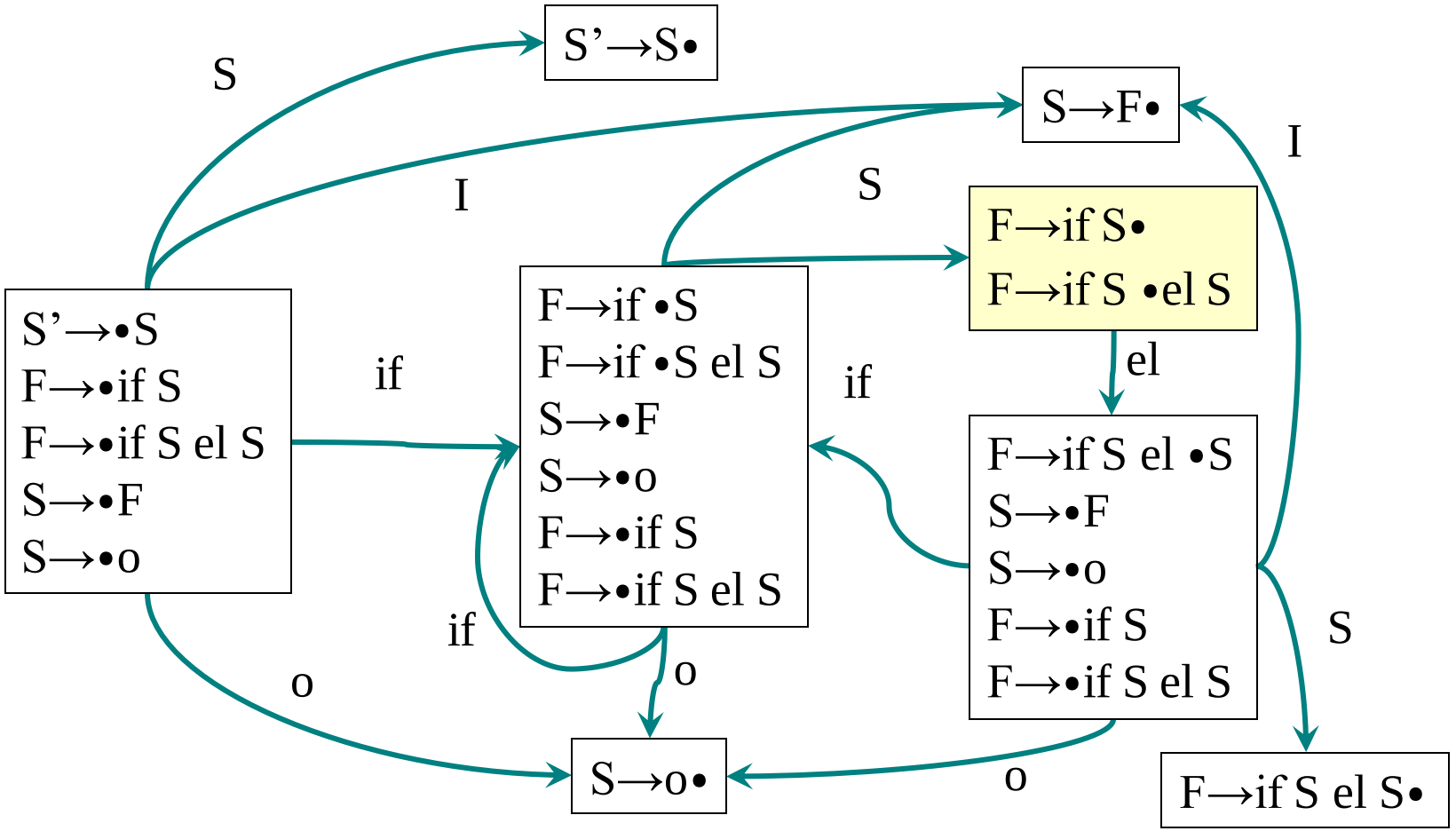
悬挂else二义性文法

- S → F | o
- F → if S | if S el S
- FOLLOW(F)={el,#}

- Š → Š ; S | S
- S → F | o
- F → if S | if S el S
- FOLLOW(F)={el,#,;}

用最近匹配原则来消解悬挂ELSE冲突

- FOLLOW(F)={el,#}
- 在该状态时，若tok为el则移进，为#则归约



其他容忍冲突方法：修剪文法推迟归约

- 遇到移进-归约冲突不能消解时，修剪文法推迟归约，从而使得移进项目和归约项目不同时在一个项目集里。
- 如：$A\rightarrow\alpha N\beta \mid \alpha\gamma\delta$ $N\rightarrow\gamma \mid \xi$
- 活前缀$\alpha\gamma$的有效项目集含有：$N\rightarrow\gamma\bullet$　　和 $A\rightarrow\alpha\gamma\bullet\delta$
- 那么，当$FIRST(\delta)\cap FOLLOW(N)=C\neq\emptyset$时，可能存在移进-归约冲突，具体地，对于$tok\in C$可以移进也可以按$N\rightarrow\gamma$归约，如果$\delta$的首符集含有c的话。

- 修剪文法以消除此类冲突，
- $A\rightarrow\alpha\gamma\beta \mid \alpha\xi\beta \mid \alpha\gamma\delta$
- 此时，针对于活前缀$\alpha\gamma$的冲突不存在了。事实上推后到归约项目$A\rightarrow\alpha\gamma\beta\bullet$和$A\rightarrow\alpha\gamma\delta\bullet$了

SLR(1)局限性

- SLR(1)+冲突消解可处理几乎所有实用语言结构。
- 局限：
  - itemset={X →α•cβ, Y →α•bγ, A →α•, B →α•}
  - 无法满足{c,b}∩FOLLOW(A)∩FOLLOW(B)=∅

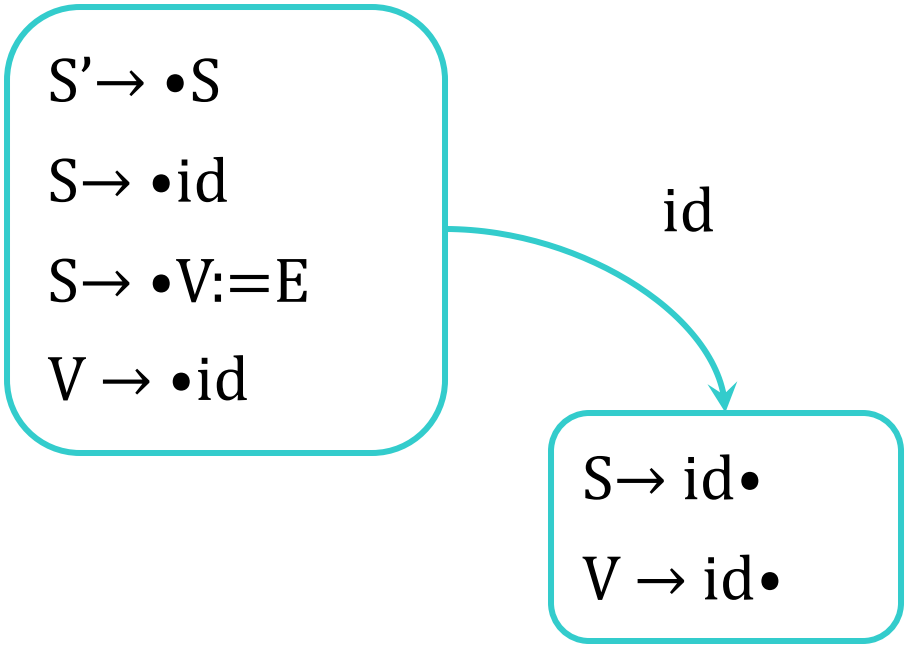
S → id|V:=E

V → id

E → V|n

Follow(S)={#}

Follow(V)={:=,#}



关于SLR(1)的结论

- 能消解的冲突都已经有了消解规则，并已经体现在分析表中
  - SLR(1)默认的冲突及消解；
  - 优先级冲突；悬挂else冲突；运算次序冲突；
  - 文法中没有变元并列的候选式。
- SLR(1)分析表已经消解了所有能消解的冲突。
- 仍然存在不为SLR(1)能消解的冲突，但已经充分了对于PL而言？

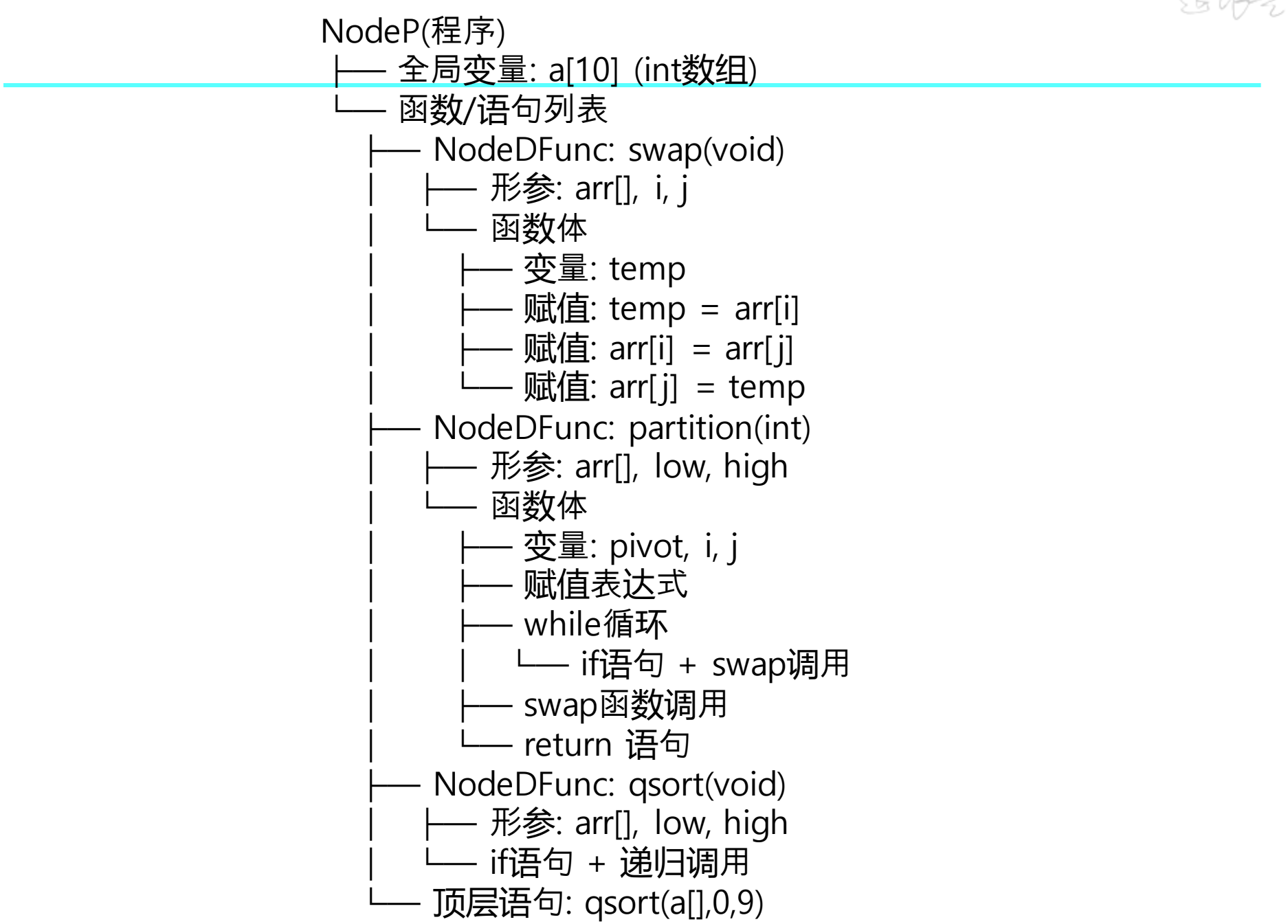
SLR(1)分析表驱动的语法分析

- 输入：分析表，w，双栈空栈。输出：分析成功与否。

```
bi-stack(1,#);
scan();
while(1){
   bi-top(m,c);
   if (ACTION[m,tok]==sn)bi-push(n,tok);
   else if(ACTION[m,tok]==rk){
          bi-pop(body-size(k));
          A=head(k);
          bi-top(m',c); bi-push(GOTO[m',A],A);
    }else if(ACTION[m,tok]==acc)break;
    else error();
}//与LR(0)的没有区别
```

生成QAST

- 根据如下的SLR(1)分析框架（上页）构建语法分析程序。
  - 完成
- 根据以下的AST结点设计利用上述程序实现。
  - 我将严格结合你设计的 AST 节点结构，在之前的双栈 SLR(1) 语法分析程序基础上，完整实现语法分析时自动构建抽象语法树 (AST)，代码可直接编译运行，完全匹配你的文法和节点定义。
  - 完成
- 采用下面的源程序测试你的语法分析程序，给出输出结果。
  - 测试用qsort程序
  - 完成：QAST示意（下页）
- 写出QAST遍历程序输出QTAC代码
  - 可以生成QTAC代码


NodeP(程序)
全局变量: a[10] (int数组)
函数/语句列表
NodeDFunc: swap(void)
形参: arr[], i, j
函数体
变量: temp
赋值: temp = arr[i]
赋值: arr[i] = arr[j]
赋值: arr[j] = temp
NodeDFunc: partition(int)
形参: arr[], low, high
函数体
变量: pivot, i, j
赋值表达式
while循环
if语句 + swap调用
swap函数调用
return 语句
NodeDFunc: qsort(void)
形参: arr[], low, high
if语句 + 递归调用
顶层语句: qsort(a[],0,9)

构建语法制导翻译框架

- SLR(1)分析框架+属性文法
  - 输出符号表
  - 输出QTAC
  - 将在第9章完成

小结

- 规范句型，规范推导，规范归约，句柄，短语，直接短语；
- 活前缀，LR(0)规范簇，规范集；
- 自下而上分析面临的问题；
- itemDFA（识别活前缀的DFA）；
- 移进-归约冲突，归约-归约冲突；
- 冲突消解规则；
- LR(0)分析表，SLR(1)分析表；
- SLR(1) 默认的冲突消解；
- SLR(1)的三种额外冲突消解规则；
- 自下而上分析的算法描述。
- 作业：习题8.1(2)、8.2(1)、8.3、大作业2of2